Zero-Knowledge Proofs

From Foundations to zk-SNARKs and STARKs

Graduate Lecture Notes (MPhil / PhD level)

November 3, 2026

Abstract

These notes give a self-contained, proof-oriented treatment of zero-knowledge proofs. We start from the definitions of interactive proofs, simulation-based security and proofs of knowledge; develop the classical protocols (graph isomorphism, quadratic residuosity, graph 3-colouring) with full proofs; study Σ-protocols and their algebra (AND/OR/equality composition, Pedersen commitments); treat the Fiat–Shamir transform and the forking lemma; and then move on to the modern *succinct* proof systems: sum-check, GKR, arithmetisation (R1CS/QAP), polynomial commitments (KZG, FRI, inner-product arguments), Groth16, PLONK and STARKs. Every algorithm is accompanied by a worked numerical example (small enough to be verified by hand) and by a diagram. Chapters end with exercises. Starred sections are optional on a first reading.

Contents

1 Introduction and Motivation **1**

1.1 The central question: proving without revealing . . . 1

1.1.1 A first example: the colour-blind verifier . . . 2

1.2 The cast: statements, witnesses, provers and verifiers . . . 2

1.2.1 What is being proved: statements and witnesses . . . 2

1.2.2 The parties: prover and verifier . . . 3

1.2.3 Honest and dishonest behaviour . . . 5

1.2.4 What "small probability" means . . . 5

1.3 The three defining properties . . . 6

1.3.1 Proofs of knowledge . . . 7

1.4 Classical proofs versus interactive proofs . . . 8

1.5 A second example: the Ali Baba cave . . . 8

1.6 Why do we care? . . . 11

1.7 A map of the territory . . . 11

1.8 Conventions and glossary . . . 12

2 Mathematical Preliminaries **14**

2.1 Asymptotics, negligible functions and indistinguishability . . . 14

2.2 Languages, relations and NP . . . 15

2.3 Algebraic background . . . 15

2.3.1 Cyclic groups and discrete logarithm assumptions . . . 15

2.3.2 Quadratic residues and the Blum integer setting . . . 16

2.3.3 Pairings . . . 16

2.3.4 Polynomials over finite fields . . . 16

2.4 One-way functions, pseudorandomness and trapdoor permutations . . . 17

2.5 Commitment schemes . . . 17

2.6 The random oracle model . . . 18

3 Interactive Proof Systems **19**

3.1 Definitions . . . 19

3.2 Example: Graph Non-Isomorphism . . . 19

3.3 Public coins and the landscape of IP 20

4 Zero-Knowledge: Definitions **22**

4.1 The simulation paradigm 22

4.2 Basic properties 23

4.3 Witness indistinguishability and proofs of knowledge 23

4.4 Non-interactive variants (preview) 24

5 Classical Zero-Knowledge Protocols **25**

5.1 Graph isomorphism (Goldreich–Micali–Wigderson) 25

5.2 Quadratic residuosity 27

5.3 Fiat–Shamir and Feige–Fiat–Shamir identification 28

6 Zero-Knowledge Proofs for all of NP **29**

6.1 Commitments as "digital envelopes" 29

6.2 3-colouring 29

6.3 Blum's protocol for Hamiltonicity 31

6.4 The main theorem 31

6.5 Round complexity and composition limits 32

7 Σ-Protocols **33**

7.1 Definition 33

7.2 Schnorr's protocol 34

7.2.1 Zero-knowledge against malicious verifiers 35

7.3 A general view: proofs about group homomorphisms 36

7.3.1 Guillou–Quisquater: knowledge of an e-th root 36

7.3.2 Okamoto's protocol: knowledge of a representation 36

7.3.3 Chaum–Pedersen: equality of discrete logarithms 37

7.4 Compound statements: AND, equality, OR 37

7.4.1 AND and equality 37

7.4.2 OR proofs (Cramer–Damgård–Schoenmakers 1994) 37

7.5 From identification to signatures: preview 39

8 Fiat–Shamir, Random Oracles and Non-Interactive Zero-Knowledge **40**

8.1 The Fiat–Shamir transform 40

8.2 Security in the random oracle model 41

8.2.1 The forking lemma 41

8.3 NIZK in the common reference string model 43

8.3.1 The hidden-bits paradigm (Feige–Lapidot–Shamir) 43

8.4 Interactive vs. non-interactive: a summary 44

9 The Sum-Check Protocol and Interactive Proofs for Computation **45**
9.1 Multivariate polynomials and multilinear extensions . . . 45
9.2 The protocol . . . 45
9.3 Application 1: #SAT, so coNP ⊆ IP . . . 47
9.4 Application 2: the proof of IP = PSPACE . . . 47
9.5 Application 3: GKR—delegating computation . . . 48

10 Arithmetisation: From Programs to Polynomial Identities **50**
10.1 Arithmetic circuits and constraint systems . . . 50
10.2 Quadratic Arithmetic Programs . . . 51
10.3 PLONKish arithmetisation . . . 52
10.4 Algebraic Intermediate Representation (AIR) . . . 52

11 Polynomial Commitment Schemes **54**
11.1 KZG commitments (Kate–Zaverucha–Goldberg 2010) . . . 54
11.1.1 The construction . . . 54
11.1.2 Extensions . . . 55
11.2 FRI: a hash-based low-degree test . . . 56
11.2.1 Reed–Solomon codes and proximity . . . 56
11.2.2 Folding . . . 56
11.3 Inner-product arguments and Bulletproofs . . . 57
11.3.1 Pedersen vector commitment . . . 57
11.3.2 The core: proving an inner product in $O(\log n)$ . . . 57
11.3.3 Application: range proofs (Bulletproofs, Bünz et al. 2018) . . . 58

12 SNARKs and STARKs **60**
12.1 Definitions . . . 60
12.2 Groth16 . . . 60
12.2.1 Algorithm . . . 60
12.2.2 Correctness . . . 61
12.2.3 Zero-knowledge and soundness . . . 62
12.3 PLONK . . . 62
12.3.1 Setup of the argument . . . 62
12.3.2 Copy constraints: the permutation argument . . . 63
12.3.3 Putting it together . . . 63
12.4 STARKs . . . 64
12.5 Comparison . . . 65
12.6 Recursion and folding . . . 66

13 Complexity-Theoretic Foundations and Limits **67**

13.1 Classes of zero-knowledge languages . . . 67
13.2 Sahai–Vadhan problems: the SD problem . . . 68
13.3 Impossibility results . . . 68
13.4 PCPs, IOPs and the origins of succinctness . . . 69
13.5 Quantum considerations . . . 69

14 Applications, Practical Considerations and Further Reading **71**
14.1 Applications . . . 71
14.2 Choosing a proof system . . . 72
14.3 Implementation pitfalls (a checklist) . . . 72
14.4 Where to go from here . . . 72

A Notation and Toy-Parameter Reference **74**
A.1 Notation . . . 74
A.2 Toy group and fields used in the examples . . . 74

B Hints and Answers to Selected Exercises **75**

Chapter 1

Introduction and Motivation

How to read this chapter

This chapter assumes *no* prior knowledge of cryptography or complexity theory. We introduce every character and every word in plain language first, and then give the precise definition. Everything defined here is stated again, with full rigour, in later chapters; here the goal is that you understand *what* the definitions say and *why* they are written the way they are. If a symbol looks scary, skip to the sentence in words just before it.

1.1 The central question: proving without revealing

Suppose you want to convince somebody that something is true. The usual way is to *show your work*.

- To prove that 91 is not prime, you write $91 = 7 \times 13$. The reader checks the multiplication and is convinced.
- To prove that you know the password of an account, you type the password. The server compares it with its records.
- To prove that a Sudoku puzzle has a solution, you hand over a completed grid. The reader checks every row, column and box.

In each case the proof *also teaches the reader something extra*: the factorisation, the password, the solution. Worse, the reader can now reuse that knowledge, for example by logging in as you, or by showing the Sudoku solution to someone else and claiming to have solved it.

> *Is it possible to convince someone that a statement is true* without giving them anything else at all*?*

In 1985 Goldwasser, Micali and Rackoff showed that the answer is *yes*. Such a convincing-but-silent argument is called a **zero-knowledge proof**. The price is that we must relax what we mean by "proof" in two ways: the proof becomes **interactive** (the two sides exchange several messages instead of one written document), and **probabilistic** (the checker may be fooled, but only with astronomically small probability). In this chapter we explain precisely what that means.

1.1.1 A first example: the colour-blind verifier

Here is the simplest story that shows the main idea. Victor is colour-blind. He holds two balls, one red and one green. To him they look identical. Peggy has normal colour vision, and she claims the balls have different colours. How can she convince Victor, *without telling him which ball is which colour*?

1. Victor holds one ball in each hand and shows them to Peggy.
2. Peggy turns away (or closes her eyes). Victor flips a fair coin. If it lands heads he swaps the balls behind his back; if tails he does nothing.
3. Peggy looks again and says "swapped" or "not swapped".
4. Victor checks her answer against his coin. If it is wrong, he rejects.

If the balls really differ in colour, Peggy always answers correctly, so Victor is never rejected. **If the balls are really identical,** Peggy sees no difference between the two situations and can only guess, so she is right with probability $\frac{1}{2}$. Repeating the game 20 times, a bluffing Peggy survives with probability $2^{-20} < 10^{-6}$, less than one in a million. And Victor learns nothing about the colours themselves (he never finds out which ball is the red one), only that the colours differ.

Remark 1.1. This story has an unusual feature. Peggy's "secret" is not a string of symbols but an *ability* (colour vision). In the rest of the notes the secret will always be a piece of data called a *witness*, and we will see how to define it precisely in Section 1.2.1.

1.2 The cast: statements, witnesses, provers and verifiers

We now give names to the ideas in the story above. Each term is introduced in words first, then formally.

1.2.1 What is being proved: statements and witnesses

A **statement** (also called an **instance**) is a claim that is either true or false, such as "this graph can be coloured with three colours", "the number N has a factor between 2 and $\sqrt{N}$", or "the person owning this public key knows the matching private key". A **witness** is a piece of private information that makes it easy to *check* that a statement is true. For "$N = 91$ is composite" a witness is the factor 7, because checking $91 \bmod 7 = 0$ takes one line.

We encode statements and witnesses as bit strings, and collect all the valid (statement, witness) pairs into one object.

Definition 1.2 (Relation, witness, language)**.** A *relation* R is a set of pairs (x, w) of bit strings. We read $(x, w) \in R$ as "w is a witness for the statement x". We require that membership in R can be tested efficiently, meaning in time polynomial in the length of x. The *language* of R is the set of true statements

$$L_R \;=\; \{\, x \;:\; \text{there exists } w \text{ with } (x, w) \in R \,\}.$$

We say "$x \in L_R$" as a short way of saying "the statement x is true".

Example 1.3 (Composite numbers)**.** Let $R_{\text{comp}} = \{(N, p) : 1 < p < N \text{ and } p \text{ divides } N\}$. Then $91 \in L_{R_{\text{comp}}}$ because $(91, 7) \in R_{\text{comp}}$, while the prime 97 is not in $L_{R_{\text{comp}}}$ because no p works.

Example 1.4 (Graph 3-colouring)**.** A statement x is a graph G. A witness w assigns one of three colours to each vertex. $(G, w) \in R_{\text{3COL}}$ means that no edge of G has both endpoints of the same colour. Checking this takes time proportional to the number of edges.

Example 1.5 (Knowing a secret key)**.** Fix a public number g and a modulus p. A statement is a number y. A witness is a secret exponent s with $g^s \bmod p = y$. This is the setting of the Schnorr identification protocol (Chapter 7): your public key is y, your private key is s, and "proving who you are" means proving that you know s.

Remark 1.6. Relations of this type, whose witnesses can be checked quickly, are exactly the problems in the class NP. We study this class in Chapter 2. Note the important asymmetry: *finding* a witness may be very hard (factoring!), but *checking* one is easy.

1.2.2 The parties: prover and verifier

Definition 1.7 (Algorithm, probabilistic algorithm)**.** An *algorithm* is a step-by-step procedure that a computer can execute. It is *probabilistic* (or *randomised*) if it may also flip fair coins during its execution, so that its output is a random variable and not a fixed value. We write $r \xleftarrow{\$} S$ to say that r is chosen uniformly at random from the finite set S.

Definition 1.8 (Efficient algorithm, PPT)**.** An algorithm is *efficient* if its running time is bounded by a polynomial in the length of its input (so doubling the input does not make the running time explode). An efficient probabilistic algorithm is called *probabilistic polynomial-time*, abbreviated PPT.

Definition 1.9 (Prover)**.** The *prover* P is the party who wants to convince the other party that a statement x is true. It receives as input the statement x (shared with the verifier) *and* a private witness w for x. The prover is probabilistic and sends messages to the verifier according to a fixed program.

Definition 1.10 (Verifier)**.** The *verifier* V is the party who must decide whether to believe the statement. It receives only the statement x; it never receives the witness. The verifier is a PPT algorithm. At the end of the interaction it outputs a single bit: 1 (*accept*) or 0 (*reject*).

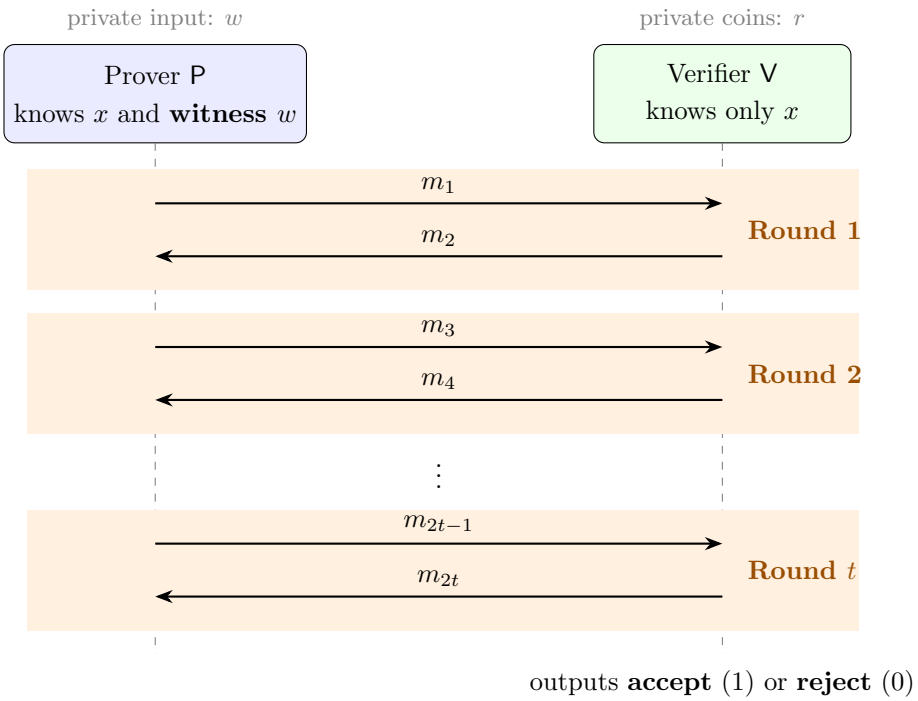


Figure: a general interactive protocol. One *round* is a message from the prover together with the verifier's reply (shaded). The number of rounds t is not fixed: it may be one, a handful, or a number that depends on the statement. Who speaks first also varies from protocol to protocol. The picture shows only the pattern. After the last round the verifier outputs its decision.

Definition 1.11 (Interactive protocol, round, transcript)**.** An *interactive protocol* is a pair (P, V) of programs that take turns sending messages to each other, starting from the common input x. Each message sent is one *move*; a prover message together with the verifier's reply is called a *round*. A protocol with an odd number of moves ends with a prover message that gets no reply, so it consists of a number of rounds plus a "half round"; for example, the commit–challenge–response protocols below have *three moves*. To avoid confusion, we call one complete run of such a short protocol a *repetition*, and we speak of repeating it k times. The verifier's final check is a private computation, not a message. The list of all messages exchanged, $(m_1, m_2, \ldots, m_n)$, is the *transcript*. We write

$$\langle \mathsf{P}(w), \mathsf{V} \rangle (x) \in \{0, 1\}$$

for the (random) output bit of V when P holds the witness w and both parties hold the statement x.

Definition 1.12 (Challenge, public-coin and private-coin protocols)**.** A *challenge* is a random message that the verifier sends to the prover (for example "come out via path A or B" in the cave below). A protocol is *public-coin* if every message of the verifier is simply the outcome of its coin tosses, sent to the prover exactly as tossed. The verifier has *no hidden randomness*: the prover sees the coins as soon as they are used. A protocol is *private-coin* if the verifier keeps some of its random choices secret and uses them later, for example to check an answer.

Example 1.13 (Public-coin versus private-coin)**. Cave: public-coin.** Victor flips a coin and shouts the result ("A" or "B"). Peggy hears the coin, so nothing is hidden. **Colour-blind game: private-coin.** Victor's swap bit must stay secret. If Peggy knew whether he swapped the balls, she could "answer" correctly without being able to tell the colours apart, and the protocol would prove nothing.

Remark 1.14 (What "public" does *not* mean). It does not mean that someone proves the coins were fair, and it does not involve any trusted authority or outside source of randomness. Each verifier simply tosses its own coins and shows the results to the prover. Public-coin protocols are the easiest to analyse, and in Chapter 8 we will see that they can be made non-interactive by replacing the verifier's coins with the output of a hash function.

What if the verifier is not honest?

"Public-coin" describes the *honest* verifier's program. It does not promise that a real verifier follows it, so we must check each property against the right kind of verifier.

- **Soundness: the verifier is honest by definition.** Soundness protects the verifier, so a verifier that rigged its own challenges would only weaken its own guarantee. The cheating party here is the *prover*, who must not be able to predict the verifier's coins.
- **Zero-knowledge: the verifier may cheat.** A cheating verifier V^* need not toss fair coins. It may always ask for B, or choose each challenge cleverly from what it has seen so far, in order to squeeze out information. That is exactly why Definition 1.21 requires a simulator for *every* efficient V^*, and not just for the honest one. A protocol that is zero-knowledge only against the honest verifier is weaker; it is called *honest-verifier zero-knowledge* and reappears with Σ-protocols in Chapter 7.

1.2.3 Honest and dishonest behaviour

A security definition has to say what can go wrong. We therefore distinguish parties that follow the rules from those that do not.

Definition 1.15 (Honest, cheating and malicious parties)**.** A party is *honest* if it follows the protocol exactly. A *cheating prover* P^* is any program that tries to make V accept a **false** statement; it does not have to follow the protocol and may deviate arbitrarily. A *cheating verifier* V^* (also *malicious*) is any efficient program that interacts with the honest prover and tries to extract extra information, again deviating arbitrarily from the specification. The starred symbols $\mathsf{P}^*, \mathsf{V}^*$ always mean "arbitrary, possibly dishonest".

Two different strengths of cheating prover are used.

Definition 1.16 (Proof and argument)**.** If soundness (defined below) holds against *every* cheating prover, even one with unlimited computing power, the protocol is called a *proof*. If it only holds against cheating provers that run in polynomial time, it is called an *argument*. Arguments are weaker in theory but are what is used in practice, because they permit much shorter and faster protocols.

1.2.4 What "small probability" means

Definition 1.17 (Negligible function)**.** Let λ be a *security parameter*, a positive integer that measures how much security we want (think "number of repetitions", or "key length in bits"). A function $\epsilon(\lambda)$ is *negligible*, written $\epsilon = \mathsf{negl}(\lambda)$, if it eventually becomes smaller than $1/\lambda^c$ for every constant c.

The standard example is $\epsilon(\lambda) = 2^{-\lambda}$. For $\lambda = 128$ this is about 3×10^{-39}: smaller than the probability of randomly guessing a particular grain of sand among all those on Earth many times over. We say an event happens with *overwhelming probability* if the probability that it fails is negligible. Chapter 2 contains the careful treatment.

1.3 The three defining properties

A zero-knowledge proof for a relation R is an interactive protocol (P, V) satisfying three properties. Each one protects someone different, and it is a good habit to always ask "*who is this property protecting, and against whom?*"

Definition 1.18 (Completeness)**.** *If the statement is true and both parties are honest, the verifier accepts* (except with negligible probability c). Formally, for every $(x, w) \in R$,

$$\Pr\big[\,\langle \mathsf{P}(w), \mathsf{V}\rangle\,(x) = 1\big] \;\geq\; 1 - c.$$

Protects the honest prover: a truthful prover is not wrongly rejected. Without it, the protocol could simply reject everyone.

Remark 1.19 (Is the completeness error c ever nonzero?). In almost all protocols in these notes (the colour-blind game, the cave, graph isomorphism, Schnorr) an honest prover is *never* rejected, so $c = 0$. This is called *perfect completeness*. The general definition keeps a tiny error c only so that it also covers protocols in which an honest prover may occasionally abort, for example because a random value it picked falls outside an allowed range; some lattice-based protocols behave this way. Note the contrast with soundness: the completeness error can be removed by careful design, whereas the soundness error s comes from a cheater guessing the verifier's challenge. In a short protocol with random challenges it is therefore always positive, and we can only make it negligible by repeating the protocol.

Do not confuse the two errors. $c = 0$ does **not** mean that the verifier is always sure when it accepts. It only says that a *true* statement proved by an honest prover is never rejected. The doubt about a *false* statement is measured by s. The result is a one-sided guarantee: if the verifier **rejects**, it is certain that something is wrong (the statement is false or the prover deviated), whereas if it **accepts**, a small doubt of at most s remains.

Definition 1.20 (Soundness)**.** *If the statement is false, no prover, however clever, can make the verifier accept* (except with small probability s). Formally, for every $x \notin L_R$ and *every* cheating prover P^*,

$$\Pr\big[\,\langle \mathsf{P}^*, \mathsf{V}\rangle\,(x) = 1\big] \;\leq\; s.$$

Protects the verifier against liars. Without it, the protocol could simply accept everyone. The number s is the *soundness error*; typical values are $s = \frac{1}{2}$ for a single repetition and $s = 2^{-k}$ after k independent repetitions.

Definition 1.21 (Zero-knowledge, first look)**.** *Whatever a verifier can compute after talking to the honest prover, it could have computed on its own, without the interaction.* The precise form is as follows. The *view* of a verifier V^* is everything it sees during the protocol: its input, its random coins and all messages it receives, written $\mathsf{View}_{\mathsf{V}^*}^{\mathsf{P}(w)}(x)$. The protocol is *zero-knowledge* if for every efficient cheating verifier V^* there is an efficient algorithm Sim, the *simulator*, that is given only the statement x (no witness!) and outputs something that is *indistinguishable* from the real view:

$$\mathsf{View}_{\mathsf{V}^*}^{\mathsf{P}(w)}(x) \;\approx\; \mathsf{Sim}(x) \qquad \text{for all } (x, w) \in R.$$

Protects the prover's secret. The words "indistinguishable" (written $\approx$) come in three strengths: *perfect* means the two distributions are identical; *statistical* means they differ by a negligible amount; *computational* means no efficient test can tell them apart, except with negligible advantage. Definition 4.2 in Chapter 4 states all of this with every quantifier written out.

Property	Statement is...	Protects...	Against...
Completeness	true	the honest prover	bad luck / protocol bugs
Soundness	false	the verifier	cheating provers P^*
Zero-knowledge	true	the prover's witness	curious or malicious verifiers V^*

Why a simulator captures "learning nothing"

Whatever the verifier learns must come from its *record* of the interaction (its view). Suppose you can show that a forger with no access to the secret could have written down an indistinguishable record. Then the record cannot contain any information about the secret: if it did, the forger could not have reproduced it. The simulator is that forger. It does not need to behave like the real prover; it just needs to output records that look exactly like real ones. The verifier's view is therefore "worthless" in the sense that anyone could have manufactured it alone. (The cave example in Section 1.5 carries this out in detail.)

Common misconceptions

- **"Zero-knowledge means the prover reveals nothing."** Not quite: the verifier learns that the statement is *true*. It learns nothing *beyond* that.
- **"Zero-knowledge is encryption."** No. Nothing is hidden by a key; the secret simply never leaves the prover.
- **"The verifier is certain after the protocol."** Only up to the soundness error s, which can be made as small as we like but never exactly zero for a short interactive protocol. (This does not contradict perfect completeness $c = 0$: completeness says that a *true* statement is never rejected, while s measures the chance of accepting a *false* one.)
- **"The verifier can show the transcript to a friend as evidence."** No: since a simulator can fabricate transcripts, a recording proves nothing to a third party. This is a feature, not a bug (it gives *deniability*).

1.3.1 Proofs of knowledge

There is a subtle difference between "*x is true*" and "*I know a witness for x*". For example, the statement "there is a private key matching this public key" is always true, so proving it is useless; what we need is a proof that the prover *actually possesses* the key. This is formalised through a second algorithm.

Definition 1.22 (Proof of knowledge, first look)**.** A protocol is a *proof of knowledge* for R if there is an algorithm Ext, the *extractor*, with the following property. Whenever some prover P^* makes V accept x with noticeable probability, Ext, given the ability to interact with P^* (including restarting it), outputs a witness w with $(x, w) \in R$. In words: *anyone who can convince the verifier must really know a witness*, because we can pull it out of them.

The extractor is the mirror image of the simulator: the simulator shows that the verifier learns no secret, the extractor shows that the prover must have one. Definition 4.6 (Chapter 4) gives the full definition.

1.4 Classical proofs versus interactive proofs

	Classical (written) proof	**Interactive zero-knowledge proof**
Format	one static message (the witness)	several messages, with random challenges
Verifier's confidence	absolute (if it checks)	overwhelming but probabilistic
What is revealed	the whole witness	only that the statement is true
Transferable to others?	yes, anyone can re-check it	no, only the verifier was convinced
Verifier's work	check the full witness	check a few random responses

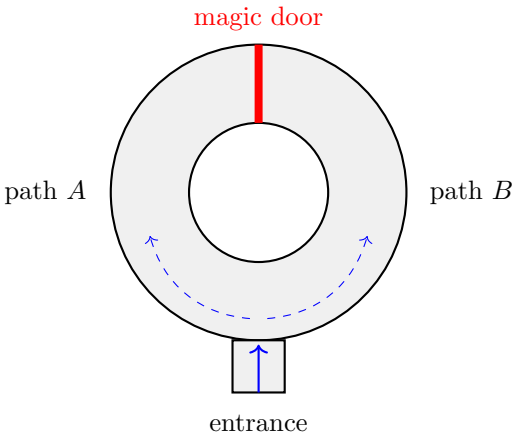
Proposition 1.23 (Why randomness and interaction are both needed (preview))**.** *If the verifier is deterministic, or if there is no interaction, then the only statements that can be proved are those in* NP*, and the proof must essentially contain the witness itself.*

Why: a deterministic verifier's questions are predictable, so the prover can simply write down all the answers in advance and the "interaction" becomes a single document. A precise statement and proof appear in Chapter 3. The surprising fact is that randomness *and* interaction together allow us to prove far more than NP.

1.5 A second example: the Ali Baba cave

We now walk through a classic story from start to finish, using all of the vocabulary above. It was popularised by Quisquater and co-authors.

Setting. A circular cave has one entrance. Deep inside, the corridor splits into two branches, path A and path B, which meet again behind a *magic door* that opens only when you say a secret word. Peggy (the prover) claims she knows the word. Victor (the verifier) wants to be sure, but Peggy does not want to reveal the word. Here the statement is "Peggy knows the secret word" and the witness is the word itself.



Protocol 1.1: Ali Baba cave (one repetition)

1. **Commit** (move 1, Peggy → Victor). Peggy walks into the cave and chooses path A or B by a fair coin. Victor waits at the entrance and cannot see which she took, but she can no longer change her mind: she has *committed*.
2. **Challenge** (move 2, Victor → Peggy). Victor walks to the fork and shouts a random side, "come out via A" or "come out via B".
3. **Response** (move 3, Peggy → Victor). Peggy walks out of the cave by the requested side, opening the magic door with the secret word if necessary.
4. **Check** (no message). Victor privately checks that Peggy emerged from the requested side, and accepts or rejects.

Moves 1 and 2 form one round (a prover message and the verifier's reply); move 3 is a final prover message. The whole protocol is therefore a *three-move* protocol, and it is run k times in a row.

Completeness. If Peggy knows the word she can reach either side whatever Victor asks, so Victor accepts in every repetition.

Soundness. Suppose Peggy does *not* know the word. She went in via some side, say A. If Victor asks for A, she just walks out. If Victor asks for B, she is stuck behind the door and fails. Because Victor's coin is unpredictable, she survives a single repetition with probability exactly $\frac{1}{2}$. After k independent repetitions she survives with probability 2^{-k}:

repetitions k	1	10	20	40
cheater survives	50%	$\approx 0.1\%$	$\approx 10^{-6}$	$\approx 10^{-12}$

Here the soundness error is $s = 2^{-k}$, and by choosing $k = \lambda$ it becomes negligible in the security parameter.

Zero-knowledge. Whatever Victor learns must come from what he *sees*. In one repetition, the *record* of everything he saw is very short: "I shouted b, and Peggy came out of side b". (Its technical name is the *transcript*, or Victor's *view*.) He never sees the secret word, he never sees the door open, and he never learns whether Peggy used the door or simply walked back the way she came.

To show that this record contains no information about the word, we ask a question: *could someone who does not know the word write down an identical-looking record entirely on their own?* In the cave, yes. Let *Sam* be a fraudster who does not know the word and is not even in the cave. To produce one repetition of the record, Sam flips a fair coin to get $b \in \{A, B\}$ and writes

"challenge b; Peggy came out of side b".

Now compare with a real record. There, Victor's challenge b is a fair coin and an honest Peggy always comes out of side b, so the real record has *exactly* the same distribution as Sam's. For three repetitions, a real record might read $(A, A), (B, B), (B, B)$ and Sam's might read $(B, B), (A, A), (B, B)$; nobody can tell which one came from the real protocol. Hence the

record cannot contain the word. (Compare: had Peggy shouted the word aloud, the real record would contain it, and Sam could not have written it down.) Sam plays the role of the *simulator*: an algorithm that never sees the witness yet outputs records distributed exactly like real ones. Because the distributions are identical, this is *perfect* zero-knowledge.

But then why is Victor convinced? Victor is convinced because he *was there*: he chose his challenge *after* Peggy had gone in, and she could not predict it. Sam's fake record has no such history; it is just ink on paper. So the record convinces Victor, but it convinces nobody else, since anyone could have written it. This is the last bullet of the warning above.

Cheating verifiers, and where rewinding comes in. A cheating verifier V^* may choose its challenge in a clever, non-random way. This does not break the argument above: Sam runs V^* as a program, sees which challenge b^* it announces, and writes (b^*, b^*). In the cave this works without any trick because Victor cannot see which side Peggy entered, so his challenge cannot depend on it. In many other protocols the prover's first message *is* visible to the verifier. Then the simulator must produce that first message before it knows the challenge, and it cannot do this honestly. The standard trick is to *guess* the challenge in advance and prepare a first message that works only for that guess (in the cave: Sam enters via the guessed side A). Sam then runs V^*. If V^* announces the guessed challenge, Sam completes the repetition and keeps it. Otherwise Sam *rewinds*: he restarts V^* from the beginning and guesses again. Since the first message hides the guess, each attempt succeeds with probability $\frac{1}{2}$, so on average two attempts per repetition suffice. We will use rewinding in later chapters.

The three ideas behind every ZK protocol

(1) Commit–challenge–response. The prover commits to something random (a side of the cave); the verifier asks one of several questions; the prover can answer *every* possible question only if she knows the witness, but any *single* answer is independent of the witness.
(2) Special soundness. If a prover could answer *two different* questions about the *same* commitment, then putting the two answers together reveals the witness. (In the cave: someone who could exit through both A and B after entering via A must have passed through the door.) Hence a prover who can answer everything must "know" the witness.
(3) Simulation by rewinding or programming. A simulator that knows the question in advance (or can try again) can fabricate a convincing transcript without the witness.

Remark 1.24 (The same skeleton again and again). Most protocols in this book have the shape of the generic three-move template below. Whenever you meet a new protocol, ask: *what is the commitment, what is the challenge, what is the response, and what does the verifier check?*

Protocol 1.2: Generic three-move template ("Σ-protocol" shape)

Prover $\mathsf{P}(x, w)$		**Verifier** $\mathsf{V}(x)$
choose fresh randomness; compute commitment a	$\xrightarrow{a}$	
	$\xleftarrow{e}$	pick random challenge e
compute response z from (x, w, a, e)	$\xrightarrow{z}$	check that (a, e, z) is valid for x; accept or reject

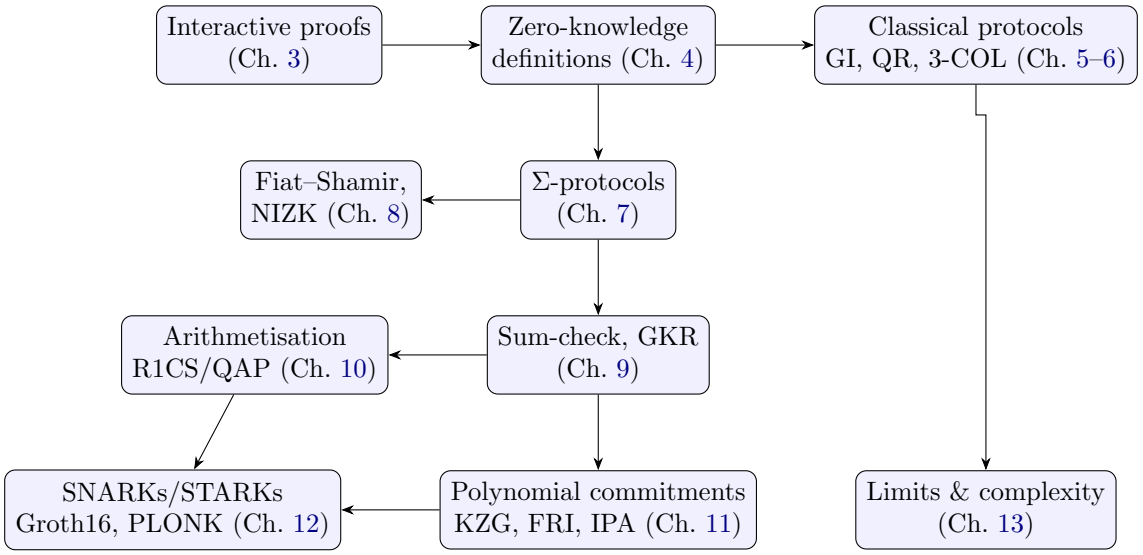
1.6 Why do we care?

Zero-knowledge is not only a curiosity. It lets two parties who do not trust each other cooperate. Typical uses:

- **Authentication.** Prove that you know a password or secret key without sending it, so a hacked server learns nothing reusable (Schnorr and Fiat–Shamir identification).
- **Digital signatures.** The Fiat–Shamir transform converts an interactive identification protocol into a signature scheme. Many real signatures are built this way.
- **Privacy-preserving credentials.** Prove "I am over 18" or "I live in this country" without revealing your birth date or address.
- **Designing secure protocols.** The GMW compiler turns any protocol that is secure against *honest-but-curious* parties into one that is secure against *malicious* parties, by having each party prove in zero-knowledge that it followed the rules.
- **Blockchains.** Zcash hides who paid whom while still letting the network verify that no coins were created from nothing. Zk-rollups let a network check many thousands of transactions by verifying one short proof.
- **Verifiable computation.** Outsource a heavy computation to a server and receive a short proof that the answer is correct, which is much cheaper to check than redoing the work.
- **Complexity theory.** The study of interactive proofs led to the theorems $\mathsf{IP} = \mathsf{PSPACE}$ and the PCP theorem, and to results on the hardness of approximation.

1.7 A map of the territory

The rest of the notes build upwards from the definitions of this chapter. Arrows show which chapters lead into which.



Suggested path for a first reading: Chapter 2 (only the parts you need), then Chapters 3, 4, 5 and 7. The later chapters on SNARKs and STARKs can be read after that.

1.8 Conventions and glossary

Throughout: λ is the security parameter and 1^λ is its unary encoding; $x \xleftarrow{\$} S$ means sampling uniformly from the finite set S; $[n] = \{1, \ldots, n\}$. For a finite group $\mathbb{G}$ of prime order q with generator g we write exponents in $\mathbb{Z}_q$. All algorithms are probabilistic, and "PPT" means probabilistic polynomial time in λ (or in $|x|$ when x is the input).

Term	Symbol	Meaning in one line
statement / instance	x	the claim being proved; known to both parties
witness	w	private evidence that x is true; known only to the prover
relation	R	the set of valid pairs (x, w), checkable in polynomial time
language	L_R	the set of true statements $\{x : \exists w, (x, w) \in R\}$
prover	P	party that holds w and tries to convince
verifier	V	efficient party that holds only x and outputs accept/reject
cheating parties	P^*, V^*	arbitrary (dishonest) strategies
transcript	$(m_1, \ldots, m_n)$	the list of all messages exchanged
view	View	everything one party sees: input, coins and received messages
completeness / soundness error	c, s	chance that an honest prover is rejected / a cheat is accepted
simulator	Sim	algorithm without a witness that fakes the verifier's view
extractor	Ext	algorithm that pulls a witness out of a successful prover
negligible	$\mathsf{negl}(\lambda)$	smaller than every inverse polynomial, e.g. $2^{-\lambda}$
proof vs. argument		soundness against all provers vs. only efficient ones

Exercises

Exercise 1.25 (Warm-up)**.** Write down a relation R for the statement "the number N is a perfect square". What is a witness for $N = 144$? Is it easy to check?

Exercise 1.26 (Soundness in the cave)**.** In the cave protocol, how many repetitions k are needed so that a cheating prover is accepted with probability below 10^{-9}? Below 2^{-128}?

Exercise 1.27 (A prover who guesses better)**.** Suppose Victor's challenge in the cave is *not* a fair coin, but he picks A with probability 0.9. Peggy does not know the word and always enters via A. What is her success probability per repetition? Why is a fair coin the best choice for Victor?

Exercise 1.28 (Colour-blind verifier)**.** Describe, in terms of completeness, soundness and zero-knowledge, what each property says in the colour-blind game of Section 1.1.1. What is the verifier's view, and how would a simulator produce it?

Exercise 1.29 (Why not just send the witness?)**.** Give two reasons, one concerning privacy and one concerning reuse, why sending the password in the clear is a poor authentication protocol. Which of the three properties does the trivial protocol "prover sends w" fail?

Exercise 1.30 (Spot the missing property)**.** For each of the following protocols for a relation R, name the property that fails. (a) V always accepts. (b) V always rejects. (c) P sends the witness w and V checks it.

Exercise 1.31 (Negligible or not?)**.** Decide which of $2^{-\lambda}$, $1/\lambda^{100}$, $2^{-\sqrt{\lambda}}$ and $1/\lambda!$ are negligible, and justify each answer.

Exercise 1.32 (Transferability)**.** Explain why a zero-knowledge proof cannot be used by the verifier to convince a third party. How does this relate to the existence of a simulator?

Chapter 2

Mathematical Preliminaries

How to use this chapter

Every zero-knowledge protocol is built from a small toolbox. This chapter introduces the tools *one at a time*. For each tool you will find (i) a formal definition, (ii) the same idea in plain English, and (iii) worked examples with numbers small enough to check by hand. You do **not** have to master everything before reading Chapter 3. Read the sections you need, and come back when a later chapter points here. All the numerical examples use the same tiny "toy" numbers ($p = 23$, $q = 11$, $N = 77$), so you can reuse your calculations.

Tool	What it gives us	First used in
Negligible, indistinguishable	precise meaning of "tiny error" and "looks the same"	Ch. 4
NP relations	what a statement and a witness are	everywhere
Cyclic groups, discrete log	a hard problem with useful algebra	Ch. 7
Quadratic residues	a second hard problem	Ch. 5
Polynomials, Schwartz–Zippel	test a huge equality at one random point	Ch. 9, 12
Pairings	check one product of hidden numbers	Ch. 12
One-way functions	the minimal assumption for ZK	Ch. 6
Commitments	a digital sealed envelope	Ch. 6, 11
Random oracle	replace the verifier's coins by a hash	Ch. 8

2.1 Asymptotics, negligible functions and indistinguishability

2.1.1 Negligible and overwhelming

Cryptographic security is never absolute, because an attacker can always *guess*. Guessing a 128-bit key succeeds with probability 2^{-128}. That is not zero, but it is so tiny that we treat it as impossible. To say this precisely we use a *security parameter* λ, a "knob" (bigger λ

means longer keys, more repetitions, more safety), and we ask how fast the attacker's success probability *shrinks* as we turn the knob.

Definition 2.1 (Negligible, overwhelming)**.** A function $\mu : \mathbb{N} \to \mathbb{R}_{\geq 0}$ is *negligible* if for every polynomial p there is N such that $\mu(\lambda) < 1/p(\lambda)$ for all $\lambda > N$. We write $\mu = \mathsf{negl}(\lambda)$. A function is *overwhelming* if $1 - \mu$ is negligible.

> **In plain English.** A function is negligible if it dies out faster than *every* fraction of the form $1/\lambda^c$, however large the constant c. A probability is overwhelming if the chance of the *opposite* outcome is negligible.

Example 2.2 (Which functions are negligible?)**.**

Function	Negligible?	Reason
$2^{-\lambda}$	yes	exponentially small; beats every $1/\lambda^c$
$2^{-\sqrt{\lambda}}$	yes	$\sqrt{\lambda}$ eventually exceeds $c \log_2 \lambda$ for every c
$1/\lambda!$	yes	$\lambda!$ grows faster than any exponential
$1/\lambda^{10}$	**no**	it is itself $1/p(\lambda)$ for the polynomial $p = \lambda^{10}$
$1/\log \lambda$	**no**	shrinks far too slowly

For scale, at $\lambda = 128$ we have $2^{-128} \approx 2.9 \times 10^{-39}$, whereas $1/\lambda^3 = 1/2\,097\,152 \approx 4.8 \times 10^{-7}$.

Example 2.3 (Why negligible functions are closed under polynomial multiplication)**.** An attacker tries λ^3 independent guesses.

- If each guess succeeds with probability $2^{-\lambda}$, the total success probability is at most $\lambda^3 \cdot 2^{-\lambda}$ (add the probabilities). At $\lambda = 128$ this is $2^{21} \cdot 2^{-128} = 2^{-107}$: still negligible.
- If each guess succeeds with probability only $1/\lambda^2$, the attacker expects $\lambda^3/\lambda^2 = \lambda$ successes: success is nearly certain.

This is why "negligible" (and not just "small for one guess") is the right notion: a negligible chance cannot be amplified to a real chance by any polynomial amount of effort.

Example 2.4 (Overwhelming probability in the cave)**.** After $k = \lambda$ repetitions of the cave protocol, a cheating Peggy is caught with probability $1 - 2^{-\lambda}$. This is overwhelming, because the opposite outcome (she survives) has the negligible probability $2^{-\lambda}$.

2.1.2 Distributions and how far apart they are

Definition 2.5 (Ensembles, statistical distance)**.** A *probability ensemble* $X = \{X_a\}_{a \in S}$ indexed by $S \subseteq \{0,1\}^*$ is a family of random variables. For distributions D_0, D_1 on a finite set Ω,

$$\mathsf{SD}(D_0, D_1) = \max_{T \subseteq \Omega} \big| D_0(T) - D_1(T) \big| = \tfrac{1}{2} \sum_{\omega} |D_0(\omega) - D_1(\omega)|.$$

> **In plain English.** An *ensemble* is just "one probability distribution for every input a" (for instance, the verifier's view for each statement $a = x$). The *statistical distance* measures how different two distributions are, on a scale from 0 (identical) to 1 (completely different). It equals the largest gap that any yes/no question about the outcome can reveal. If you are handed a single sample and must guess which distribution it came from, the best possible strategy wins with probability $\frac{1}{2} + \frac{1}{2}\mathsf{SD}(D_0, D_1)$.

Example 2.6 (A fair coin versus a biased coin)**.** Let D_0 be a fair coin ($\Pr[\text{heads}] = 0.5$) and D_1 a coin with $\Pr[\text{heads}] = 0.6$. Using the sum formula,

$$\mathsf{SD}(D_0, D_1) = \tfrac{1}{2}\big(|0.5 - 0.6| + |0.5 - 0.4|\big) = 0.1.$$

Using the max formula, the best test is $T = \{\text{heads}\}$, which gives the gap $|0.5 - 0.6| = 0.1$. The best one-look guesser wins with probability 0.55.

Example 2.7 (Distance zero: the cave records)**.** In Section 1.5 the real record of one cave repetition is (b, b) with b a fair coin, and Sam's fake record is also (b, b) with b a fair coin. The two distributions are identical, so $\mathsf{SD} = 0$.

Example 2.8 (Small but nonzero: random numbers versus random units)**.** Pick a uniform number from $\{0, \ldots, N-1\}$ and compare it with a uniform *invertible* number from $\mathbb{Z}_N^*$ (those coprime to N). They differ only because the first can hit a non-invertible number, so $\mathsf{SD} = \dfrac{N - \varphi(N)}{N} = \dfrac{P + Q - 1}{N}$ for $N = PQ$. For the toy $N = 77 = 7 \cdot 11$ this is $\frac{17}{77} \approx 0.22$, which is large. For a 2048-bit RSA modulus with $P, Q \approx 2^{1024}$ it is about 2^{-1023}, which is negligible. So here the two distributions are *statistically indistinguishable*.

2.1.3 Three levels of "looks the same"

Definition 2.9 (Indistinguishability)**.** Two ensembles $X = \{X_a\}$, $Y = \{Y_a\}$ are

- *perfectly indistinguishable* ($X \equiv Y$) if X_a and Y_a have identical distributions for every a;
- *statistically indistinguishable* ($X \approx_s Y$) if $\mathsf{SD}(X_a, Y_a) = \mathsf{negl}(|a|)$;
- *computationally indistinguishable* ($X \approx_c Y$) if for every non-uniform PPT distinguisher D the advantage $\big|\Pr[D(a, X_a) = 1] - \Pr[D(a, Y_a) = 1]\big| = \mathsf{negl}(|a|)$.

> **In plain English.** Think of two sealed bags of marbles. *Perfect:* the bags hold exactly the same mix of marbles. *Statistical:* the mixes differ by a negligible amount, so even an unlimited-time examiner who draws a marble learns essentially nothing. *Computational:* the mixes may genuinely differ, but finding out would take an examiner more time than any realistic computer has. A *distinguisher* D is the examiner: an algorithm that receives one sample and outputs a guess bit. Its *advantage* is how differently it behaves on the two inputs. ("Non-uniform" only means that D may carry a fixed cheat-sheet depending on the input length; just think of it as "any efficient algorithm".)

Example 2.10 (One example of each level)**. Perfect:** the real cave record and Sam's fake record (Example above, $\mathsf{SD} = 0$).
Statistical: a uniform element of $\mathbb{Z}_N$ versus a uniform element of $\mathbb{Z}_N^*$ for a huge N (Example **??**).
Computational: the output of a pseudorandom generator versus a truly random string. Let $G : \{0,1\}^n \to \{0,1\}^{2n}$. Only 2^n of the 2^{2n} possible strings are outputs of G, so a distinguisher with unlimited time could try all 2^n seeds and succeed with advantage $1 - 2^{-n}$. The two distributions are therefore *not* statistically close; they are only computationally close, because that exhaustive search is not an efficient algorithm.

Lemma 2.11 (Hybrid argument)**.** *Let $X^{(0)}, \ldots, X^{(m)}$ be distributions, $m = \text{poly}$. If D distinguishes $X^{(0)}$ from $X^{(m)}$ with advantage ε, then for some $i < m$ it distinguishes $X^{(i)}$ from $X^{(i+1)}$ with advantage at least ε/m.*

Proof. Let $p_i = \Pr[D(X^{(i)}) = 1]$. Then $\varepsilon \le |p_0 - p_m| = \big|\sum_{i<m}(p_i - p_{i+1})\big| \le \sum_{i<m}|p_i - p_{i+1}|$, so some term is at least ε/m. □

In plain English. Imagine a staircase with m steps. If the top and the bottom are at very different heights, then at least one single step must be a big one. To show that two *far-apart* distributions look alike, it is enough to find a chain of intermediate distributions where each *neighbouring* pair looks alike.

Example 2.12 (The staircase with numbers)**.** Let $m = 3$ and suppose D outputs 1 with probability $p_0 = 0.9,\ p_1 = 0.85,\ p_2 = 0.35,\ p_3 = 0.3$ on $X^{(0)}, \ldots, X^{(3)}$. The endpoints differ by $\varepsilon = 0.9 - 0.3 = 0.6$. The three steps have sizes 0.05, 0.5, 0.05, and indeed the biggest step (0.5) is at least $\varepsilon/m = 0.2$. So D also tells $X^{(1)}$ from $X^{(2)}$ well.

Example 2.13 (Where the hybrid argument is used)**.** A verifier V^* talks to the prover in m sessions. Let hybrid H_i be the view in which the first i sessions are simulated and the rest are real. Then H_0 is the real view, H_m is the fully simulated view, and neighbouring hybrids differ in *one* session only. If each single session is indistinguishable, the lemma shows that the whole view is, with the error multiplied by m. This is why m must be polynomial: $m \cdot \mathsf{negl}$ is still negligible, whereas ε/m stays non-negligible whenever ε is. Chapter 6 uses exactly this pattern.

2.2 Languages, relations and NP

Definition 2.14. A binary relation $R \subseteq \{0,1\}^* \times \{0,1\}^*$ is an NP-*relation* if membership $(x, w) \in R$ is decidable in time $\mathrm{poly}(|x|)$ and there is a polynomial p with $(x, w) \in R \Rightarrow |w| \leq p(|x|)$. Its language is $L_R = \{x : \exists w, (x, w) \in R\}$; w is a *witness* for x. We write $R(x)$ for the set of witnesses.

In plain English. An NP-relation describes a problem where an answer is *short* and *quick to check*, even if it may be very hard to *find*. The statement x is the question; the witness w is a proposed answer; the check is a fast program. The language L_R is the set of questions that have at least one valid answer. The length bound is there so that a witness can at least be read in polynomial time.

Example 2.15. Each relation below comes with a tiny instance, so that you can run the check yourself.

Relation	**Statement** x	**Witness** w	**Check**
Discrete log $R_{\mathrm{DL}} = \{((\mathbb{G}, g, y), x) : g^x = y\}$	toy group, $g = 2$, $y = 13$	$x = 7$	$2^7 = 128 = 5 \cdot 23 + 13$, so $2^7 \equiv 13$ ✓
Graph isomorphism $R_{\mathrm{GI}} = \{((G_0, G_1), \pi) : \pi(G_0) = G_1\}$	G_0: edges $12, 23, 34$; G_1: edges $24, 41, 13$	$\pi(1) = 2, \pi(2) = 4, \pi(3) = 1, \pi(4) = 3$	apply π to each edge of G_0: $12 \to 24$, $23 \to 41$, $34 \to 13$ ✓
Quadratic residuosity $R_{\mathrm{QR}} = \{((N, v), s) : s^2 \equiv v\}$	$N = 77$, $v = 4$	$s = 9$	$9^2 = 81 \equiv 4 \pmod{77}$ ✓

Relation	Statement x	Witness w	Check
Circuit satisfiability $R_{\text{SAT}} = \{(C, w) : C(w) = 1\}$ (NP-complete)	$C(w_1, w_2, w_3) = (w_1 \vee w_2) \wedge \neg w_3$	$w = (1, 0, 0)$	$(1 \vee 0) = 1$, $\neg 0 = 1$, so $C(w) = 1$ ✓
3-colourability (NP-complete)	triangle on vertices $1, 2, 3$	colours red, green, blue	each of the 3 edges has two different colours ✓
Hamiltonian cycle (NP-complete)	square $1-2-3-4-1$	order $1, 2, 3, 4$	each consecutive pair is an edge, every vertex used once ✓

Example 2.16 (Finding is hard, checking is easy)**.** Given $N = 8633$, *finding* a factor means trying many candidates. Given the witness $w = 89$, *checking* takes one division: $8633 = 89 \cdot 97$. The same gap appears in Sudoku: solving a hard puzzle can take hours, but checking a completed grid takes a minute.

Example 2.17 (A statement with no known short witness)**.** "This graph is *not* 3-colourable" is a perfectly good statement, but nobody knows a short witness for it (it belongs to the class coNP, not known to be in NP). The colour-blind game of Section 1.1.1 proves a similar "negative" fact ("the balls are *different*"), and in Chapter 3 we will see interactive proofs for such statements. This is one reason interaction is powerful.

> **In plain English.** Proofs *of membership* ($x \in L$) and proofs *of knowledge* (P knows w with $(x, w) \in R$) are different goals; Chapter 4 treats the difference. For instance, in a cyclic group *every* element y has a discrete logarithm, so "y has a discrete logarithm" is always true and proving it is pointless. The meaningful claim is "*I know* x with $g^x = y$".

2.3 Algebraic background

2.3.1 Modular arithmetic and groups in a nutshell

Definition 2.18 (Congruence, $\mathbb{Z}_n$, $\mathbb{Z}_n^*$)**.** For $n \geq 2$ we write $a \equiv b \pmod{n}$ if n divides $a - b$. The set $\mathbb{Z}_n = \{0, 1, \ldots, n-1\}$ with addition and multiplication mod n is "clock arithmetic". The set $\mathbb{Z}_n^*$ consists of those elements of $\mathbb{Z}_n$ that have a multiplicative inverse, which are exactly those coprime to n.

> **In plain English.** Mod-n arithmetic is like a clock with n hours: after $n - 1$ you wrap around to 0. $\mathbb{Z}_n^*$ is the set of numbers you are allowed to "divide by".

Example 2.19 (Clock arithmetic mod 23)**.** $17 + 9 = 26 \equiv 3$. $5 \cdot 9 = 45 \equiv 22$, and $22 \equiv -1$. The inverse of 5 is 14, because $5 \cdot 14 = 70 = 3 \cdot 23 + 1 \equiv 1$. Since 23 is prime, $\mathbb{Z}_{23}^* = \{1, \ldots, 22\}$ has 22 elements. For $n = 77 = 7 \cdot 11$ only numbers not divisible by 7 or 11 are invertible, so $|\mathbb{Z}_{77}^*| = 6 \cdot 10 = 60$.

Definition 2.20 (Group, order, cyclic group)**.** A *group* is a set with an operation that is associative, has an identity, and in which every element has an inverse. The *order* of a group is its number of elements, and the *order* $\text{ord}(a)$ of an element a is the least $k \geq 1$ with $a^k = 1$. A group is *cyclic* if it equals $\langle g \rangle = \{g^0, g^1, g^2, \ldots\}$ for some *generator* g.

In plain English. A cyclic group is what you get by multiplying one fixed number g by itself over and over until you return to 1. All the elements are powers of g.

2.3.2 Cyclic groups and discrete logarithm assumptions

Let $\mathbb{G} = \langle g \rangle$ be cyclic of prime order q. Throughout we use a toy group for numerical examples:

Toy group. $p = 23$, $q = 11$, $\mathbb{G} = \langle 2 \rangle \subset \mathbb{Z}_{23}^*$, $g = 2$.
Indeed $2^{11} = 2048 = 89 \cdot 23 + 1 \equiv 1$, so $\mathrm{ord}(2) = 11$. $\mathbb{G}$ is the set of squares mod 23: $\{1, 2, 3, 4, 6, 8, 9, 12, 13, 16, 18\}$.

Powers of 2 mod 23 (the *discrete-log table*, handy for all later examples):

k	0	1	2	3	4	5	6	7	8	9	10
$2^k \bmod 23$	1	2	4	8	16	9	18	13	3	6	12

Example 2.21 (Using the table, and a pitfall)**.** Going *forward* is a table lookup: $2^8 = 3$. Going *backward* means searching the table: which k gives $2^k = 13$? Row 7, so $k = 7$. Exponents live *mod* $q = 11$, not mod $p = 23$: $2^{13} = 2^{13 \bmod 11} = 2^2 = 4$. (Check: $2^{13} = 8192 = 356 \cdot 23 + 4$.) Since 11 is prime, every non-identity element generates $\mathbb{G}$; for instance $3 = 2^8$ is also a generator.

Example 2.22 (Fast exponentiation)**.** To compute 2^{13} without a table, write $13 = 8 + 4 + 1$ and multiply the precomputed squarings 2, $2^2 = 4$, $2^4 = 16$, $2^8 = 256 \equiv 3$: $2^{13} = 2^8 \cdot 2^4 \cdot 2^1 = 3 \cdot 16 \cdot 2 = 96 \equiv 4 \pmod{23}$. This takes about $\log_2(13)$ multiplications, so exponentiation stays fast even when the exponent has 256 bits.

Definition 2.23 (Discrete logarithm, CDH, DDH)**.** For a group $\mathbb{G}$ of prime order q with generator g:

- **DL**: given g^a ($a \xleftarrow{\$} \mathbb{Z}_q$) find a.
- **CDH**: given (g^a, g^b) find g^{ab}.
- **DDH**: distinguish (g^a, g^b, g^{ab}) from (g^a, g^b, g^c) with $a, b, c \xleftarrow{\$} \mathbb{Z}_q$.

Each assumption states that every PPT algorithm succeeds with negligible advantage. The assumptions get stronger from DL to DDH: if DDH is hard then CDH is hard, and if CDH is hard then DL is hard (an algorithm solving DL immediately solves CDH, and one solving CDH immediately distinguishes DDH tuples).

In plain English. *DL:* from the public number g^a, recover the secret exponent. *CDH:* from g^a and g^b, compute the "shared secret" g^{ab}. *DDH:* given a candidate Z, decide whether it really is g^{ab} or just a random group element. Each problem is at least as easy as the next.

Example 2.24 (Diffie–Hellman in the toy group)**.** Alice picks $a = 3$ and publishes $g^a = 2^3 = 8$. Bob picks $b = 5$ and publishes $g^b = 2^5 = 9$. Each can compute the shared key: Alice $9^3 = 729 \equiv 16$, Bob $8^5 = 32768 \equiv 16$, and indeed $g^{ab} = 2^{15} = 2^4 = 16$. An eavesdropper sees only $(8, 9)$. *CDH* asks for 16. *DDH* asks: given $(8, 9, Z)$, is $Z = 16$? *DL* would let the eavesdropper recover $a = 3$ and then compute $9^3 = 16$ themselves, which is why CDH is no harder than DL. In the toy group the table breaks all three at once; the assumptions are meant for groups so large that no table can be built.

Example 2.25 (Why DDH fails in $\mathbb{Z}_p^*$ itself, and why we use prime-order subgroups)**.** In $\mathbb{Z}_{23}^*$ take the generator 5 (order 22). The squares mod 23 are $\{1, 2, 3, 4, 6, 8, 9, 12, 13, 16, 18\}$, and

5^k is a square exactly when k is even. So 5^{ab} is a *non-square* exactly when a and b are both odd, i.e. exactly when 5^a and 5^b are both non-squares. Take $a = 3,\ b = 5$: $5^a = 10$ and $5^b = 20$ are both non-squares, and the true $5^{ab} = 5^{15} = 19$ is a non-square, as predicted. A random $Z = 5^6 = 8$ is a square, so the test "Z is a non-square iff X and Y both are" *rejects* it. Random triples pass this test only half of the time, while real triples always pass, which gives a cheap distinguisher. Working in the prime-order subgroup (where every element is a square) removes this leak.

2.3.3 Quadratic residues and the Blum integer setting

Let $N = PQ$ with P, Q distinct odd primes. $\mathrm{QR}_N = \{a \in \mathbb{Z}_N^* : \exists s,\ s^2 \equiv a\}$. The Jacobi symbol $\left(\frac{a}{N}\right) = \left(\frac{a}{P}\right)\left(\frac{a}{Q}\right)$ is efficiently computable without the factorisation. Let $J_N^{+1} = \{a : \left(\frac{a}{N}\right) = 1\}$; then $\mathrm{QR}_N \subsetneq J_N^{+1}$ and $|J_N^{+1}| = 2|\,\mathrm{QR}_N\,|$.

> **In plain English.** A *quadratic residue* (QR) is a number that is a perfect square mod N. The Legendre symbol $\left(\frac{a}{P}\right)$ is $+1$ if a is a square mod the prime P and -1 if not; the Jacobi symbol multiplies the two. The surprising fact is that a number with Jacobi symbol $+1$ is a square mod P and mod Q *or* a non-square mod both, and without knowing P, Q you cannot tell which.

Example 2.26 (Squares and Jacobi symbols modulo $77 = 7 \cdot 11$)**.** The squares mod 7 are $\{1, 2, 4\}$ and the squares mod 11 are $\{1, 3, 4, 5, 9\}$.

a	$\left(\frac{a}{7}\right)$	$\left(\frac{a}{11}\right)$	Jacobi $\left(\frac{a}{77}\right)$	in QR_{77}?
4	$+1$	$+1$	$+1$	yes ($4 = 2^2$)
6	-1	-1	$+1$	**no**: Jacobi $+1$ but not a square
2	$+1$	-1	-1	no

There are 60 elements in $\mathbb{Z}_{77}^*$, of which 30 have Jacobi symbol $+1$ and only 15 are squares: $\mathrm{QR}_{77} = \{1, 4, 9, 15, 16, 23, 25, 36, 37, 53, 58, 60, 64, 67, 71\}$. The numbers 4 and 6 both look the same to the Jacobi symbol, yet only 4 is a square.

Definition 2.27 (Quadratic residuosity assumption (QRA))**.** Given N (of unknown factorisation) and a uniform $a \in J_N^{+1}$, deciding whether $a \in \mathrm{QR}_N$ is hard for PPT algorithms.

> **In plain English.** If you know the factors 7 and 11, deciding whether 6 is a square is easy (check mod 7 and mod 11). If you know only N and it is huge, nobody knows how to decide this efficiently.

Lemma 2.28 (Square roots $\Leftrightarrow$ factoring)**.** *Given $N = PQ$ and two square roots s, s' of $a \in \mathrm{QR}_N$ with $s' \not\equiv \pm s$, one can factor N.*

Proof. $s^2 \equiv s'^2$ gives $N \mid (s - s')(s + s')$ but $N \nmid (s \pm s')$; hence $\gcd(s - s', N) \in \{P, Q\}$. □

Every $a \in \mathrm{QR}_N$ has exactly four roots $\pm s_1, \pm s_2$ (CRT). **Toy modulus:** $N = 77 = 7 \cdot 11$.

Example 2.29 (Four roots, and factoring from two of them)**.** The number $a = 4$ has the obvious roots ± 2, i.e. 2 and 75. Solving $s \equiv 2 \pmod 7$ and $s \equiv -2 \pmod{11}$ gives the other pair: $s = 9$ (since $9 \equiv 2 \bmod 7$ and $9 \equiv -2 \bmod 11$) and $-9 = 68$. Check: $9^2 = 81 \equiv 4$ ✓. So

the four roots of 4 are $\{2, 9, 68, 75\}$. Now suppose someone shows you the two roots 2 and 9 ($9 \not\equiv \pm 2$). Then

$$\gcd(9-2,\,77) = \gcd(7,77) = 7, \qquad \gcd(9+2,\,77) = \gcd(11,77) = 11,$$

and 77 is factored. Two different square roots of the same number are therefore as good as the factorisation itself. This is the engine behind many soundness proofs (Chapter 5).

Remark 2.30. A *Blum integer* is $N = PQ$ with $P \equiv Q \equiv 3 \pmod 4$; the toy $77 = 7 \cdot 11$ is one ($7 \equiv 11 \equiv 3$). For such N, every square has *exactly one* root that is itself a square: among $\{2, 9, 68, 75\}$ only 9 is in QR_{77}.

2.3.4 Pairings

A (type-3) *bilinear pairing* is an efficiently computable map $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ between groups of prime order q, with generators g_1, g_2 and $e(g_1, g_2) \neq 1$, satisfying $e(g_1^a, g_2^b) = e(g_1, g_2)^{ab}$. We use additive-bracket notation $[a]_1 = g_1^a$, $[a]_2 = g_2^a$, $[a]_T = e(g_1, g_2)^a$ so that $e([a]_1, [b]_2) = [ab]_T$. Pairings let us verify *one* multiplicative relation between hidden exponents—the key to SNARKs (Ch. 11–12). Instantiated by BN254 or BLS12-381.

> **In plain English.** Group elements hide their exponents, and in an ordinary group you can *add* hidden exponents (multiply the elements) but never *multiply* them. A pairing is a special gadget that lets you multiply two hidden exponents once, with the result landing in a third group.

Example 2.31 (Exponent bookkeeping)**.** Suppose $a = 3$ and $b = 5$ are hidden inside $[3]_1$ and $[5]_2$. The pairing gives $e([3]_1, [5]_2) = [15]_T$. A verifier who is handed a claimed product $[c]_T$ simply compares: $[15]_T$ passes the test, while $[16]_T$ fails. The verifier checked $3 \cdot 5 = 15$ without ever seeing 3 or 5. (Here we wrote the exponents down for bookkeeping; in a real pairing group the verifier sees only group elements.)

2.3.5 Polynomials over finite fields

Definition 2.32 (Finite field, polynomial)**.** For a prime p the set $\mathbb{F}_p = \mathbb{Z}_p$ with arithmetic mod p is a *field*: every non-zero element has an inverse, so you can add, subtract, multiply and divide. A *polynomial* over $\mathbb{F}$ is $f(X) = c_0 + c_1 X + \cdots + c_d X^d$ with coefficients in $\mathbb{F}$ ($c_d \neq 0$); $d = \deg f$ is its *degree*.

Example 2.33 (Arithmetic in $\mathbb{F}_{17}$, and roots)**.** Division is multiplication by the inverse: $2^{-1} = 9$ because $2 \cdot 9 = 18 \equiv 1$, so $\frac{6}{2} = 6 \cdot 9 = 54 \equiv 3$ (as expected). The polynomial $X^2 - 4$ has degree 2 and exactly two roots in $\mathbb{F}_{17}$, namely $X = 2$ and $X = 15 \equiv -2$. In general *a non-zero polynomial of degree d has at most d roots*, just as a line crosses the x-axis at most once.

Lemma 2.34 (Schwartz–Zippel)**.** *Let $f \in \mathbb{F}[X_1, \ldots, X_n]$ be a nonzero polynomial of total degree d and $S \subseteq \mathbb{F}$ finite. For $r_1, \ldots, r_n \xleftarrow{\$} S$, $\Pr[f(r_1, \ldots, r_n) = 0] \leq d/|S|$.*

Proof. Induction on n. For $n = 1$, a nonzero degree-d polynomial has at most d roots. For $n > 1$ write $f = \sum_{i=0}^{k} X_n^i f_i(X_1, \ldots, X_{n-1})$ with $f_k \neq 0$, $\deg f_k \leq d - k$. Let E be the event $f_k(r_1..r_{n-1}) = 0$; $\Pr[E] \leq (d-k)/|S|$ by induction. If $\neg E$, then $f(r_1, .., r_{n-1}, X_n)$ is a nonzero univariate polynomial of degree k so vanishes at r_n with probability $\leq k/|S|$. Hence $\Pr[f(\vec{r}) = 0] \leq \Pr[E] + \Pr[\neg E \wedge f = 0] \leq (d-k)/|S| + k/|S| = d/|S|$. □

In plain English. A nonzero polynomial cannot be zero at many points: the fraction of points where it vanishes is at most its degree divided by the number of points available. So if two polynomials $f \neq g$, then $f - g$ is a nonzero polynomial and a *random* point exposes the difference with overwhelming probability. (Proof idea: do one variable at a time; each variable can "use up" at most its own share of the degree.)

Intuition

Schwartz–Zippel says: *a random point is a perfect polynomial-identity test.* To check two huge polynomials $f = g$, evaluate $f - g$ at a random point. This single lemma underlies sum-check, QAP verification, KZG and FRI.

Example 2.35 (Two polynomials, one random test)**.** Over $\mathbb{F}_{17}$ let $f(X) = (X-2)^2 = X^2 - 4X + 4$ and $g(X) = X^2$. Then $f - g = -4X + 4 = -4(X-1)$ has the single root $X = 1$. So f and g agree at exactly one point ($f(1) = g(1) = 1$) and differ at the other 16. A random test point catches the difference with probability $16/17$. For scale: two polynomials of degree 10^6 over a field with 2^{256} elements are mistakenly declared equal with probability at most $10^6/2^{256} \approx 2^{-236}$, after evaluating each at *one* point, instead of comparing a million coefficients.

Example 2.36 (Schwartz–Zippel in action)**.** Let $f(X,Y) = XY - X = X(Y-1)$, total degree 2, over $\mathbb{F}_{17}$. It vanishes iff $X = 0$ or $Y = 1$, so for uniform (x,y) the exact probability is $\frac{1}{17} + \frac{1}{17} - \frac{1}{289} = \frac{33}{289} \approx 0.114$, which indeed is at most $d/|\mathbb{F}| = 2/17 = \frac{34}{289}$. The bound is nearly tight here.

Lemma 2.37 (Lagrange interpolation)**.** *For distinct $x_1, \ldots, x_n \in \mathbb{F}$ and any $y_1..y_n$, the unique polynomial of degree $< n$ with $f(x_i) = y_i$ is $f(X) = \sum_i y_i L_i(X)$ with $L_i(X) = \prod_{j \neq i} \frac{X - x_j}{x_i - x_j}$.*

In plain English. n points determine exactly one polynomial of degree less than n: two points determine a line, three points a parabola, and so on. The formula builds the polynomial as a sum of "switches" L_i, where L_i equals 1 at x_i and 0 at all the other x_j.

Example 2.38 (Two points give a line)**.** Over $\mathbb{F}_{17}$ interpolate $(1,3), (2,5)$: $L_1 = \frac{X-2}{1-2} = -(X-2)$ and $L_2 = \frac{X-1}{2-1} = X - 1$, so $f = 3 \cdot (-(X-2)) + 5(X-1) = 2X + 1$. Check: $f(1) = 3$, $f(2) = 5$. ✓

Example 2.39 (Lagrange interpolation over $\mathbb{F}_{17}$)**.** Interpolate $(1,1), (2,0), (3,1)$. $L_1(X) = \frac{(X-2)(X-3)}{(1-2)(1-3)} = \frac{(X-2)(X-3)}{2}$, $L_3(X) = \frac{(X-1)(X-2)}{(3-1)(3-2)} = \frac{(X-1)(X-2)}{2}$, and L_2 is irrelevant since $y_2 = 0$. So $f = L_1 + L_3 = \frac{(X-2)\left[(X-3)+(X-1)\right]}{2} = (X-2)^2 = X^2 - 4X + 4$. Check: $f(1) = 1$, $f(2) = 0$, $f(3) = 1$. ✓Once we have f we can predict new values, e.g. $f(4) = (4-2)^2 = 4$. This "recover everything from a few points" behaviour is the basis of secret sharing and of the polynomial protocols in later chapters.

Lemma 2.40 (Factor theorem / division)**.** *$f(z) = y$ if and only if $(X - z)$ divides $f(X) - y$ in $\mathbb{F}[X]$.*

In plain English. Saying "f takes the value y at z" is the same as saying that $f(X) - y$ has z as a root, i.e. that you can factor out $(X - z)$ with nothing left over.

Example 2.41 (Checking a claimed value by division)**.** Take $f(X) = (X-2)^2$ over $\mathbb{F}_{17}$. Claim: $f(5) = 9$. Indeed $f(X) - 9 = X^2 - 4X - 5 = (X-5)(X+1)$, so $(X-5)$ divides it, and the quotient is $q(X) = X + 1$. A false claim, say $f(4) = 9$, fails: $f(X) - 9$ at $X = 4$ equals $16 - 16 - 5 = -5 \neq 0$, so $(X-4)$ does not divide it.

This last fact powers KZG openings (Chapter 11): to prove $f(z) = y$, the prover supplies the quotient q with $f(X) - y = (X - z)q(X)$, and the verifier checks this identity at one hidden random point.

2.4 One-way functions, pseudorandomness and trapdoor permutations

Definition 2.42. A polynomial-time computable $f : \{0,1\}^* \to \{0,1\}^*$ is a *one-way function* (OWF) if for every PPT $\mathcal{A}$, $\Pr_{x \xleftarrow{\$} \{0,1\}^n}\left[f(\mathcal{A}(f(x))) = f(x)\right] = \mathsf{negl}(n)$. A *pseudorandom generator* (PRG) is a deterministic polynomial-time $G : \{0,1\}^n \to \{0,1\}^{\ell(n)}$ with $\ell(n) > n$ whose output on a uniform seed is computationally indistinguishable from a uniform $\ell(n)$-bit string. A *trapdoor permutation* (TDP) is a family of permutations f_{pk} that are easy to evaluate and one-way to invert, but easy to invert given a trapdoor sk (e.g. RSA: $f_{N,e}(x) = x^e \bmod N$).

> **In plain English.** *One-way function:* easy to compute, practically impossible to undo, like scrambling an egg. *PRG:* a machine that stretches a short random seed into a long string that looks random to every efficient observer. *Trapdoor permutation:* a one-way function with a secret "back door": whoever holds the trapdoor can undo it.

Example 2.43 (Candidate one-way functions and non-examples)**. Multiplication:** $f(P, Q) = PQ$. Going forward is easy ($7 \cdot 13 = 91$); undoing it means factoring, which is believed hard when P, Q have 1000 bits each. **Exponentiation:** $f(x) = 2^x \bmod 23$; forward is fast, backward needs the table (infeasible in a huge group). **Non-examples:** $f(x) = x + 1$ is undone by subtracting 1, and $f(x) = x^2$ over the integers is undone by taking a square root; neither is one-way.

Example 2.44 (Bad generators and a distinguisher)**.** $G(s) = s \| s$ stretches n bits to $2n$ but is *not* pseudorandom: the distinguisher "output 1 iff the first half equals the second half" says yes on every output of G but on a truly random string only with probability 2^{-n}. Likewise $G(s) = s \| \text{parity}(s)$ (one extra bit equal to the XOR of the bits of s) fails: the test "does the last bit equal the parity of the first n bits?" always says yes for G and says yes for a random string only half of the time, an advantage of $\frac{1}{2}$.

Example 2.45 (Toy RSA trapdoor permutation)**.** Take $N = 77$, $\varphi(N) = 60$, public exponent $e = 7$ (coprime to 60). The trapdoor is $d = 43$ since $7 \cdot 43 = 301 = 5 \cdot 60 + 1$. The permutation is $f(x) = x^7 \bmod 77$: $f(2) = 128 \equiv 51$. Without d you cannot invert (unless you factor 77); with d you compute $51^{43} \bmod 77 = 2$. ✓

Håstad–Impagliazzo–Levin–Luby: OWF $\Leftrightarrow$ PRG. These are the *minimal* assumptions for computational ZK proofs of NP (Theorem 6.4 and its converse in Chapter 13).

Example 2.46 (Naor's commitment from a PRG)**.** Let $G : \{0,1\}^n \to \{0,1\}^{3n}$ be a PRG. *Commit to a bit b:* the receiver sends a random $R \in \{0,1\}^{3n}$; the committer picks a seed s and sends $c = G(s)$ if $b = 0$ and $c = G(s) \oplus R$ if $b = 1$. *Hiding:* $G(s)$ and $G(s) \oplus R$ are both pseudorandom, hence indistinguishable. *Binding:* a cheating committer needs s, s' with $G(s) = G(s') \oplus R$, i.e. $R \in \{G(s) \oplus G(s') : s, s'\}$, a set of size at most 2^{2n}; a uniform R lies in it with probability at most $2^{2n}/2^{3n} = 2^{-n}$. So binding is *statistical*, hiding is computational, and only OWFs are needed.

Tiny numbers ($n = 2$, so $G : \{0,1\}^2 \to \{0,1\}^6$). Use the toy (insecure, for illustration only) table $G(00) = 010110$, $G(01) = 111000$, $G(10) = 001101$, $G(11) = 100010$. The receiver sends $R = 101010$.

- Commit $b = 0$ with seed $s = 01$: send $c = G(01) = 111000$.
- Commit $b = 1$ with seed $s = 10$: send $c = G(10) \oplus R = 001101 \oplus 101010 = 100111$.

Both look like arbitrary 6-bit strings (hiding). To cheat, the committer needs a single c that opens both ways, i.e. $R = G(s) \oplus G(s')$ for some seeds. The XORs of pairs of outputs are $\{000000, 011010, 011011, 101110, 101111, 110100, 110101\}$ (at most $2^{2n} = 16$ values out of $2^6 = 64$ possible R). Our $R = 101010$ is not among them, so no cheating is possible. A "bad" R would be, for instance, $101110 = G(00) \oplus G(01)$: then $c = G(00) = 010110$ opens as 0 (seed 00) and as 1 (seed 01, since $G(01) \oplus R = 010110$). The chance that a random R is bad is at most $16/64 = 2^{-n}$.

2.5 Commitment schemes

Definition 2.47 (Commitment scheme)**.** A commitment scheme consists of PPT algorithms $\mathsf{Setup}(1^\lambda) \to ck$, $\mathsf{Com}_{ck}(m; r) \to c$, with opening (m, r) to c. It must be

- **Hiding**: for all m_0, m_1 the distributions $\mathsf{Com}(m_0; r)$ and $\mathsf{Com}(m_1; r)$ are indistinguishable (perfectly / statistically / computationally);
- **Binding**: no PPT adversary outputs c and two openings $(m, r) \neq (m', r')$, $m \neq m'$, with $\mathsf{Com}(m; r) = \mathsf{Com}(m'; r') = c$ (except negligible probability; perfect binding if impossible).

> **In plain English.** A commitment is a *sealed envelope*. You write a message, seal it, and hand it over. *Hiding:* nobody can read it through the paper. *Binding:* you cannot swap the message after sealing. Later you *open* the envelope by revealing the message and the randomness r used for sealing. The cave protocol's "Peggy enters and fixes a side" is exactly such a commitment.

Example 2.48 (Two broken schemes, one for each property)**.** $\mathsf{Com}(m) = m$ (just send the message) is perfectly *binding* but not *hiding*: the envelope is transparent. $\mathsf{Com}(m) = 0$ (always send 0) is perfectly *hiding* but not *binding*: the envelope reveals nothing, but you can later claim any message you like. A useful scheme must achieve both at once.

Example 2.49 (Coin flipping over the phone)**.** Alice and Bob want to flip a fair coin by phone, without trusting each other. Alice picks a secret bit a and sends a commitment $c = \mathsf{Com}(a; r)$. Bob replies with his own bit b in the clear. Alice then opens c, and the coin is $a \oplus b$. Alice cannot change a after seeing b (binding), and Bob cannot learn a before choosing b (hiding). If either property failed, one party could force the outcome.

Theorem 2.50 (No scheme is both perfectly hiding and perfectly binding)**.** *Perfect hiding means every c has openings to every message; so a computationally unbounded committer can equivocate: binding is at best computational. The reverse holds as well.*

> **In plain English.** If an envelope hides the message *perfectly*, then the same envelope content is consistent with every possible message, so a committer with unlimited time can find the openings for another message and cheat. Hence you must choose which property you want to be unconditional and which one rests on a hardness assumption (Pedersen: hiding is perfect; Blum/GM below: binding is perfect).

Example 2.51 (Pedersen commitment (perfectly hiding, computationally binding))**.** Let $g, h \in \mathbb{G}$ generate a group of prime order q with $\log_g h$ *unknown*. Define $\mathsf{Com}(m; r) = g^m h^r$ for

$m, r \in \mathbb{Z}_q$. *Hiding:* for each m, $g^m h^r$ is uniform in $\mathbb{G}$ when r is uniform, so c is independent of m. *Binding:* an opening pair $g^m h^r = g^{m'} h^{r'}$ with $m \neq m'$ gives $\log_g h = (m - m')(r' - r)^{-1} \bmod q$, breaking DL. *Homomorphic:* $\mathsf{Com}(m; r) \cdot \mathsf{Com}(m'; r') = \mathsf{Com}(m + m'; r + r')$.

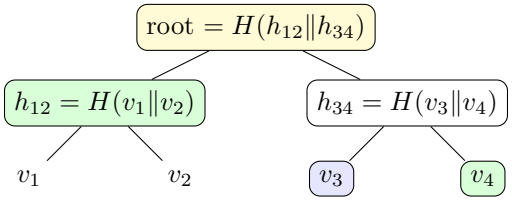
Numbers. Toy group, $g = 2$, $h = 3 = 2^8$ (so $\log_g h = 8$, which in reality must be unknown). Commit $m = 4, r = 6$: $g^4 h^6 = 16 \cdot 3^6 = 16 \cdot 16 = 256 \equiv 3$. Commit $m' = 2, r' = 3$: $4 \cdot 3^3 = 4 \cdot 4 = 16$. Product $3 \cdot 16 = 48 \equiv 2$, and $\mathsf{Com}(6; 9) = 2^6 \cdot 3^9 = 18 \cdot 18 = 324 \equiv 2$. ✓
Equivocation with the trapdoor $t = 8$: to open $c = 3$ (committed to 4 with $r = 6$) as $m^* = 7$ we need $4 + 8 \cdot 6 = 7 + 8r^* \pmod{11}$, i.e. $52 \equiv 8 \equiv 7 + 8r^*$, so $8r^* \equiv 1$, $r^* = 7$ (as $8 \cdot 7 = 56 \equiv 1$). Check: $2^7 \cdot 3^7 = 13 \cdot 2 = 26 \equiv 3$. ✓

Perfect hiding, seen in numbers. The single commitment $c = 3$ has an opening for *every* message, so c says nothing about m:

message m	0	1	4	7
randomness r	1	5	6	7
$2^m 3^r \bmod 23$	3	3	3	3

(For example $2^1 \cdot 3^5 = 2 \cdot 13 = 26 \equiv 3$.) Only someone who knows $t = \log_g h = 8$ can *find* these alternative openings, and that is exactly why $\log_g h$ must stay unknown to the committer.

Example 2.52 (Hash-based commitment)**.** $\mathsf{Com}(m; r) = H(m \| r)$ with H modelled as a random oracle (or collision resistant + hiding via randomness extraction). Computationally hiding and binding; the basis of Merkle trees used in FRI/STARKs. A *Merkle tree* commits to a vector $(v_1..v_n)$ with a single root and opens position i with a log-size authentication path.



Example (opening v_3). The prover sends v_3 with the authentication path (v_4, h_{12}) (green nodes). The verifier computes $h_{34} = H(v_3 \| v_4)$ and then $H(h_{12} \| h_{34})$ and compares it with the root. Any other value v_3' accepted for the same root yields a collision of H, so the commitment is binding; the path has $\log_2 n$ hashes. For a vector of $n = 2^{20} \approx 10^6$ entries the root is one hash, and opening any single entry needs only 20 hashes, instead of sending all 10^6 values.

Example 2.53 (Perfectly binding: Blum / Goldwasser–Micali style)**.** Given $N = PQ$, commit to a bit b by $c = z^2 \cdot y^b \bmod N$ for a fixed non-residue $y \in J_N^{+1}$ and random z. Computationally hiding under QRA and perfectly binding (a QR cannot also be a non-residue). Hence for each bit the commitment determines it uniquely.

Numbers. $N = 77$ and $y = 6$ (Jacobi symbol $+1$ but not a square, see the table above). With $z = 10$: committing $b = 0$ gives $c = z^2 = 100 \equiv 23$, a square; committing $b = 1$ gives $c = 23 \cdot 6 = 138 \equiv 61$, a non-square. Someone who knows 7 and 11 can see the difference ($61 \equiv 5 \pmod 7$ and 5 is not a square mod 7), and so can read the bit. Without the factorisation, 23 and 61 both have Jacobi symbol $+1$ and look alike (QRA), so the bit stays hidden; and since 23 is a square and 61 is not, one commitment can never be opened both ways.

Scheme	Hiding	Binding	Assumption
Naor (PRG)	computational	statistical	one-way functions
Pedersen	**perfect**	computational	discrete log
Blum / GM	computational	**perfect**	quadratic residuosity
Hash / Merkle	computational	computational	collision resistance / ROM

2.6 The random oracle model

A *random oracle* $H : \{0,1\}^* \to \{0,1\}^\ell$ is a function chosen uniformly at random; all parties (including the adversary) access it as a black box. In proofs the simulator *programs* H (answers queries lazily and consistently, possibly choosing outputs in advance) and *observes* the adversary's queries. The ROM is a heuristic: instantiating H with SHA-3 voids the formal guarantee, but no natural attack is known for natural protocols; contrived counterexamples exist [36].

> **In plain English.** Imagine a giant book that lists, for every possible question, a perfectly random answer. Everybody (including the attacker) can look up entries but nobody can see the whole book. A real hash function such as SHA-3 plays the role of this book. The model is a convenient idealisation: security proofs become possible, at the price of trusting that the real hash function behaves like the book.

Example 2.54 (A lazily sampled oracle with 4-bit answers)**.** The oracle does not need to write the whole book in advance: it picks a random answer the first time a question is asked and remembers it.

query #	question	answer
1	"a1"	0110 (fresh random)
2	"b7"	1011 (fresh random)
3	"a1"	0110 (repeat: *same* answer as query 1)

Example 2.55 (Programming the oracle)**.** A simulator that wants to fake a transcript (a, e, z) *without* the witness can pick the challenge e first and the response z next, compute the matching first message a, and then *define* the oracle's answer on the question "$x \| a$" to be e. To everyone else this is indistinguishable from a freshly sampled random answer, as long as "$x \| a$" was not asked before. This programming step is how Chapter 8 shows that Fiat–Shamir proofs leak nothing.

2.7 Chapter summary

Concept	One-line meaning	Worked example
negligible	shrinks faster than every $1/\text{poly}$	Ex. **??**
statistical distance	biggest gap any test can show	coin 0.5 vs 0.6: 0.1
three flavours of "same"	identical / differ negligibly / no efficient test finds a difference	Ex. **??**
hybrid argument	big gap $\Rightarrow$ some small step has a gap	$0.9, 0.85, 0.35, 0.3$
NP relation	short witness, fast check	Ex. 2.6
discrete log, CDH, DDH	recover / compute / recognise hidden exponents	DH with $a = 3, b = 5$
quadratic residue	a perfect square mod N; hard to recognise without factors	4 vs 6 mod 77
two roots $\Rightarrow$ factors	$\gcd(s - s', N)$ reveals a prime	Ex. **??**
Schwartz–Zippel	a random point exposes a polynomial difference	$f = (X-2)^2$ vs $g = X^2$
commitment	sealed envelope: hiding + binding	Pedersen, Ex. 2.19
random oracle	idealised random hash function	lazy table

Exercises

Exercise 2.56. Decide which of $2^{-\lambda/2}$, $\lambda^{-\log\lambda}$, $1/\lambda^{50}$, $1/\sqrt{\lambda}$ are negligible, and justify each answer.

Exercise 2.57. Let D_0 be uniform on $\{1, 2, 3, 4\}$ and D_1 have probabilities $(0.4, 0.3, 0.2, 0.1)$. Compute $\mathsf{SD}(D_0, D_1)$ by the sum formula and find a test T that achieves it.

Exercise 2.58. In the toy group find $\log_2 12$ from the table, and compute 2^{25} without a table by reducing the exponent mod 11.

Exercise 2.59. Run Diffie–Hellman in the toy group with $a = 4$ and $b = 6$. Write down the two public values and the shared key, and confirm that both parties obtain the same key.

Exercise 2.60. Show that 16 is a square mod 77, find all four of its square roots, and use two roots $s \not\equiv \pm s'$ to factor 77 with a gcd.

Exercise 2.61. Over $\mathbb{F}_{17}$ find the polynomial of degree ≤ 2 through $(0, 1), (1, 3), (2, 7)$, and use the factor theorem to check the claim $f(3) = 13$.

Exercise 2.62. In the toy group with $g = 2$, $h = 3$ compute $\mathsf{Com}(5; 2) = g^5 h^2$. Then use the trapdoor $t = \log_g h = 8$ to open the same commitment to the message $m^* = 0$, i.e. find r^*.

Exercise 2.63. In the Naor toy example of Example 2.16, let the receiver send $R' = 011011$ instead of $R = 101010$. Show that now a cheating committer *can* open some commitment both ways, and give the commitment and the two seeds.

Chapter 3

Interactive Proof Systems

3.1 Definitions

An *interactive protocol* is a pair (P, V) of interactive probabilistic algorithms that alternately send messages. $\langle \mathsf{P}(w), \mathsf{V} \rangle (x)$ denotes the (random) output of V (accept = 1 / reject = 0) when P has private input w and both have common input x. V is PPT; P is computationally unbounded in the *information-theoretic* setting.

Definition 3.1 (Interactive proof system)**.** (P, V) is an interactive proof system for L with completeness error c and soundness error s ($c + s < 1$ with a gap, typically $c = 0$, $s = 1/2$) if

Completeness $\forall x \in L$: $\Pr[\langle \mathsf{P}, \mathsf{V} \rangle (x) = 1] \geq 1 - c(|x|)$.

Soundness $\forall x \notin L$, $\forall$ (unbounded) P^*: $\Pr[\langle \mathsf{P}^*, \mathsf{V} \rangle (x) = 1] \leq s(|x|)$.

IP is the class of languages with an interactive proof system with c, s at most $1/3$ (resp. negl after amplification) and polynomially many rounds. An *argument system* restricts soundness to PPT provers.

Why interaction and randomness? Without randomness and without interaction one gets exactly NP (P sends w). Allowing only interaction but a deterministic verifier also gives NP (the prover can predict every verifier message). Randomness *and* interaction jointly give PSPACE.

Theorem 3.2 (Error reduction)**.** *If* (P, V) *has* $c = 0$ *and soundness* s*, then the* k*-fold* sequential *repetition, accepting iff all* k *runs accept, has soundness* s^k*. The same bound holds for* parallel *repetition when the protocol is public-coin or has only three messages.*

Proof (sequential case). Fix $x \notin L$ and any P^*. In each run, conditioned on any history, P^* with the fresh randomness of V is a prover for a single run, so it wins that run with probability $\leq s$. Multiply conditional probabilities. (Parallel repetition is subtler: s^k is proved by a different argument for public-coin and three-message protocols, while for private-coin protocols and for computationally sound arguments with more than three rounds parallel repetition may fail to reduce the error at the expected rate [34].) □

3.2 Example: Graph Non-Isomorphism

$\mathrm{GNI} = \{(G_0, G_1) : G_0 \not\cong G_1\}$ is not known to be in NP (it lies in coNP, but a certificate of non-isomorphism would have to rule out all $n!$ permutations), yet it has a very short interactive proof.

Protocol 3.1: GNI $\in$ IP **(Goldreich–Micali–Wigderson)**

Common input: graphs G_0, G_1 on vertex set $[n]$.

Prover (unbounded)		**Verifier**
		$b \xleftarrow{\$} \{0,1\}$, $\sigma \xleftarrow{\$} S_n$
		$H = \sigma(G_b)$
	$\xleftarrow{\quad H \quad}$	
find b' with $H \cong G_{b'}$		
	$\xrightarrow{\quad b' \quad}$	accept iff $b' = b$

Theorem 3.3. *The protocol has completeness* 1 *and soundness error* $1/2$.

Proof. Completeness. If $G_0 \ncong G_1$, then H is isomorphic to exactly one of G_0, G_1, namely G_b. The unbounded prover finds it and answers $b' = b$.
Soundness. If $G_0 \cong G_1$: the distribution of $H = \sigma(G_b)$ is uniform on the isomorphism class of G_0 (for fixed b, uniform σ yields uniform output, because each graph in the class has the same number $|\operatorname{Aut}(G_0)|$ of permutations mapping G_b to it) and does not depend on b. So H carries no information about b and any prover guesses it with probability exactly $1/2$. □

Example 3.4. G_0: path 1–2–3–4, edges $\{12, 23, 34\}$. G_1: star centred at 1, edges $\{12, 13, 14\}$. Degree sequences $(1, 2, 2, 1)$ vs. $(3, 1, 1, 1)$ so non-isomorphic. Verifier picks $b = 1$, $\sigma = (1 \mapsto 2, 2 \mapsto 4, 3 \mapsto 1, 4 \mapsto 3)$: edges $12 \mapsto 24$, $13 \mapsto 21$, $14 \mapsto 23$, so $H = \{12, 23, 24\}$, a star centred at 2. The prover sees a vertex of degree 3, so $H \cong G_1$ and answers $b' = 1$. If the graphs had been isomorphic, a vertex of degree 3 would be there with the same chance whichever b was used.

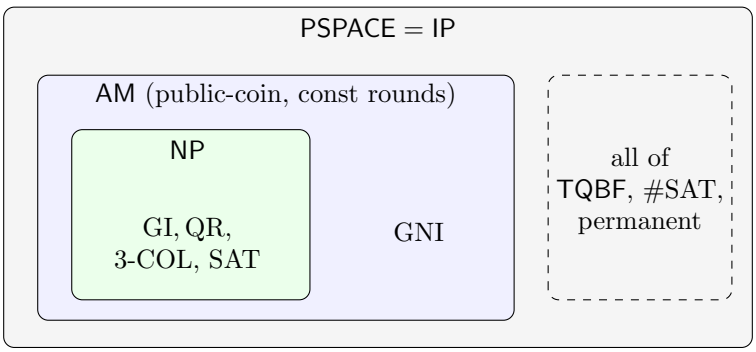
Remark 3.5. This protocol is *honest-verifier* zero-knowledge (a simulator picks b itself and outputs (H, b)), but not zero-knowledge against a cheating V^*: V^* could send an H of his own choosing (e.g. a graph isomorphic to a third graph G_2) and learn from the prover whether $H \cong G_0$ or G_1. A fix by the verifier first proving he knows b, σ makes it full ZK [2]. This illustrates why ZK must be defined against *arbitrary* (malicious) verifiers.

3.3 Public coins and the landscape of IP

A protocol is *public-coin* (Arthur–Merlin) if the verifier's messages are just his coin tosses. The GNI protocol above uses private coins (σ is hidden). Goldwasser–Sipser showed that private coins can be converted into public coins at the cost of two extra rounds (set-lower-bound protocol), so $\mathsf{IP}[k] \subseteq \mathsf{AM}[k+2]$. Public-coin protocols are important for us: *only* public-coin protocols can be made non-interactive by Fiat–Shamir.

Theorem 3.6 (Shamir 1990 [11]; LFKN 1990)**.** $\mathsf{IP} = \mathsf{PSPACE}$.

The proof arithmetises a PSPACE computation (quantified Boolean formula, TQBF) into a low-degree polynomial identity and checks it with a variant of the sum-check protocol. We prove sum-check in Chapter 9 and discuss the arithmetisation there.



Exercises

Exercise 3.7. Show that if V is deterministic then $L \in \mathsf{IP} \Rightarrow L \in \mathsf{NP}$.

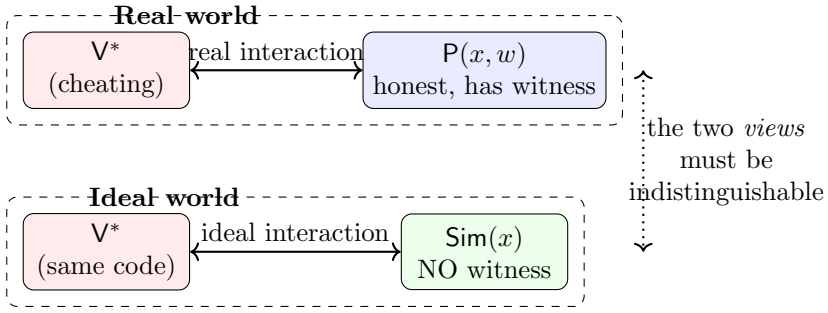
Exercise 3.8. Prove that in Protocol 3.1 the verifier may use fresh randomness σ and b per round, and compute the soundness after k parallel repetitions. Why is private-coin parallel repetition fine here?

Chapter 4

Zero-Knowledge: Definitions

4.1 The simulation paradigm

The informal slogan "the verifier learns nothing" is made precise through a thought experiment of two worlds.



If V^* can compute some function $f(x)$ after talking to P, the same V^* talking to Sim (who has no witness) computes $f(x)$ with about the same probability: so the interaction contributed nothing. The simulator may *interact with* V^* *as a black box with rewinding*, and may run in *expected* polynomial time.

Definition 4.1 (View)**.** For an interactive pair and V^* with auxiliary input z, $\mathsf{View}^{\mathsf{P}(w)}_{\mathsf{V}^*(z)}(x)$ is the random variable $(r, m_1, m_2, \ldots, m_t)$ where r is the content of V^*'s random tape and m_i are the messages that V^* received from P. (The messages V^* sent are determined by x, z, r and $m_1..m_{i-1}$.)

Definition 4.2 (Zero-knowledge, GMR/GMW)**.** Let (P, V) be an interactive proof for the NP-relation R. It is *perfect / statistical / computational zero-knowledge* if for every PPT V^* there is a (expected) PPT simulator $\mathsf{Sim}_{\mathsf{V}^*}$ such that

$$\left\{\mathsf{View}^{\mathsf{P}(w)}_{\mathsf{V}^*(z)}(x)\right\}_{(x,w)\in R,\ z\in\{0,1\}^*} \equiv \ / \approx_s / \approx_c \ \left\{\mathsf{Sim}_{\mathsf{V}^*}(x,z)\right\}_{(x,w)\in R,\ z\in\{0,1\}^*}.$$

If this only holds for the honest V (with $z = \perp$) it is *honest-verifier* zero-knowledge (HVZK).

Points to note.

(a) **Quantifier order.** The simulator may depend on V^* (it "knows" its code) but not on (x, w, z) other than as input.

(b) **Auxiliary input z.** It represents whatever the adversary knew before (earlier protocol runs, partial information on w). Without it, ZK is not closed under sequential composition [32]; with it, it is (Theorem 4.4).

(c) **Perfect vs computational.** Perfect ZK: distributions identical; no assumption needed; typical for GI, QR. Computational ZK: needs a commitment scheme, typical for 3-COL / all of NP.

(d) **Black-box simulation.** Almost all known simulators only use V^* as an oracle $\mathsf{Sim}^{\mathsf{V}^*}(x)$ and rewind it; *non-black-box* simulators (Barak 2001) exist but are exotic.

(e) **Expected polynomial time.** A rewinding simulator may need a random number of attempts. (Strict polynomial-time simulation is possible for fewer protocols.)

Intuition

Why does the definition use *views* and not only outputs? Because V^* could output its entire view; so view-indistinguishability is the strongest notion and equivalent to output-indistinguishability for all V^*.

4.2 Basic properties

Proposition 4.3 (ZK for BPP is trivial)**.** *If $L \in \mathsf{BPP}$, the trivial protocol in which P sends nothing and V decides $x \in L$ is perfect ZK with the simulator outputting (r) only. So ZK is only interesting for $L \notin \mathsf{BPP}$ (e.g. assuming $\mathrm{GI} \notin \mathsf{BPP}$).*

Theorem 4.4 (Sequential composition)**.** *If (P, V) is (auxiliary-input) computational ZK, then the protocol obtained by running it $m = \mathrm{poly}$ times sequentially (with independent randomness) is also computational ZK.*

Proof sketch. Let V^* interact with P for m sessions. Define hybrids H_i: the first i sessions are simulated and the remaining $m - i$ are real. H_0 is the real view, H_m the simulated one. If a distinguisher separates H_{i-1} from H_i, construct a V' which plays the first $i-1$ sessions via the simulator, session i with the real prover (the challenge), and the rest honestly with P; its auxiliary input encodes the state of V^* after session $i-1$ — *this is where auxiliary input is needed.* The two hybrids differ only in whether session i is real or simulated, which contradicts single-session ZK for V'. Finish by Lemma 2.4 (loss factor m). □

Parallel composition may break ZK

Running a ZK protocol k times *in parallel* need not preserve ZK: a rewinding simulator must guess all k challenge bits at once and succeeds with probability 2^{-k} only. Indeed, Goldreich and Krawczyk [31] proved that no 3-round public-coin protocol with negligible soundness error can be *black-box* ZK unless $L \in \mathsf{BPP}$. Remedies: (i) accept constant soundness error per round and repeat sequentially; (ii) use more rounds; (iii) use HVZK with parallel repetition (this *is* safe: HVZK is preserved because the simulator generates the challenge itself); (iv) switch to non-interactive proofs.

4.3 Witness indistinguishability and proofs of knowledge

Definition 4.5 (Witness indistinguishability, WI)**.** (P, V) is WI if for every PPT V^*, all x, any two witnesses $w_1, w_2 \in R(x)$ and every z: $\mathsf{View}^{\mathsf{P}(w_1)}_{\mathsf{V}^*(z)}(x) \approx_c \mathsf{View}^{\mathsf{P}(w_2)}_{\mathsf{V}^*(z)}(x)$.

ZK $\Rightarrow$ WI (both views are indistinguishable from $\mathsf{Sim}(x, z)$). WI is weaker but is preserved under parallel and concurrent composition and suffices when the statement has multiple witnesses (OR-proofs, ring signatures, Chapter 7).

Definition 4.6 (Proof of knowledge (Bellare–Goldreich))**.** (P, V) is a *proof of knowledge* for R with knowledge error κ if there is a probabilistic *knowledge extractor* Ext such that for every prover P^* and every x, with $\varepsilon(x) = \Pr[\langle \mathsf{P}^*, \mathsf{V} \rangle (x) = 1]$: if $\varepsilon(x) > \kappa(x)$ then $\mathsf{Ext}^{\mathsf{P}^*}(x)$ outputs $w \in R(x)$ in expected time at most $\mathrm{poly}(|x|)/(\varepsilon(x) - \kappa(x))$.

Intuition

"P^* knows a witness" means: a witness can be efficiently *computed from* P^* (given rewinding access to it) with success tied to how often P^* convinces V. A prover who convinces the verifier only 1% of the time "knows" it at that level, requiring $\approx 100\times$ more work to extract.

Example 4.7 (Why PoK $\neq$ membership proofs)**.** The statement "$y \in \mathbb{G}$ has a discrete logarithm" is true for *every* $y \in \mathbb{G}$, so the language is trivial. But the *knowledge* of x with $g^x = y$ is a meaningful (indeed identification-enabling) claim. Only the PoK definition can express it.

Remark 4.8 (Arguments). If soundness (or knowledge soundness) only holds against PPT provers, we speak of an *argument (of knowledge)*. Pairing-based SNARKs (Ch. 12) are *arguments of knowledge* (AoK). Statistical ZK arguments for NP exist from collision-resistant hashing; perfect-hiding commitments yield them, whereas statistical ZK *proofs* for all of NP are unlikely (Theorem 13.2).

4.4 Non-interactive variants (preview)

A *non-interactive zero-knowledge (NIZK)* proof consists of $\mathsf{Setup} \to \mathsf{crs}$, $\mathsf{Prove}(\mathsf{crs}, x, w) \to \pi$, $\mathsf{Verify}(\mathsf{crs}, x, \pi) \to \{0, 1\}$ with completeness, soundness and ZK defined via a simulator $(\mathsf{Sim}_1, \mathsf{Sim}_2)$ that produces the CRS *together with a trapdoor* and then proofs without a witness. In the *random oracle model* the simulator programs the oracle instead. See Chapter 8.

Exercises

Exercise 4.9. Show that if a simulator is allowed to output any distribution at all (no indistinguishability) the definition is trivially satisfied. What stops a protocol with soundness error 1 from being ZK?

Exercise 4.10. Prove that ZK implies WI. Give an example of a WI protocol that is not ZK.

Exercise 4.11. Let $\epsilon = \Pr[\langle \mathsf{P}^*, \mathsf{V} \rangle (x) = 1]$. For a protocol with challenge space of size 2 and special soundness (Chapter 7), design an extractor with expected running time $O(1/\epsilon)$.

Chapter 5

Classical Zero-Knowledge Protocols

In this chapter we study the first three zero-knowledge proofs, each exhibiting the same three-move pattern *commit* $\to$ *challenge* $\to$ *response*, and we prove every property in full.

5.1 Graph isomorphism (Goldreich–Micali–Wigderson)

Statement: G_0, G_1 are isomorphic graphs on $[n]$. **Witness:** $\pi \in S_n$ with $G_1 = \pi(G_0)$ (meaning: $\{u,v\} \in E_0 \iff \{\pi(u),\pi(v)\} \in E_1$).

Protocol 5.1: Zero-knowledge proof of graph isomorphism

Common input (G_0, G_1). **Prover's witness** π with $\pi(G_0) = G_1$.

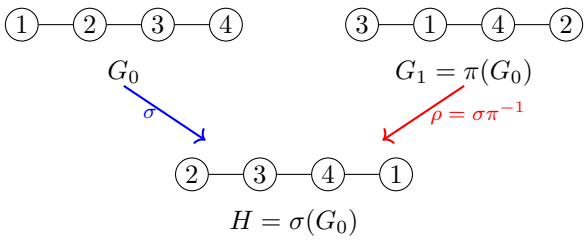
Prover (G_0, G_1, π)		**Verifier** (G_0, G_1)
$\sigma \xleftarrow{\$} S_n$; $H := \sigma(G_0)$	$\xrightarrow{\quad H \quad}$	
	$\xleftarrow{\quad b \quad}$	$b \xleftarrow{\$} \{0,1\}$
$\rho := \sigma$ if $b = 0$		
$\rho := \sigma \circ \pi^{-1}$ if $b = 1$	$\xrightarrow{\quad \rho \quad}$	accept iff $H = \rho(G_b)$

Intuition

H is a freshly shuffled copy of the graph. The verifier asks either "show me that H is a relabelling of G_0" or "…of G_1". Either answer alone is a random permutation (it reveals nothing about π), but if the prover could answer both, then $\rho_1^{-1} \circ \rho_0$ would be an isomorphism $G_0 \to G_1$.

Example 5.1. G_0: edges $\{12, 23, 34\}$ (path $1-2-3-4$). The secret $\pi = (1 \mapsto 3, 2 \mapsto 1, 3 \mapsto 4, 4 \mapsto 2)$ gives $G_1 = \{13, 14, 24\}$ (path 3–1–4–2). Prover picks $\sigma = (1 \mapsto 2, 2 \mapsto 3, 3 \mapsto 4, 4 \mapsto 1)$ and sends $H = \sigma(G_0) = \{23, 34, 41\}$.

- $b = 0$: sends $\rho = \sigma$. Check: $\rho(G_0) = H$. ✓
- $b = 1$: $\pi^{-1} = (1 \mapsto 2, 2 \mapsto 4, 3 \mapsto 1, 4 \mapsto 3)$ so $\rho = \sigma \circ \pi^{-1} = (1 \mapsto 3, 2 \mapsto 1, 3 \mapsto 2, 4 \mapsto 4)$. Then $\rho(G_1)$: $13 \mapsto 32$, $14 \mapsto 34$, $24 \mapsto 14$, i.e. $\{23, 34, 14\} = H$. ✓



Theorem 5.2. *Protocol 5.1 is a perfect zero-knowledge proof system for* GI *with completeness* 1 *and soundness error* 1/2*; it is also a proof of knowledge of* π *with knowledge error* 1/2*.*

Proof. **Completeness.** $b = 0$: $\rho(G_0) = \sigma(G_0) = H$. $b = 1$: $\rho(G_1) = \sigma(\pi^{-1}(\pi(G_0))) = \sigma(G_0) = H$.

Soundness. Suppose $G_0 \ncong G_1$. Whatever H the (unbounded) prover sends, H is isomorphic to at most one of G_0, G_1. So at most one challenge value b can be answered; V picks b after seeing H and independently, hence accepts with probability $\leq 1/2$.

Special soundness / extraction. If the prover answers both challenges for the same H with ρ_0, ρ_1, then $\rho_0(G_0) = H = \rho_1(G_1)$ and $\pi' = \rho_1^{-1} \circ \rho_0$ satisfies $\pi'(G_0) = G_1$: a witness. The extractor runs the prover, then rewinds it to just after H and asks the other challenge; if ε is the success probability this takes $O(1/\varepsilon)$ expected attempts.

Zero-knowledge. Let V^* be any (efficient, possibly randomized) verifier with randomness r and auxiliary input z; The simulator samples r once, uniformly, and keeps it fixed across all rewinds (the output view contains r), so we may regard V^* as a deterministic function of the messages it has received. We define a simulator Sim (using V^* as a black box) that does *not* use π:

1. Pick $b' \xleftarrow{\$} \{0,1\}$ and $\rho' \xleftarrow{\$} S_n$; set $H' := \rho'(G_{b'})$.
2. Run $\mathsf{V}^*(H')$ to get its challenge b.
3. If $b = b'$: output (H', ρ') together with V^*'s tape. Otherwise rewind V^* to the start and go to step 1.

Claim A. H' is uniform over the isomorphism class $\mathcal{C}$ of G_0 irrespective of b' (since $G_0 \cong G_1$, $\rho'(G_{b'})$ is uniform over $\mathcal{C}$ for either value of b': each $H \in \mathcal{C}$ has exactly $|\operatorname{Aut}(G_0)|$ preimages ρ'). Consequently the challenge $b = \mathsf{V}^*(H')$ is independent of b' and $\Pr[b = b'] = 1/2$ in each attempt. The expected number of attempts is 2.

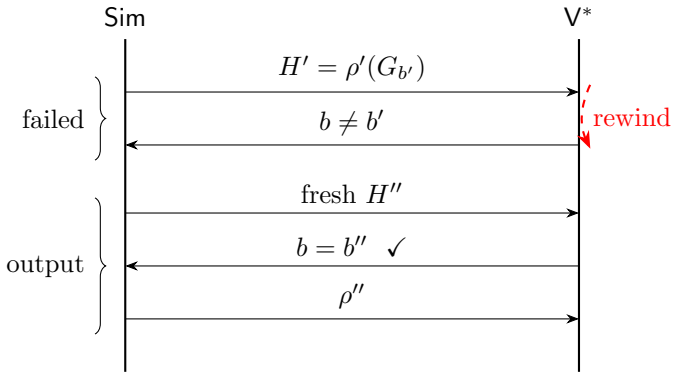
Claim B. The output has exactly the real distribution. In the real interaction, (H, ρ) is distributed as: H uniform on $\mathcal{C}$; given H, $b = \mathsf{V}^*(H)$ is determined; ρ is uniform over the $|\operatorname{Aut}(G_0)|$ isomorphisms $G_b \to H$. In the simulation, by the above, a successful attempt outputs (H', ρ') with

$$\Pr[(H', \rho') = (H, \rho) \mid \text{success}] = \frac{\frac{1}{2} \cdot \frac{1}{|\mathcal{C}|\,|\operatorname{Aut}|} \cdot \mathbf{1}[\rho(G_{b(H)}) = H]}{\frac{1}{2}} = \frac{1}{|\mathcal{C}|\,|\operatorname{Aut}|},$$

because for (H, ρ) with $\rho(G_{b(H)}) = H$ exactly one value $b' = b(H)$ succeeds, and $\Pr[\text{success}] = \sum_H \frac{1}{|\mathcal{C}|} \cdot \frac{1}{2} = \frac{1}{2}$. This is the real probability. Hence the distributions are *identical*: perfect ZK. □

Example 5.3 (Simulating on Example 5.1)**.** The simulator never sees π. It picks $b' = 1$, $\rho' = (1 \mapsto 2, 2 \mapsto 4, 3 \mapsto 1, 4 \mapsto 3)$, so $H' = \rho'(G_1) = \{13, 14, 24\}^{\rho'} = \{21, 23, 43\} = \{12, 23, 34\}$

(which happens to be G_0 itself — a legal element of $\mathcal{C}$). If V^* answers $b = 1$ the simulator outputs (H', ρ'); if $b = 0$ it rewinds and retries with fresh randomness.



5.2 Quadratic residuosity

Let $N = PQ$ and $v \in \mathbb{Z}_N^*$. The prover wants to show $v \in \mathrm{QR}_N$, i.e. knows s with $s^2 \equiv v$.

Protocol 5.2: ZK proof of quadratic residuosity (GMR 1985)

Prover (N, v, s), $s^2 = v$		**Verifier** (N, v)
$r \xleftarrow{\$} \mathbb{Z}_N^*$, $x := r^2$	$\xrightarrow{\quad x \quad}$	
	$\xleftarrow{\quad b \quad}$	$b \xleftarrow{\$} \{0,1\}$
$y := r \cdot s^b$	$\xrightarrow{\quad y \quad}$	accept iff $y^2 \equiv x \cdot v^b \pmod N$

Theorem 5.4. *Protocol 5.2 is a perfect ZK proof (with soundness 1/2) of $v \in \mathrm{QR}_N$, and a proof of knowledge of a square root.*

Proof. Completeness: $y^2 = r^2 s^{2b} = x v^b$. *Soundness:* If $v \notin \mathrm{QR}_N$ and both challenges can be answered for x, then $x = y_0^2 \in \mathrm{QR}_N$ and $xv = y_1^2$, so $v = y_1^2 / y_0^2 \in \mathrm{QR}_N$ (the quadratic residues form a group): contradiction. So at most one challenge is answerable. *Extraction:* with answers y_0, y_1: $s = y_1 / y_0$ since $(y_1/y_0)^2 = xv/x = v$. *ZK:* The simulator guesses $b' \xleftarrow{\$} \{0,1\}$, picks $y \xleftarrow{\$} \mathbb{Z}_N^*$ and sets $x := y^2 v^{-b'}$. Because $v \in \mathrm{QR}_N$ and squaring is a 4-to-1 map onto QR_N, x is uniform over QR_N for both values of b', exactly as the real $x = r^2$. Therefore the challenge $b = \mathsf{V}^*(x)$ equals b' with probability exactly $1/2$ and, given a match, (x, y) is distributed as in the real interaction: x uniform in QR_N and y uniform among the 4 square roots of xv^b. □

Example 5.5. $N = 77 = 7 \cdot 11$, $s = 10$, $v = s^2 = 100 \equiv 23 \pmod{77}$.
Prover: $r = 3$, $x = 9$.

- $b = 0$: $y = 3$; check $y^2 = 9 = x$. ✓
- $b = 1$: $y = 3 \cdot 10 = 30$; $y^2 = 900 \equiv 53$ $(900 - 11 \cdot 77 = 53)$, and $x \cdot v = 9 \cdot 23 = 207 \equiv 53$. ✓

Simulator (no s): guess $b' = 1$, pick $y = 5$; $v^{-1} \equiv 67 \pmod{77}$ (as $23 \cdot 67 = 1541 = 20 \cdot 77 + 1$); $x = y^2 v^{-1} = 25 \cdot 67 = 1675 \equiv 58$. Then if the verifier asks $b = 1$, answer $y = 5$: $y^2 = 25$ and $xv = 58 \cdot 23 = 1334 \equiv 25$. ✓

5.3 Fiat–Shamir and Feige–Fiat–Shamir identification

In an identification scheme, Alice publishes a public key and then repeatedly proves knowledge of the secret key. Feige–Fiat–Shamir uses k secrets simultaneously to shrink the soundness error to 2^{-k} per round.

Protocol 5.3: Feige–Fiat–Shamir identification

Keys. Secret $s_1..s_k \in \mathbb{Z}_N^*$; public $v_i = s_i^2$.

Prover		**Verifier**
$r \xleftarrow{\$} \mathbb{Z}_N^*$, $x = r^2$	$\xrightarrow{\quad x \quad}$	
	$\xleftarrow{\quad e_1..e_k \quad}$	$e \xleftarrow{\$} \{0,1\}^k$
$y = r \prod_i s_i^{e_i}$	$\xrightarrow{\quad y \quad}$	accept iff $y^2 = x \prod_i v_i^{e_i}$

Example 5.6. $N = 77$, $s_1 = 10, s_2 = 4$, $v_1 = 23$, $v_2 = 16$. Prover: $r = 3$, $x = 9$. Verifier: $e = (1,1)$. $y = 3 \cdot 10 \cdot 4 = 120 \equiv 43$. Check: $43^2 = 1849 = 24 \cdot 77 + 1 \equiv 1$ and $x v_1 v_2 = 9 \cdot 23 \cdot 16 = 3312 = 43 \cdot 77 + 1 \equiv 1$. ✓

Remark 5.7 (Soundness/ZK trade-off). *Extraction.* From two accepting answers y, y' to the same x with challenges e, e' we get $(y/y')^2 = \prod_j v_j^{e_j - e'_j}$, so $y/y' = \pm \prod_j s_j^{e_j - e'_j}$. If e, e' differ in exactly one coordinate i, this is $\pm s_i^{\pm 1}$, a square root of $v_i^{\pm 1}$; a prover that answers all 2^k challenges for one x therefore yields all s_i with k such pairs. *HVZK* holds for any k: the simulator picks e, y and sets $x = y^2 \prod_i v_i^{-e_i}$. Full ZK against malicious verifiers requires the simulator to guess e, probability 2^{-k}, so $k = O(\log \lambda)$ is the limit for one round; for bigger k one passes to HVZK + the techniques of Chapter 7 (e.g. committing to e in advance).

Exercises

Exercise 5.8. Write out the knowledge extractor for Theorem 5.2 in detail: given a prover P^* that makes V accept with probability $\varepsilon > 1/2$, show that Ext outputs an isomorphism in expected time $O(1/(\varepsilon - 1/2))$.

Exercise 5.9. Why must v be required to lie in $\mathbb{Z}_N^*$ and the prover's x also checked to be in $\mathbb{Z}_N^*$? (Hint: consider r a multiple of P.)

Exercise 5.10. Show that Protocol 5.2 is *not* sound for the language $\{v : (\frac{v}{N}) = 1\}$ and conclude why the Jacobi symbol does not help.

Exercise 5.11. Implement the GI simulator for $n = 5$ and check empirically that the distribution of transcripts is the same as the real one.

Chapter 6

Zero-Knowledge Proofs for all of NP

We now show: *every language in* NP *has a (computational) zero-knowledge proof* (Goldreich–Micali–Wigderson 1986/1991), assuming one-way functions exist. The strategy: give a ZK proof for an NP-complete problem (graph 3-colouring or Hamiltonicity) and compose it with the Karp reduction.

6.1 Commitments as "digital envelopes"

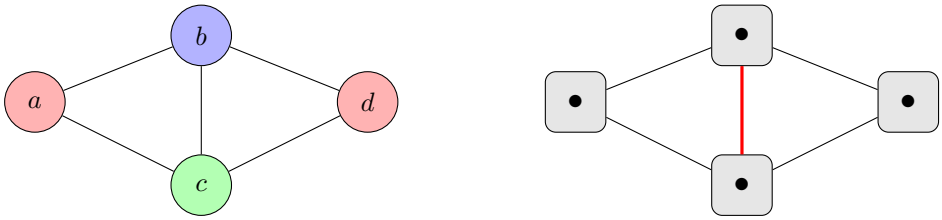
Recall (Chapter 2) that a commitment hides a value yet binds the committer. In the following protocols we need a *non-interactive, perfectly (or statistically) binding, computationally hiding* commitment Com; Naor's construction from a PRG [29] requires only one-way functions (one round of interaction), and the Goldwasser–Micali commitment gives one non-interactively under QRA. Think of it as locking a value in an envelope.

6.2 3-colouring

Protocol 6.1: ZK proof of 3-colourability (one round)

Input $G = (V, E)$. **Prover's witness** a proper colouring $\varphi : V \to \{1, 2, 3\}$.

Prover		**Verifier**
$\pi \xleftarrow{\$} S_3$ (colour permutation);		
$c_v := \mathsf{Com}(\pi(\varphi(v)); r_v)\ \forall v$	$\xrightarrow{(c_v)_{v \in V}}$	
	$\xleftarrow{\{u, w\}}$	$\{u, w\} \xleftarrow{\$} E$
open c_u, c_w:	$\xrightarrow{(a, r_u), (a', r_w)}$	accept iff openings valid, $a, a' \in \{1, 2, 3\}$ and $a \neq a'$



Graph with a proper 3-colouring φ; commitments; V picks edge bc and sees two different colours

Example 6.1. G: vertices a, b, c, d, edges ab, ac, bc, bd, cd. Witness $\varphi = (a, b, c, d) \mapsto (1, 2, 3, 1)$. Prover picks $\pi : 1 \mapsto 3, 2 \mapsto 1, 3 \mapsto 2$, so commits to $(3, 1, 2, 3)$. V asks bc; prover opens $(1, 2)$; $1 \neq 2$: accept. Had V asked ab he would have seen $(3, 1)$; for bd he would have seen $(1, 3)$. In each round the pair of opened colours is a uniformly random ordered pair of *distinct* colours, whatever φ is.

Cheating prover: for K_4 (6 edges, not 3-colourable) any assignment has ≥ 1 monochromatic edge, so a round detects the cheat w.p. $\geq 1/6$. After $6k$ sequential rounds the error is $\leq (5/6)^{6k} < e^{-k}$.

Theorem 6.2. *Repeating Protocol 6.1 $|E| \cdot k$ times sequentially (independent π, fresh commitments) yields a computational zero-knowledge proof system for 3-COL with completeness* 1 *and soundness error $\leq e^{-k}$, provided* Com *is perfectly binding and computationally hiding. (With a computationally binding commitment one gets an* argument*.)*

Proof. Completeness. $\pi \circ \varphi$ is a proper colouring so the opened pair always differs.

Soundness. If G is not 3-colourable, then perfect binding ensures that the commitments (c_v) determine a unique assignment $\psi : V \to \{1, 2, 3, \perp\}$ (where $\perp$ means "not a valid commitment to a colour"). Since G has no proper colouring, some edge e^* has equal or invalid endpoint values. V asks for e^* with probability $\geq 1/|E|$, and then the prover cannot open validly. So a single round accepts with probability $\leq 1 - 1/|E|$; by Theorem 3.2 the $|E|k$-fold repetition gives $(1 - 1/|E|)^{|E|k} \leq e^{-k}$.

Zero-knowledge (single round, then sequential composition by Thm. 4.4). Given V^*, define Sim:

1. Pick a random edge $\{u, w\} \in E$ and a uniform pair of distinct colours (a, a').
2. Set $c_u = \mathsf{Com}(a)$, $c_w = \mathsf{Com}(a')$ and $c_v = \mathsf{Com}(1)$ for every other v (all with fresh randomness).
3. Send (c_v) to V^* and obtain its edge e^*. If $e^* = \{u, w\}$, open c_u, c_w and output V^*'s view. Otherwise rewind to step 1 (at most $|E| \cdot n$ times, then output $\perp$).

Define distributions over commitment vectors: $\mathcal{R}$ (the real one, to $\pi \circ \varphi$) and, for each edge e, $\mathcal{D}_e$ (the simulated one when e is guessed). In both, the plaintext on e is a uniform pair of distinct colours; they differ in the plaintexts of the remaining $\leq n - 2$ vertices. By the hybrid argument (Lemma 2.4), replacing the commitments of vertices one at a time and using hiding,

$$\mathcal{R} \approx_c \mathcal{D}_e \quad \text{for each } e \in E.$$

Let $p_e = \Pr_{\mathcal{R}}[\mathsf{V}^* \text{ asks } e]$, so $\sum_e p_e = 1$. The test "V^* asks e" is an efficient test so $|\Pr_{\mathcal{D}_e}[\mathsf{V}^* \text{ asks } e] - p_e| = \mathsf{negl}$. Hence in one iteration

$$\Pr[\text{success}] = \frac{1}{|E|} \sum_e \Pr_{\mathcal{D}_e}[\mathsf{V}^* \text{ asks } e] = \frac{1}{|E|} \pm \mathsf{negl},$$

so the expected number of iterations is $|E|(1 \pm \mathsf{negl})$ and the simulator is expected polynomial time. Conditioned on success, the output is a view with edge e with probability $\frac{(1/|E|)\Pr_{\mathcal{D}_e}[e]}{\Pr[\text{success}]} \approx p_e$, matching the real world; and conditioned on e, the view (commitments to $\mathcal{D}_e$ vs. $\mathcal{R}$, plus the opened colours, uniformly random distinct in both worlds) are computationally indistinguishable by the same hybrid argument. Summing over the polynomially many e gives $\mathsf{View}_{V^*} \approx_c \mathsf{Sim}(G)$. □

Intuition

The verifier tests one edge, sees two distinct random colours, and nothing else: all other envelopes remain closed. The permutation π guarantees that the two colours are just "two different random colours" no matter what. If a coloured graph is invalid, the verifier will, with probability at least $1/|E|$, ask for a violated edge. The need for $|E|$ repetitions is a weakness addressed by Blum's protocol.

6.3 Blum's protocol for Hamiltonicity

Statement: G on $[n]$ has a Hamiltonian cycle C. Soundness error $1/2$ per round.

Protocol 6.2: Blum's ZK proof for Hamiltonicity

Prover (G, C)		**Verifier**
$\pi \xleftarrow{\$} S_n$; $A :=$ adjacency matrix of $\pi(G)$		
commit entrywise: $c_{ij} = \mathsf{Com}(A_{ij})$	$\xrightarrow{(c_{ij})}$	
	$\xleftarrow{b}$	$b \xleftarrow{\$} \{0,1\}$
$b = 0$: reveal π and all openings	$\longrightarrow$	check $A = \mathrm{adj}(\pi(G))$
$b = 1$: open only the n entries of $\pi(C)$	$\longrightarrow$	check these are all 1 and form an n-cycle

Why it works. For $b = 0$ the prover shows the commitments are a faithful relabelled copy of G; for $b = 1$ she shows a Hamiltonian cycle exists in the committed graph; she cannot do both for a non-Hamiltonian G (otherwise $\pi^{-1}(\pi(C))$ would be a Hamiltonian cycle of G). Revealed information: for $b = 0$ a random permutation of G; for $b = 1$ a random n-cycle on random labels. Simulation: guess b'; if $b' = 0$ commit to $\pi(G)$, if $b' = 1$ commit to a random n-cycle plus zeros; rewind on mismatch (expected 2 attempts). The soundness error $1/2$ per round, not $1 - 1/|E|$, is the main advantage over Protocol 6.1.

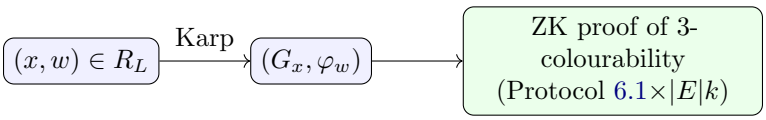
Example 6.3. $G = K_4$ minus edge $\{13\}$; Hamiltonian cycle $C : 1-2-3-4-1$. With $\pi = (1 \mapsto 3, 2 \mapsto 4, 3 \mapsto 1, 4 \mapsto 2)$, $\pi(C)$ is the cycle $3{-}4{-}1{-}2{-}3$. For $b = 1$ she opens the four commitments $c_{34}, c_{41}, c_{12}, c_{23}$, all containing 1.

6.4 The main theorem

Theorem 6.4 (Goldreich–Micali–Wigderson [2])**.** *If one-way functions exist, every language in* NP *has a computational zero-knowledge proof system. Moreover the prover can be implemented*

in probabilistic polynomial time given a witness.

Proof sketch. (i) OWF $\Rightarrow$ PRG [30] $\Rightarrow$ perfectly binding, computationally hiding commitment [29]. (ii) Theorem 6.2 gives a ZK proof for 3-COL, NP-complete. (iii) For $L \in \mathsf{NP}$ let f be a Karp reduction to 3-COL and map witnesses by the (efficient) witness-translation of the reduction; run Protocol 6.1 on $f(x)$. Soundness and ZK transfer because $x \in L \iff f(x) \in$ 3-COL. □



Remark 6.5 (Efficiency). Via Karp reductions the statement size blows up polynomially (and with a big constant): this is irrelevant for feasibility but is terrible in practice—a motivation for the direct arithmetisation techniques of Chapters 9–12, where one gets *succinct* proofs.

6.5 Round complexity and composition limits

- The protocols above use $\omega(\log \lambda)$ rounds when repeated sequentially. **Constant-round** ZK proofs for NP exist (Goldreich–Kahan 1996; Feige–Shamir) at the cost of non-trivial constructions; by Goldreich–Krawczyk, *black-box* ZK with 3 rounds is impossible outside BPP [31].
- **Concurrent ZK**: if the verifier interleaves many sessions, rewinding-based simulators blow up. Richardson–Kilian showed that concurrent ZK is possible but needs $\omega(\log \lambda)$ rounds for black-box simulation (Canetti–Kilian–Petrank–Rosen). Barak's *non-black-box* simulation circumvents black-box lower bounds, giving constant-round bounded-concurrent ZK [33].
- **Non-interactive ZK** (Chapter 8) removes the issue entirely, at the price of a CRS or the random oracle.

Exercises

Exercise 6.6. Show formally that if the commitment in Protocol 6.1 is only computationally binding, the protocol is an argument but not a proof, and explain what a computationally unbounded prover can do.

Exercise 6.7. The simulator in the proof of Theorem 6.2 fixes all other colours to 1. Why would the simulator break if it also revealed those colours? What if the commitment were not hiding?

Exercise 6.8. Prove the completeness, soundness and perfect ZK (with a perfectly hiding commitment!) of Protocol 6.2 formally. Which type of binding do you need, and why does a perfectly hiding commitment only give an *argument*?

Exercise 6.9. Compute exactly the number of rounds of Protocol 6.1 needed for soundness 2^{-80} on a graph with $|E| = 1000$.

Chapter 7

Σ-Protocols

Σ-protocols are three-move public-coin proofs of knowledge with a very clean algebraic structure. They are the "assembly language" of practical zero-knowledge: Schnorr identification and signatures, Pedersen commitment proofs, the ring signatures of CDS, confidential transactions, and the building blocks of Bulletproofs/Halo2 are all Σ-protocols.

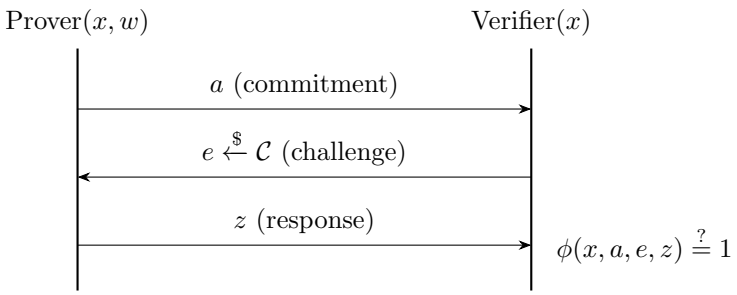
7.1 Definition

Definition 7.1 (Σ-protocol)**.** A Σ-protocol for an NP-relation R is a 3-move protocol: $\mathsf{P}(x, w)$ sends a *commitment* a; $\mathsf{V}(x)$ replies with a uniformly random *challenge* $e \xleftarrow{\$} \mathcal{C}$ ($\mathcal{C}$ finite, often $\mathbb{Z}_q$); P sends a *response* z; V accepts iff $\phi(x, a, e, z) = 1$ for a deterministic predicate ϕ. It must satisfy:

Completeness if $(x, w) \in R$ and both follow the protocol, V accepts.

Special soundness there is an efficient extractor Ext which, given x and two accepting transcripts (a, e, z), (a, e', z') with the *same* first message and $e \neq e'$, outputs w with $(x, w) \in R$.

Special honest-verifier ZK (SHVZK) there is an efficient simulator Sim which on input (x, e) outputs (a, z) such that (a, e, z) is distributed identically (or indistinguishably) to a real accepting transcript with challenge e.



Why special soundness gives knowledge

Two accepting answers to *different* questions about the *same* commitment can only be produced by someone who could "open the commitment in two ways", which reveals the witness. The prover cannot change her commitment after seeing the challenge, so a prover that succeeds for many challenges for a given a must know w (rewinding formalises this).

Theorem 7.2 (Special soundness $\Rightarrow$ proof of knowledge)**.** *Let* (P,V) *be a* Σ*-protocol with* $|\mathcal{C}| = N$*. Then it is a proof of knowledge with knowledge error* $1/N$*. In the simple form proved below: if a prover* P^* *makes* V *accept with probability* $\varepsilon \geq 4/N$*, there is an extractor running in expected time* $O(1/\varepsilon)$ *(calls to* P^**) that outputs a witness. (Constants are not optimised; the sharp bound* $\mathrm{poly}/(\varepsilon - 1/N)$ *follows from a finer analysis.)*

Proof. Fix the internal randomness ω of P^*; then its first message $a = a(\omega)$ is determined. Let $\varepsilon_\omega = \Pr_e[\mathsf{V} \text{ accepts} \mid \omega]$ so that $\mathbb{E}_\omega[\varepsilon_\omega] = \varepsilon$. Call ω *good* if $\varepsilon_\omega \geq \varepsilon/2$. Bad ω's contribute less than $\varepsilon/2$ to the acceptance probability, hence

$$\Pr[\omega \text{ good} \mid \text{accept}] \;\geq\; \tfrac{1}{2}.$$

Extractor.

1. Sample (ω, e) and run P^* until V accepts; this takes $1/\varepsilon$ attempts in expectation. Let (a, e, z) be the accepting transcript.
2. Keeping ω (so a) fixed, sample fresh challenges e' until V accepts with $e' \neq e$, giving up after $8/\varepsilon$ samples.
3. If a second transcript (a, e', z') was found, output $\mathsf{Ext}(x, (a, e, z), (a, e', z'))$ (special soundness).

Analysis. If ω is good, at least $\varepsilon N/2$ challenges are accepted; removing e leaves at least $\varepsilon N/2 - 1 \geq \varepsilon N/4$ (using $\varepsilon N \geq 4$). Thus each sample in step 2 succeeds with probability at least $\varepsilon/4$, and $8/\varepsilon$ samples fail with probability at most $(1 - \varepsilon/4)^{8/\varepsilon} \leq e^{-2} < 0.14$. One pass therefore succeeds with probability at least $\frac{1}{2} \cdot 0.86 > 0.4$ and costs at most $1/\varepsilon + 8/\varepsilon$ calls; repeating passes until success takes expected $O(1/\varepsilon)$ calls. □

Theorem 7.3 (Parallel repetition and AND)**.** *Running* Σ*-protocols for* R_1, R_2 *in parallel with a* single *common challenge (when* $\mathcal{C}_1 = \mathcal{C}_2$*) yields a* Σ*-protocol for* $R_1 \wedge R_2 = \{((x_1, x_2), (w_1, w_2))\}$*. Special soundness follows by applying the two extractors; SHVZK by running the two simulators with the same* e*.*

7.2 Schnorr's protocol

Let $\mathbb{G} = \langle g \rangle$ of prime order q, $y = g^x$. The prover claims to know x.

Protocol 7.1: Schnorr identification / proof of knowledge of discrete log

Prover $(g, y; x)$		**Verifier** (g, y)
$r \xleftarrow{\$} \mathbb{Z}_q$, $t := g^r$	$\xrightarrow{\quad t \quad}$	
	$\xleftarrow{\quad c \quad}$	$c \xleftarrow{\$} \mathbb{Z}_q$
$s := r + cx \bmod q$	$\xrightarrow{\quad s \quad}$	accept iff $g^s = t \cdot y^c$

Theorem 7.4. *Schnorr's protocol is a* Σ*-protocol for* R_{DL} *with perfect SHVZK; hence a proof of knowledge with knowledge error* $1/q$*.*

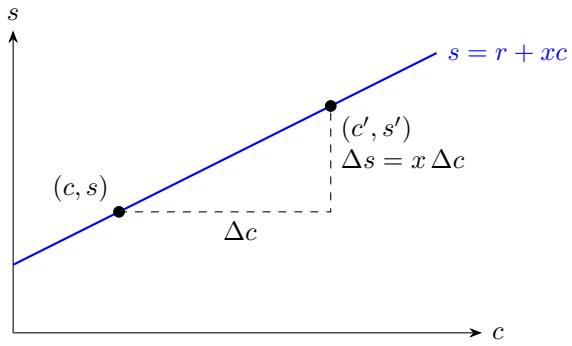
Proof. Completeness: $g^s = g^{r+cx} = g^r (g^x)^c = t y^c$.
Special soundness: from $g^s = t y^c$ and $g^{s'} = t y^{c'}$, $c \neq c'$, divide: $g^{s-s'} = y^{c-c'}$, so $y = g^{(s-s')(c-c')^{-1}}$ and $x := (s - s')(c - c')^{-1} \bmod q$ is a witness.
SHVZK: Given c, Sim picks $s \xleftarrow{\$} \mathbb{Z}_q$ and sets $t := g^s y^{-c}$. In a real transcript with challenge c,

$s = r + cx$ is uniform in $\mathbb{Z}_q$ (as r is), and t is then determined by $t = g^s y^{-c}$ (verification equation). In the simulation, s is uniform and t is determined the same way. The two distributions on (t, s) are identical. □

Example 7.5 (Toy group: $q = 11$, $g = 2$ in $\mathbb{Z}_{23}^*$)**.** Secret $x = 7$, public $y = 2^7 = 13$.

- **Real run.** $r = 8$: $t = 2^8 = 3$. Challenge $c = 4$. $s = 8 + 4 \cdot 7 = 36 \equiv 3 \pmod{11}$. Verify: $g^s = 2^3 = 8$; $t\,y^c = 3 \cdot 13^4 = 3 \cdot 18 = 54 \equiv 8$. ✓ ($13^2 = 169 \equiv 8$, $13^4 \equiv 64 \equiv 18$.)
- **Extraction.** Rewind, challenge $c' = 9$: $s' = 8 + 63 = 71 \equiv 5$. Then $x = (3-5)(4-9)^{-1} = (-2)(-5)^{-1} = 2 \cdot 5^{-1} \equiv 2 \cdot 9 = 18 \equiv 7$ (since $5 \cdot 9 = 45 \equiv 1$). ✓
- **Simulation** for $c = 4$ without x: pick $s = 6$, $t = 2^6 \cdot 13^{-4} = 18 \cdot 18^{-1} = 1$. Transcript $(t, c, s) = (1, 4, 6)$; verify: $2^6 = 18$, $t\,y^4 = 1 \cdot 18$. ✓

Note that the simulated $t = 1$ happens to be the identity; in a larger group the simulated commitment looks uniform.



Two points on the line determine the slope x (⇒ special soundness); one point reveals nothing, as r is a uniform intercept (⇒ ZK).

7.2.1 Zero-knowledge against malicious verifiers

Schnorr is only HVZK as stated. A malicious V^* may choose c as a function of t, and the simulator cannot program t *after* seeing c.

- **Small challenge space.** If $|\mathcal{C}| = \text{poly}$ the simulator guesses c', builds (t, s) by SHVZK, and rewinds if $\mathsf{V}^*(t) \neq c'$; expected $|\mathcal{C}|$ attempts. So Σ-protocols with polynomial-size challenge space are *perfect ZK*, but need superlogarithmically many sequential repetitions for negligible soundness.
- **Large challenge space, Damgård's trick.** Add a first move: V commits to his challenge, $\mathsf{cm} = \mathsf{Com}(c; \rho)$, using a perfectly hiding, computationally binding scheme (e.g. Pedersen). Then P sends t, V opens (c, ρ), and P checks the opening and answers s. *Soundness* is unaffected because cm perfectly hides c from the prover. *Simulation:* run V^* once with a random t_0 to learn the challenge c that it opens; rewind V^* to just after it sent cm and send t produced by the SHVZK simulator for this c. By binding, V^* must open cm to the same c (otherwise it breaks the commitment), so the simulated transcript is accepting. The usual argument shows the expected number of rewinds is $O(1)$.
- **Fiat–Shamir** (Ch. 8) removes the malicious verifier by making the challenge a hash.

7.3 A general view: proofs about group homomorphisms

Theorem 7.6 (Maurer 2009)**.** *Let $\varphi : (\mathbb{Z}_q^m, +) \to (\mathbb{G}, \cdot)$ be a group homomorphism (with $\mathbb{G}$ of prime order q), $y \in \mathbb{G}$, and the relation $R_\varphi = \{(y, \vec{w}) : \varphi(\vec{w}) = y\}$. Then the protocol*

$$t = \varphi(\vec{r}),\ \vec{r} \xleftarrow{\$} \mathbb{Z}_q^m; \qquad c \xleftarrow{\$} \mathbb{Z}_q; \qquad \vec{s} = \vec{r} + c\vec{w}; \qquad \text{check } \varphi(\vec{s}) = t \cdot y^c$$

is a Σ-protocol (special soundness: $\vec{w} = (\vec{s} - \vec{s}')/(c - c')$; SHVZK: $\vec{s} \xleftarrow{\$} \mathbb{Z}_q^m$, $t = \varphi(\vec{s})y^{-c}$).

Proof. Verbatim the proof of Theorem 7.4 using $\varphi(\vec{s} - \vec{s}') = y^{c-c'}$ and the homomorphism property $\varphi(\lambda\vec{v}) = \varphi(\vec{v})^\lambda$. □

Different choices of φ give a whole zoo:

Name	φ	Proves knowledge of
Schnorr	$x \mapsto g^x$	discrete log x
Okamoto/Pedersen opening	$(x_1, x_2) \mapsto g^{x_1}h^{x_2}$	representation / opening of a Pedersen commitment
Chaum–Pedersen	$x \mapsto (g^x, h^x) \in \mathbb{G}^2$	x with $\log_g u = \log_h v$ (DDH tuple)
Multi-exponentiation	$\vec{x} \mapsto \prod g_i^{x_i}$	vector commitment opening
Guillou–Quisquater (RSA)	$x \mapsto x^e$ in $\mathbb{Z}_N^*$	e-th root (needs $c < e$, gcd arguments)

7.3.1 Guillou–Quisquater: knowledge of an e-th root

Let $N = PQ$, e a prime with $\gcd(e, \varphi(N)) = 1$, public $Y = s^e \bmod N$ and secret s. Here the homomorphism is $\varphi(s) = s^e$ on the *multiplicative* group $\mathbb{Z}_N^*$ and the challenge space is $\mathcal{C} = \{0, \ldots, e-1\}$.

Protocol 7.2: Guillou–Quisquater

Prover $(N, e, Y; s)$		**Verifier**
$r \xleftarrow{\$} \mathbb{Z}_N^*$, $t := r^e$	$\xrightarrow{\quad t \quad}$	
	$\xleftarrow{\quad c \quad}$	$c \xleftarrow{\$} \{0, .., e-1\}$
$z := r \cdot s^c$	$\xrightarrow{\quad z \quad}$	accept iff $z^e \equiv t \cdot Y^c \pmod N$

Completeness: $z^e = r^e(s^e)^c = tY^c$. *Special soundness:* from $(t, c, z), (t, c', z')$ with $c \neq c'$: $(z/z')^e = Y^{c-c'}$. Since e is prime and $0 < |c - c'| < e$, $\gcd(c - c', e) = 1$; with Bezout $u(c - c') + ve = 1$ we get $s = (z/z')^u Y^v$, because $\left((z/z')^u Y^v\right)^e = Y^{u(c-c')}Y^{ve} = Y$ and e-th roots in $\mathbb{Z}_N^*$ are unique. *SHVZK:* pick $z \xleftarrow{\$} \mathbb{Z}_N^*$, $t := z^e Y^{-c}$.

Example 7.7. $N = 77$, $e = 7$ (coprime to $\varphi(77) = 60$), $s = 2$, so $Y = 2^7 = 128 \equiv 51$. Prover: $r = 3$, $t = 3^7 = 2187 \equiv 31$ ($77 \cdot 28 = 2156$). Challenge $c = 5$: $z = 3 \cdot 2^5 = 96 \equiv 19$. Verify: $z^7 = 19^7 \equiv 68$ (as $19^2 \equiv 53$, $19^4 \equiv 37$, $19^6 \equiv 36$, $19^7 \equiv 68$) and $tY^5 = 31 \cdot 51^5 \equiv 31 \cdot 32 = 992 \equiv 68$ (as $51^2 \equiv 60$, $51^4 \equiv 58$, $51^5 \equiv 32$). ✓*Extraction* with a second challenge: only $c - c'$ invertible modulo e matters, which is why $\mathcal{C} = \{0, .., e-1\}$ with e prime.

7.3.2 Okamoto's protocol: knowledge of a representation

Given $y = g^{x_1}h^{x_2}$ prove knowledge of (x_1, x_2)—the opening of a Pedersen commitment.

Protocol 7.3: Okamoto / opening of a Pedersen commitment

Prover (x_1, x_2)		**Verifier**
$k_1, k_2 \xleftarrow{\$} \mathbb{Z}_q$; $t = g^{k_1} h^{k_2}$	$\xrightarrow{t}$	
	$\xleftarrow{c}$	$c \xleftarrow{\$} \mathbb{Z}_q$
$s_i = k_i + c x_i$	$\xrightarrow{s_1, s_2}$	accept iff $g^{s_1} h^{s_2} = t\, y^c$

Example 7.8. Toy group, $g = 2$, $h = 3$, $y = g^4 h^6 = 3$ (the commitment of Example 2.19), witness $(4, 6)$. Pick $k = (2, 5)$: $t = 2^2 \cdot 3^5 = 4 \cdot 13 = 52 \equiv 6$. Challenge $c = 3$: $s_1 = 2 + 12 = 14 \equiv 3$, $s_2 = 5 + 18 = 23 \equiv 1$. Verify: $g^3 h^1 = 8 \cdot 3 = 24 \equiv 1$; $t\, y^3 = 6 \cdot 27 \equiv 6 \cdot 4 = 24 \equiv 1$. ✓

Special soundness: two transcripts with $c \neq c'$ give (x_1, x_2). Since Pedersen commitments are *binding*, a prover who knows two different representations of the same y would break DL, hence the *unique* witness.

7.3.3 Chaum–Pedersen: equality of discrete logarithms

Statement: (g, h, u, v) with $u = g^x$, $v = h^x$ (equivalently (g, h, u, v) is a DH-tuple).

Protocol 7.4: Chaum–Pedersen

Prover (x)		**Verifier**
$r \xleftarrow{\$} \mathbb{Z}_q$; $a = g^r$, $b = h^r$	$\xrightarrow{a, b}$	
	$\xleftarrow{c}$	$c \xleftarrow{\$} \mathbb{Z}_q$
$s = r + cx$	$\xrightarrow{s}$	accept iff $g^s = a\, u^c$ and $h^s = b\, v^c$

Example 7.9. Toy group; $g = 2$, $h = 3$, $x = 7$: $u = 2^7 = 13$; $v = 3^7 = 2187 = 95 \cdot 23 + 2 \equiv 2$. Prover $r = 5$: $a = 2^5 = 9$, $b = 3^5 = 243 = 10 \cdot 23 + 13 \equiv 13$. Challenge $c = 4$: $s = 5 + 28 = 33 \equiv 0$. Check: $g^0 = 1$ and $a\, u^4 = 9 \cdot 18 = 162 = 7 \cdot 23 + 1 \equiv 1$ ✓; $h^0 = 1$ and $b\, v^4 = 13 \cdot 16 = 208 = 9 \cdot 23 + 1 \equiv 1$ ✓.

Proof as in Theorem 7.6 with $\varphi(x) = (g^x, h^x)$. *Applications:* proving correct decryption of ElGamal ($\log_g pk = \log_{c_1}(c_1^{sk})$), correct re-randomisation in mixnets, verifiable random functions (VRFs), threshold cryptography (proof that a partial decryption share is honest).

7.4 Compound statements: AND, equality, OR

7.4.1 AND and equality

To prove $(x_1, x_2) \in R_1 \wedge R_2$, run the two protocols with the same challenge. To prove two statements with the *same* witness (e.g. $y_1 = g_1^x$, $y_2 = g_2^x$) use the same response s—this is exactly Chaum–Pedersen.

7.4.2 OR proofs (Cramer–Damgård–Schoenmakers 1994)

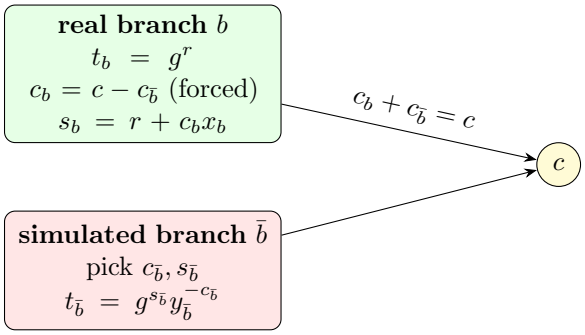
Prove: "I know w_0 with $(x_0, w_0) \in R_0$ *or* w_1 with $(x_1, w_1) \in R_1$" without revealing which. The trick: the prover simulates the branch whose witness she lacks, and splits the verifier's

challenge.

Protocol 7.5: Schnorr OR-proof (knows $\log_g y_0$ or $\log_g y_1$)

Suppose the prover knows x_b, $b \in \{0,1\}$, and let $\bar{b} = 1 - b$.

1. **Commit.** Real branch: $r \xleftarrow{\$} \mathbb{Z}_q$, $t_b = g^r$. Simulated branch: choose $c_{\bar{b}}, s_{\bar{b}} \xleftarrow{\$} \mathbb{Z}_q$, set $t_{\bar{b}} = g^{s_{\bar{b}}} y_{\bar{b}}^{-c_{\bar{b}}}$. Send (t_0, t_1).
2. **Challenge.** Verifier sends $c \xleftarrow{\$} \mathbb{Z}_q$.
3. **Respond.** $c_b = c - c_{\bar{b}}$, $s_b = r + c_b x_b$. Send (c_0, c_1, s_0, s_1).
4. **Verify.** $c_0 + c_1 \equiv c$, and $g^{s_i} = t_i y_i^{c_i}$ for $i = 0, 1$.



the verifier's c is *free*, but the prover has only one degree of freedom: whichever $c_{\bar{b}}$ she fixed in advance, c_b is determined.

Theorem 7.10. *Protocol 7.5 is a Σ-protocol for $R_0 \vee R_1$ (with $\mathcal{C} = \mathbb{Z}_q$) and is* witness indistinguishable*: the verifier's view is identically distributed for $b = 0$ and $b = 1$.*

Proof. Completeness: for the real branch Schnorr completeness; for the simulated branch $g^{s_{\bar{b}}} = t_{\bar{b}} y_{\bar{b}}^{c_{\bar{b}}}$ by construction; $c_0 + c_1 = c$. *Special soundness:* two accepting transcripts with the same (t_0, t_1) and challenges $c \neq c'$ yield (c_0, c_1), (c_0', c_1') with $c_0 + c_1 \neq c_0' + c_1'$, hence $c_i \neq c_i'$ for some i. For this i, the transcripts (t_i, c_i, s_i), (t_i, c_i', s_i') satisfy Schnorr's special soundness and give x_i. (The prover "cannot have cheated in both branches", since $c_0 + c_1$ varies with c.) *WI:* In both cases the distribution of $(t_0, t_1, c_0, c_1, s_0, s_1)$ is: c_0 uniform in $\mathbb{Z}_q$, $c_1 = c - c_0$, s_i uniform, $t_i = g^{s_i} y_i^{-c_i}$. In the real branch $c_b = c - c_{\bar{b}}$ is uniform as $c_{\bar{b}}$ is; $s_b = r + c_b x_b$ is uniform since r is. The distribution does not depend on b. □

Example 7.11 (Numerical OR-proof)**.** Toy group. Statement: $y_0 = 13$ (prover knows $x_0 = 7$) OR $y_1 = 3$ (prover pretends not to know $\log_2 3$). So $b = 0$.

- Simulated branch 1: choose $c_1 = 3$, $s_1 = 4$; $t_1 = g^4 y_1^{-3} = 16 \cdot 3^{-3} = 16 \cdot 27^{-1} = 16 \cdot 4^{-1} = 16 \cdot 6 = 96 \equiv 4$ (since $4 \cdot 6 = 24 \equiv 1$).
- Real branch 0: $r = 5$, $t_0 = 2^5 = 9$. Send $(t_0, t_1) = (9, 4)$.
- Verifier's $c = 8$. Then $c_0 = c - c_1 = 5$, $s_0 = r + c_0 x_0 = 5 + 35 = 40 \equiv 7$.
- Verify: $c_0 + c_1 = 8$ ✓; $g^{s_0} = 2^7 = 13$ and $t_0 y_0^{c_0} = 9 \cdot 13^5 = 9 \cdot 4 = 36 \equiv 13$ ✓($13^5 = 13^4 \cdot 13 = 18 \cdot 13 = 234 \equiv 4$); $g^{s_1} = 2^4 = 16$ and $t_1 y_1^{c_1} = 4 \cdot 27 \equiv 4 \cdot 4 = 16$ ✓.

Example 7.12 (Proving a committed bit; building block for range proofs)**.** Let $C = g^b h^r$ be a Pedersen commitment. To show $b \in \{0,1\}$ prove "$C = h^r$ or $C/g = h^r$"—an OR of two Schnorr proofs w.r.t. base h for statements C and Cg^{-1}. Applying it to bits $b_0..b_{n-1}$ and showing $C = \prod C_i^{2^i}$ (using the homomorphism) proves C commits to a value in $[0, 2^n)$: a range proof of size $O(n)$ (Bulletproofs improve this to $O(\log n)$, Chapter 11).

Remark 7.13 (Threshold / general access structures). CDS generalise OR to "know witnesses for k of n statements" by sharing the verifier's challenge via Shamir secret sharing: the simulated branches get $n-k$ free shares which determine a degree-$(n-k)$ polynomial f with $f(0) = c$. This yields ring signatures (Rivest–Shamir–Tauman; Abe–Ohkubo–Suzuki) and privacy-preserving credentials.

7.5 From identification to signatures: preview

A Schnorr transcript (t, c, s) where $c = H(t, m)$ is a signature on m. Security and the forking lemma are the subject of Chapter 8.

Exercises

Exercise 7.14. Show that Schnorr's protocol is not special-sound if $\mathcal{C}$ is allowed to contain c, c' with $c - c'$ not invertible mod the group order (e.g. over $\mathbb{Z}_{pq}$).

Exercise 7.15. Prove that if the prover reuses the nonce r across two proofs with different challenges, the secret key is revealed. (This is how Sony's PS3 ECDSA key was extracted.)

Exercise 7.16. Design a Σ-protocol for "$y_1 = g^x$ and $y_2 = h^{x^2}$" (hint: Pedersen-commit to x, prove a multiplicative relation).

Exercise 7.17. Write the 1-out-of-3 OR proof for Schnorr statements and prove WI. How does the challenge splitting generalise?

Exercise 7.18. Prove that the Okamoto protocol is perfectly HVZK and that, for a given y, a prover who can produce two different representations of y can compute $\log_g h$.

Chapter 8

Fiat–Shamir, Random Oracles and Non-Interactive Zero-Knowledge

Interaction is costly: a verifier must be online, and a transcript convinces only the verifier who took part. Can the challenge be generated without a verifier? Fiat and Shamir (1986) suggested: let the prover compute the challenge herself as a hash of everything said so far.

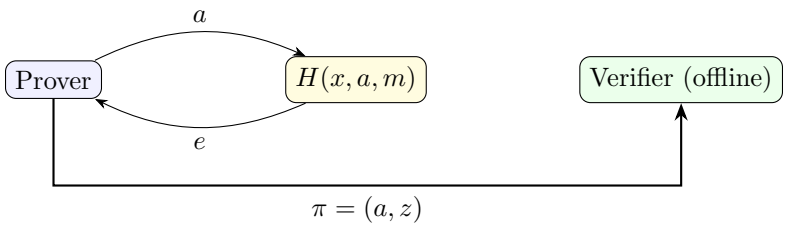
8.1 The Fiat–Shamir transform

Let (P, V) be a public-coin Σ-protocol with transcript (a, e, z) and $H : \{0,1\}^* \to \mathcal{C}$ a hash function.

Algorithm 8.1: Fiat–Shamir transform (non-interactive proof of knowledge)

Prove$(x, w;\, m)$: compute $a \leftarrow \mathsf{P}(x, w)$; set $e := H(x, a, m)$; compute $z \leftarrow \mathsf{P}(x, w, a, e)$; output $\pi = (a, z)$ (or (e, z)).
Verify$(x, \pi = (a, z);\, m)$: $e := H(x, a, m)$; accept iff $\phi(x, a, e, z) = 1$.

The optional message m turns the proof into a *signature of knowledge / signature on* m.



Example 8.1 (Schnorr signature, toy group)**.** Key: $x = 7$, $y = 13$. Message m. Sign: $r = 8$, $t = 3$; suppose $H(t \| m) = c = 4 \pmod{11}$. Then $s = r + cx = 3$; signature $(t, s) = (3, 3)$ (or (c, s)). Verify: recompute $c = H(3 \| m) = 4$; check $g^3 = 8$ equals $t\, y^c = 3 \cdot 18 = 54 \equiv 8$. ✓In the compact form $(c, s) = (4, 3)$ the verifier recomputes $t' = g^s y^{-c} = 8 \cdot 18^{-1} = 8 \cdot 9 = 72 \equiv 3$ (as $18 \cdot 9 = 162 \equiv 1$) and checks $H(t' \| m) = c$. Anyone can verify; nobody can forge without x (see below).

8.2 Security in the random oracle model

Theorem 8.2 (FS preserves ZK and soundness in the ROM)**.** *Let Π be a Σ-protocol with SHVZK and special soundness and with $\mathcal{C}$ super-polynomially large. Then FS(Π) in the ROM is*

1. ***Non-interactive zero-knowledge**: there is a simulator that, with the power to program H, outputs proofs indistinguishable from real;*
2. ***Sound / proof of knowledge**: for every PPT adversary $\mathcal{A}$ making Q random-oracle queries and outputting an accepting (x, π) with probability ε, there is an extractor which outputs a witness with probability roughly $\varepsilon^2/Q - \varepsilon/|\mathcal{C}|$ (forking lemma).*

Proof of ZK part. Simulator: on a request to prove x (for $x \in L$), sample $e \xleftarrow{\$} \mathcal{C}$, run the SHVZK simulator to get (a, z), and *program* $H(x, a, m) := e$. If H was already queried on that input (probability $\leq Q/$(entropy of a), negligible if a has superlogarithmic min-entropy) abort. Because e is uniform, $H(x, a, m) = e$ is a perfectly valid lazily-sampled answer, and (a, e, z) has the real distribution by SHVZK. □

8.2.1 The forking lemma

The extractor must obtain two accepting transcripts with the same a but different e from a non-interactive adversary: it *reruns the adversary with the oracle answers changed from the critical query onward.*

Lemma 8.3 (General forking lemma, Bellare–Neven)**.** *Let $\mathcal{A}$ be a randomized algorithm that takes random coins ρ and oracle answers $h_1, \ldots, h_Q \in \mathcal{C}$ ($|\mathcal{C}| = N$) and outputs either $\perp$ or a pair (I, out) with $I \in [Q]$. Let* $\mathrm{acc} = \Pr[\mathcal{A} \neq \perp]$ *over uniform $\rho, \vec{h}$. The forking algorithm $\mathcal{F}_{\mathcal{A}}$: pick ρ, $h_1..h_Q$; run $\mathcal{A}$; if $\perp$ return $\perp$; else let I be its index; pick fresh $h'_I..h'_Q$ and rerun $\mathcal{A}(\rho; h_1..h_{I-1}, h'_I..h'_Q)$; if it returns the same index I and $h_I \neq h'_I$ output success. Then*

$$\mathrm{frk} := \Pr[\mathcal{F}_{\mathcal{A}} \textit{ succeeds}] \;\geq\; \mathrm{acc}\Big(\frac{\mathrm{acc}}{Q} - \frac{1}{N}\Big).$$

Proof. Let S_i be the set of $(\rho, \vec{h})$ on which $\mathcal{A}$ succeeds with index i, so $\Pr[S_i]$ sums over i to acc. Fix i and a prefix $(\rho, h_1, \ldots, h_{i-1})$ and let

$$\alpha_i(\rho, h_{<i}) = \Pr_{h_i, \ldots, h_Q}\big[(\rho, \vec{h}) \in S_i\big].$$

Conditioned on the prefix, the two runs of $\mathcal{F}_{\mathcal{A}}$ use independent suffixes, so both land in S_i with probability α_i^2. The probability that additionally $h_i = h'_i$ is at most α_i/N (given the first run, the fresh h'_i equals h_i with probability $1/N$). Hence the probability of a successful fork at index i is at least $\alpha_i^2 - \alpha_i/N$. Averaging over prefixes and using Jensen's inequality $\mathbb{E}[\alpha_i^2] \geq \mathbb{E}[\alpha_i]^2$ with $\beta_i := \mathbb{E}[\alpha_i] = \Pr[S_i]$:

$$\mathrm{frk} \;\geq\; \sum_{i=1}^{Q} \Big(\beta_i^2 - \frac{\beta_i}{N}\Big) \;\geq\; \frac{(\sum_i \beta_i)^2}{Q} - \frac{\sum_i \beta_i}{N} = \frac{\mathrm{acc}^2}{Q} - \frac{\mathrm{acc}}{N},$$

where we used Cauchy–Schwarz, $\sum_i \beta_i^2 \geq (\sum_i \beta_i)^2/Q$. □

Intuition

Run the cheater once. Find the oracle query that determined its commitment a. Wind time back to just before that query and give a *different* random answer. If the cheater still produces a valid proof (about $1/Q$ of the time it is "the same query" again), you hold two proofs of the same a under different challenges—special soundness extracts w. The price is the quadratic loss ε^2/Q: forking proofs are *not tight*.

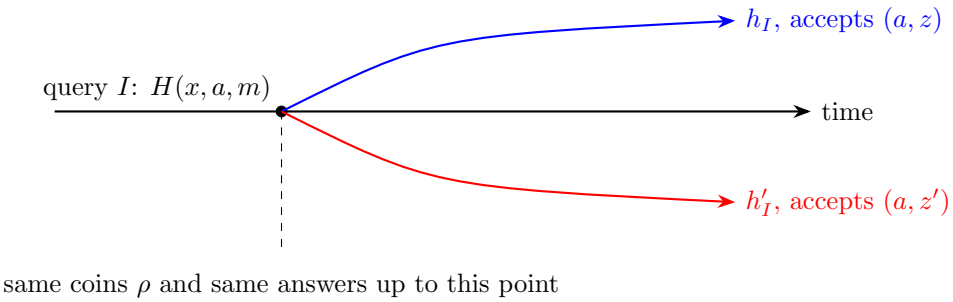


Figure 8.1: Forking: two executions diverge at the critical oracle query.

Corollary 8.4 (EUF-CMA security of Schnorr signatures in the ROM)**.** *If DL is hard in* $\mathbb{G}$*, Schnorr signatures are existentially unforgeable under chosen message attacks in the random oracle model (Pointcheval–Stern 1996, with signing-oracle queries answered by the ZK simulator that programs* H*).*

Idea. Given a DL instance y, set the public key to y. Answer signing queries via the simulator (no secret needed); answer H queries lazily and consistently. When the forger outputs a forgery (t, s) on a fresh m^*, fork on the query $H(t\|m^*)$ to obtain another accepting transcript with the same t and a different c; special soundness yields $x = \operatorname{dlog}_g y$. □

Pitfalls: "weak" Fiat–Shamir and the Frozen Heart family

- **Hash the statement!** The challenge must bind the *statement* x (and all public parameters) as well as the commitment. Omitting x ("weak FS") allows a prover to pick (a, e, z) first and then solve for a statement x consistent with it (e.g. Schnorr: pick s, c, set $y = (g^s/t)^{1/c}$ for any t). This is exactly the class of bugs known as *Frozen Heart* in several zk-SNARK/Bulletproofs implementations (2022).
- **Nonce reuse/bias.** The randomness r of Schnorr/ECDSA must be fresh (or derived deterministically: RFC 6979/EdDSA). Reuse leaks x.
- **Multi-round protocols.** For $(2\mu + 1)$-move protocols, *every* challenge must hash the full transcript so far ("strong FS"); security then requires *round-by-round soundness* or $(k_1, \ldots, k_\mu)$-special soundness; the knowledge error grows with the number of rounds (Attema–Fehr–Klooß).
- **Grinding.** If the proof has soundness error $2^{-\kappa}$ per round and the adversary can retry, repeat/grind; parameters must absorb the adversary's number of hash queries ($\kappa + \log Q$ bits).
- **Not secure for all protocols.** There exist (contrived) protocols secure interactively whose FS compilation is insecure for every instantiation of H [35]; for natural protocols, FS from correlation-intractable hash functions (Canetti et al. 2019, Peikert–Shiehian) is secure from standard assumptions such as LWE.

8.3 NIZK in the common reference string model

The CRS model assumes a trusted setup that samples a string crs from some distribution known to all.

Definition 8.5 (NIZK)**.** A tuple $(\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$ is a NIZK argument/proof for R if:

Completeness $\Pr[\mathsf{Verify}(\mathsf{crs}, x, \mathsf{Prove}(\mathsf{crs}, x, w)) = 1] = 1$ for $(x, w) \in R$.

Soundness For all (PPT) $\mathcal{A}$, $\Pr[\mathsf{Verify}(\mathsf{crs}, x, \pi) = 1 \wedge x \notin L] \le \mathsf{negl}$.

Zero-knowledge There is a PPT $\mathsf{Sim} = (\mathsf{Sim}_1, \mathsf{Sim}_2)$ with $(\mathsf{crs}_{sim}, \tau) \leftarrow \mathsf{Sim}_1(1^\lambda)$, $\pi \leftarrow \mathsf{Sim}_2(\mathsf{crs}_{sim}, \tau, x)$ so that $(\mathsf{crs}, \mathsf{Prove}(\mathsf{crs}, x, w)) \approx_c (\mathsf{crs}_{sim}, \mathsf{Sim}_2(\mathsf{crs}_{sim}, \tau, x))$.

Knowledge soundness (optional) An extractor $\mathsf{Ext}_{\mathsf{crs}}$ with a trapdoor outputs w from any accepting proof.

Remark 8.6. The simulator *chooses the CRS*: it knows a trapdoor τ that the real prover does not. Whoever knows τ can forge proofs for false statements; hence the setup ceremony must be trusted or run as a multi-party computation (Zcash Powers of Tau). NIZK is impossible in the plain model for nontrivial languages (Goldreich–Oren): a deterministic non-interactive verifier lets the simulator output a proof, then a decider for L exists.

8.3.1 The hidden-bits paradigm (Feige–Lapidot–Shamir)

The classical construction (Blum–Feldman–Micali 1988; Feige–Lapidot–Shamir 1990) proceeds in two steps.

Step 1: the hidden-bits model (HBM). The CRS is a uniformly random string $r \in \{0,1\}^m$ which the *prover* sees but the verifier does not. The prover sends a proof π together with a set $I \subseteq [m]$ of positions to *reveal*; the verifier learns only $(r_i)_{i \in I}$ and checks (x, π, I, r_I). In this model one can mimic Blum's Hamiltonicity protocol without any commitments: the hidden random bits play the role of commitments that are *automatically* binding (the prover cannot change r) and hiding (the verifier cannot see the unrevealed bits). The prover uses blocks of r as adjacency matrices of random graphs, selects a block that happens to be the adjacency matrix of a Hamiltonian cycle on a relabelled copy of G, and reveals exactly the positions of the cycle edges. Zero-knowledge holds because the revealed bits are all 1s, and a simulator can choose (I, r_I) first and fill the unrevealed bits at random. (The technical details of how blocks are generated are in [26], and Goldreich, *Foundations of Cryptography Vol. 1*, §4.10.)

Step 2: from hidden bits to a real CRS. Use a trapdoor permutation f with hard-core predicate B. The CRS is a random string $\sigma = (\sigma_1, \dots, \sigma_m)$ with σ_i in the domain of f. The prover generates (f, f^{-1}) herself and defines the hidden bits $r_i := B(f^{-1}(\sigma_i))$. To reveal position $i \in I$, she sends the preimage $\rho_i = f^{-1}(\sigma_i)$; the verifier checks $f(\rho_i) = \sigma_i$ and computes $r_i = B(\rho_i)$. The other bits $B(f^{-1}(\sigma_j))$, $j \notin I$, stay computationally hidden by the hard-core property, and they are fixed by σ, so the prover cannot change them (binding). The verifier must also be convinced that f is a permutation (for RSA this needs an extra argument, e.g. a certified key).

Theorem 8.7 (FLS90)**.** *If trapdoor permutations exist (e.g. RSA), every $L \in \mathsf{NP}$ has a (statistical) NIZK argument with perfect completeness and the prover efficient given w.*

Theorem 8.8 (Groth–Ostrovsky–Sahai 2006; Groth–Sahai 2008)**.** *Pairing-based NIZK for satisfiability of pairing-product equations exists under DLIN/SXDH: practical for proving statements such as "this ciphertext decrypts to a valid signature", without going through NP-reductions. Proofs are linear in the number of variables.*

Theorem 8.9 (Peikert–Shiehian 2019; Canetti et al. 2019)**.** *NIZK for* NP *exists from LWE, in the CRS (or even the common random string) model, by instantiating FS with a correlation-intractable hash function.*

8.4 Interactive vs. non-interactive: a summary

	Interactive ZK	NIZK
Prover/verifier	online both	prover offline; verifier any time
Transferability	transcript deniable (simulatable)	proof can be forwarded/verified publicly
Setup	none	CRS or random oracle
Soundness against	unbounded or PPT provers	PPT (arguments) typically
Example	Schnorr ID, GMW 3-COL	Schnorr signature, Groth16, STARK

Exercises

Exercise 8.10. Show that Schnorr signatures with hash $H(t\|m)$ are insecure if m is replaced by a hash that omits t (i.e. $c = H(m)$).

Exercise 8.11. Prove that the weak Fiat–Shamir variant $c = H(t)$ (statement omitted) allows forging proofs for *some* statement of the adversary's choice. Give the attack for Schnorr.

Exercise 8.12. Work through the forking lemma with $Q = 1$, and conclude that in this special case $\mathrm{frk} \geq \mathrm{acc}^2 - \mathrm{acc}/N$. Compare with the extraction in Theorem 7.2.

Exercise 8.13. Using your answer to the previous exercise, estimate the security loss: if the best attacker against the DL problem in the group runs in time 2^{128} and $Q = 2^{60}$, what bit security can the forking-lemma reduction guarantee for Schnorr signatures?

Chapter 9

The Sum-Check Protocol and Interactive Proofs for Computation

From here on our goal changes: we want proofs that are *succinct*—much shorter and faster to verify than the computation they certify. The first tool is the sum-check protocol of Lund, Fortnow, Karloff and Nisan (1990), the engine behind IP = PSPACE, GKR, and many modern SNARKs (Spartan, HyperPlonk, Lasso/Jolt).

9.1 Multivariate polynomials and multilinear extensions

Definition 9.1 (Multilinear extension)**.** For $f : \{0,1\}^v \to \mathbb{F}$ the *multilinear extension* (MLE) is the unique polynomial $\tilde{f} \in \mathbb{F}[X_1..X_v]$ of degree ≤ 1 in each variable agreeing with f on $\{0,1\}^v$:

$$\tilde{f}(\vec{X}) = \sum_{\vec{b}\in\{0,1\}^v} f(\vec{b}) \cdot \chi_{\vec{b}}(\vec{X}), \qquad \chi_{\vec{b}}(\vec{X}) = \prod_{i=1}^{v} \big(b_i X_i + (1-b_i)(1-X_i)\big).$$

Example 9.2. $f : \{0,1\}^2 \to \mathbb{F}$ with $f(00) = 6$, $f(01) = 9$, $f(10) = 1$, $f(11) = 4$. Then $\tilde{f}(X_1, X_2) = 6(1-X_1)(1-X_2) + 9(1-X_1)X_2 + X_1(1-X_2) + 4X_1X_2$. At $(X_1, X_2) = (2, 3)$: $6(-1)(-2) + 9(-1)(3) + 2(-2) + 4 \cdot 6 = 12 - 27 - 4 + 24 = 5$.

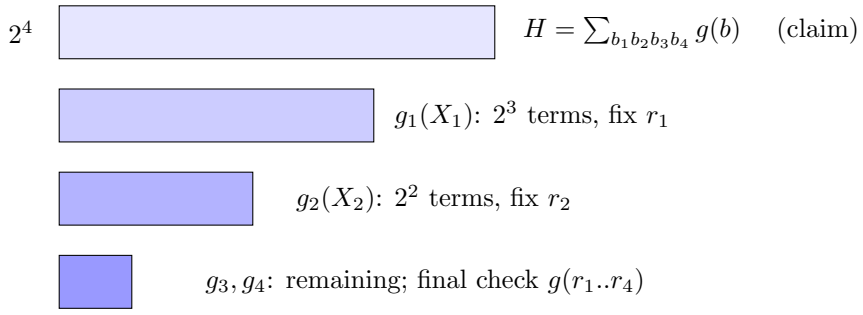
Uniqueness: a multilinear polynomial vanishing on $\{0,1\}^v$ is zero (induction on v). An MLE of a vector of length 2^v is evaluable at any point in $O(2^v)$ operations. Key advantage over univariate encodings: *no FFT is required*, and evaluations on the Boolean hypercube correspond to the original data.

9.2 The protocol

Claim: $H = \sum_{\vec{b}\in\{0,1\}^v} g(\vec{b})$ where $g \in \mathbb{F}[X_1..X_v]$ has degree $\leq d$ in each variable. The verifier can evaluate g at a single random point (via an oracle, or through a commitment, or via recursion).

Protocol 9.1: Sum-check

Round 1. P sends univariate $g_1(X) = \sum_{\vec{b}\in\{0,1\}^{v-1}} g(X, \vec{b})$. V checks $g_1(0) + g_1(1) = H$, picks $r_1 \xleftarrow{\$} \mathbb{F}$, sends it.
Round i **(**$2 \le i \le v$**).** P sends $g_i(X) = \sum_{\vec{b}\in\{0,1\}^{v-i}} g(r_1, \ldots, r_{i-1}, X, \vec{b})$. V checks $g_i(0) + g_i(1) = g_{i-1}(r_{i-1})$, picks r_i.
Final check. V checks $g_v(r_v) = g(r_1, \ldots, r_v)$ (one evaluation of g).
(Each g_i has degree $\le d$, so is sent as $d+1$ field elements.)



Theorem 9.3 (Completeness and soundness)**.** *The protocol has perfect completeness. If* $H \neq \sum_{\vec{b}} g(\vec{b})$*, then for every prover* P***, V *accepts with probability at most* $vd/|\mathbb{F}|$*.*

Proof. Completeness is immediate from $g_i(0) + g_i(1) = \sum_{b_i} \sum_{\vec{b'}} g(r_{<i}, b_i, \vec{b'}) = g_{i-1}(r_{i-1})$.

Soundness by induction on v. For $v = 1$: P* sends some polynomial g_1^* of degree $\le d$ with $g_1^*(0) + g_1^*(1) = H$ (otherwise reject). Since $H \neq g(0) + g(1)$, $g_1^* \neq g$ as polynomials, so they agree on at most d points: $\Pr_{r_1}[g_1^*(r_1) = g(r_1)] \le d/|\mathbb{F}|$, and the final check passes only then. For $v > 1$: let g_1^* be the first message. If $g_1^* = g_1$ (the true polynomial) then $g_1(0) + g_1(1) = \sum_{\vec{b}} g(\vec{b}) \neq H$ and the verifier rejects at once. So $g_1^* \neq g_1$, and with probability $\ge 1 - d/|\mathbb{F}|$ over r_1, $g_1^*(r_1) \neq g_1(r_1) = \sum_{\vec{b'}} g(r_1, \vec{b'})$. Then the remaining $(v-1)$-round protocol is on a *false* claim about $g(r_1, \cdot)$, which by the inductive hypothesis is accepted with probability $\le (v-1)d/|\mathbb{F}|$. A union bound gives $vd/|\mathbb{F}|$. □

Example 9.4. Let $g(x_1, x_2, x_3) = 2x_1^3 + x_1x_3 + x_2x_3$ over a large field (here integers suffice for the arithmetic). The true sum over $\{0,1\}^3$ is $H = 12$: on Boolean points $x_1^3 = x_1$ so the $2x_1$ term contributes $2 \cdot 4 = 8$, x_1x_3 contributes 2 and x_2x_3 contributes 2.

1. P: $g_1(X) = \sum_{x_2,x_3} g(X, x_2, x_3) = 8X^3 + 2X + 1$. V: $g_1(0) + g_1(1) = 1 + 11 = 12 = H$ ✓. $r_1 = 2$; $g_1(2) = 64 + 4 + 1 = 69$.
2. P: $g_2(X) = \sum_{x_3} g(2, X, x_3) = g(2, X, 0) + g(2, X, 1) = 16 + (16 + 2 + X) = 34 + X$. V: $g_2(0) + g_2(1) = 34 + 35 = 69 = g_1(2)$ ✓. $r_2 = 3$; $g_2(3) = 37$.
3. P: $g_3(X) = g(2, 3, X) = 16 + 2X + 3X = 16 + 5X$. V: $g_3(0) + g_3(1) = 16 + 21 = 37$ ✓. $r_3 = 5$; $g_3(5) = 41$.
4. Final: V computes $g(2, 3, 5) = 16 + 10 + 15 = 41$. ✓

A cheating prover claiming $H' = 13$ must send a g_1^* with $g_1^*(0) + g_1^*(1) = 13$, hence $g_1^* \neq g_1$; it is caught unless r_1 is one of the ≤ 3 roots of $g_1^* - g_1$.

Theorem 9.5 (Efficiency)**.** *The verifier does* $O(vd)$ *field operations plus one evaluation of* g *and communicates* $O(vd)$ *elements. If* g *is given as a table of* 2^v *values (e.g. products of MLEs of tables), the prover can be implemented in time* $O(2^v \cdot \mathrm{poly}(d))$ *by halving the table in each round (Thaler/Libra/Virgo).*

9.3 Application 1: #SAT, so coNP $\subseteq$ IP

Arithmetise a 3-CNF φ with polynomial p_φ: variable $x \mapsto x$; $\neg x \mapsto 1-x$; clause $\ell_1 \vee \ell_2 \vee \ell_3 \mapsto 1 - \prod_j (1-\ell_j)$; $\varphi = \bigwedge_k C_k \mapsto \prod_k p_{C_k}$. On Boolean inputs $p_\varphi(\vec{b}) = \varphi(\vec{b}) \in \{0,1\}$, hence $\#\mathrm{SAT}(\varphi) = \sum_{\vec{b}} p_\varphi(\vec{b})$, with deg $\leq m$ (the number of clauses) in total and per variable $\leq$ clause occurrences.

Example 9.6. $\varphi = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_2)$. $p = \big(x_1 + x_2 - x_1x_2\big)\big(1 - x_1 + x_1x_2\big)$, per-variable degree 2. Boolean evaluations: $p(00) = 0$, $p(01) = 1$, $p(10) = 0$, $p(11) = 1$, so #SAT $= 2$ $(x_2 = 1)$.
Sum-check over $\mathbb{F}_{17}$: $g_1(X) = p(X,0) + p(X,1) = X(1-X) + 1 = 1 + X - X^2$; $g_1(0) + g_1(1) = 1 + 1 = 2$ ✓. $r_1 = 5$: $g_1(5) = 1 + 5 - 25 = -19 \equiv 15$. $g_2(X) = p(5,X) = (5 - 4X)(-4 + 5X) = -20 + 41X - 20X^2$; $g_2(0) + g_2(1) = -20 + 1 = -19 \equiv 15$ ✓. $r_2 = 3$: $g_2(3) = -20 + 123 - 180 = -77 \equiv 8$. Final: $p(5,3) = (5 + 3 - 15)(1 - 5 + 15) = (-7)(11) = -77 \equiv 8$ ✓.

Since coNP-complete UNSAT $= \{\varphi : \#\mathrm{SAT}(\varphi) = 0\}$, we get coNP $\subseteq$ IP—a result that astonished the community in 1989–90 because coNP proofs were thought to need exponential length. For IP = PSPACE one must handle the degree blow-up in the quantifier arithmetisation (LFKN's "degree-reduction" linearisation operators).

9.4 Application 2: the proof of IP = PSPACE

Since TQBF (true quantified Boolean formulas) is PSPACE-complete, it suffices to give an interactive proof for $\Phi = \exists x_1 \forall x_2 \exists x_3 \cdots \psi(x_1, \ldots, x_n)$ with ψ a 3-CNF.

Arithmetisation. Replace ψ by its polynomial p_ψ (Section 9.3) and quantifiers by operators acting on polynomials:

$$\exists x\, p \;\mapsto\; \textstyle\sum_{x\in\{0,1\}} p = p|_{x=0} + p|_{x=1}, \qquad \forall x\, p \;\mapsto\; \prod_{x\in\{0,1\}} p = p|_{x=0} \cdot p|_{x=1}.$$

On Boolean inputs $p_\psi \in \{0,1\}$, so Φ is true iff the resulting integer $V = \sum_{x_1} \prod_{x_2} \sum_{x_3} \cdots p_\psi(\vec{x})$ is nonzero.

Example 9.7. $\Phi = \exists x \forall y\, (x \vee y)$ with $p_\psi = x + y - xy$. The $\forall y$ step gives $p_\psi(x,0)p_\psi(x,1) = x \cdot 1 = x$; the $\exists x$ step gives $0 + 1 = 1 \neq 0$. So Φ is true $(x = 1)$.

The obstacle. Each $\prod$ *squares the degree* in the remaining variables, so after n quantifiers the degree is exponential and the prover's univariate polynomials cannot be sent. **The fix (Shamir):** insert *linearisation operators*

$$L_x p = (1 - x)\, p|_{x=0} + x\, p|_{x=1},$$

which return the multilinear polynomial that agrees with p on $\{0,1\}$ in x and thus does not change any Boolean value. Applying L_{x_j} for all j after each quantifier keeps every variable at degree ≤ 1 in the intermediate polynomials (e.g. $p = x^2 \Rightarrow L_x p = x$, which agrees with x^2 on $\{0,1\}$).

Protocol 9.2: Shamir's protocol (outline)

The claim is $v = O_1 O_2 \cdots O_N\, p_\psi$ where each $O_k \in \{\sum_x, \prod_x, L_x\}$ ($N = O(n^2)$ operators). Round k peels off the leftmost remaining operator O_k acting on variable x:

1. P sends the univariate polynomial $g(X)$ obtained by evaluating the remaining

expression with $x := X$ (and previous variables fixed to the verifier's random points $r_1, \dots$).

2. V checks consistency with the current claim v: $g(0) + g(1) = v$ for $\sum$; $g(0)g(1) = v$ for $\prod$; $(1-r)g(0) + r\,g(1) = v$ for L_x where r is the current value of x.
3. V picks a fresh random $r' \in \mathbb{F}$, sets the new claim $v' := g(r')$ for the remaining operators with $x := r'$ (for L_x the verifier first sends r' and the prover answers with g as above).

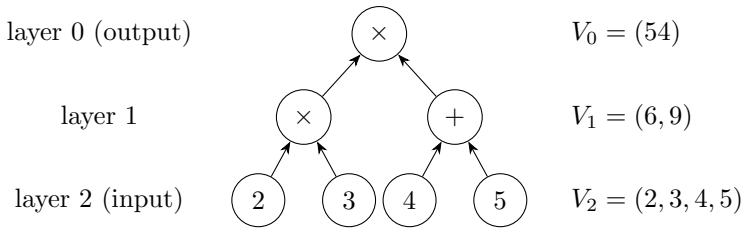
At the end the verifier evaluates $p_\psi(r_1, .., r_n)$ itself and compares.

Theorem 9.8 (LFKN/Shamir)**.** *Over a field $|\mathbb{F}| \geq 2^n$ the protocol is complete and has soundness error $O(n^2 m/|\mathbb{F}|)$, with m the number of clauses, since each of the $O(n^2)$ rounds is a sum-check step with polynomial degree $\leq \max(2, \deg_x p_\psi) \leq m$. Hence* PSPACE $\subseteq$ IP*; with* IP $\subseteq$ PSPACE *(the honest prover's optimal strategy can be computed in polynomial space) we obtain* IP = PSPACE*.*

Proof of soundness. Exactly as in Theorem 9.3: if the current claim v is false, the polynomial g^* sent by a cheating prover differs from the true g (otherwise the consistency check fails at once); two distinct polynomials of degree $\leq m$ agree on at most m points, so with probability $\geq 1 - m/|\mathbb{F}|$ the new claim $g^*(r')$ is again false. A union bound over the $N = O(n^2)$ rounds, and the fact that a false initial claim implies a false last claim, gives the bound. □

9.5 Application 3: GKR—delegating computation

Goldwasser, Kalai and Rothblum (2008) verify a layered arithmetic circuit C of size S, depth D, fan-in 2 with a prover of time $\tilde{O}(S)$ and verifier of time $\tilde{O}(n + D \log S)$—far less than S when C is low-depth and "regular".



Layer function. Let $V_i : \{0,1\}^{k_i} \to \mathbb{F}$ give the gate values at layer i and $\mathrm{add}_i, \mathrm{mult}_i : \{0,1\}^{k_i + 2k_{i+1}} \to \{0,1\}$ be wiring predicates (1 iff gate z at layer i is an add/mult of gates x, y at layer $i+1$). Then for all z,

$$V_i(z) = \sum_{x,y \in \{0,1\}^{k_{i+1}}} \Big[\mathrm{add}_i(z,x,y)\big(V_{i+1}(x) + V_{i+1}(y)\big) + \mathrm{mult}_i(z,x,y) V_{i+1}(x) V_{i+1}(y) \Big].$$

Replacing every function by its MLE extends this identity to all $z \in \mathbb{F}^{k_i}$: $\tilde{V}_i(z) = \sum_{x,y} f_z(x,y)$ where f_z is a low-degree polynomial in (x,y).

Algorithm 9.3: GKR (outline)

1. P sends claimed output $\tilde{V}_0$; V picks random z_0 and obtains the claim $\tilde{V}_0(z_0) = m_0$.
2. For $i = 0, \dots, D-1$: run sum-check on $\tilde{V}_i(z_i) = \sum_{x,y} f_{z_i}(x,y)$. At the end the verifier needs $\tilde{V}_{i+1}(x^*)$ and $\tilde{V}_{i+1}(y^*)$ at two random points. The prover claims two values; to avoid doubling the claims at each layer, V sends a line $\ell(t)$ through x^*, y^*, P sends $\tilde{V}_{i+1} \circ \ell$ (degree $\le k_{i+1}$), V checks consistency at $t = 0, 1$, picks t^* and defines $z_{i+1} = \ell(t^*)$ with claim $m_{i+1} = \tilde{V}_{i+1} \circ \ell(t^*)$.
3. At the input layer, V computes $\tilde{V}_D(z_D)$ directly from the inputs (an MLE evaluation, $O(n)$) and compares with m_D.

Example 9.9 (One layer of GKR on the circuit above)**.** Output $V_0 = 54$. The single output gate multiplies gates 0 and 1 of layer 1, so $\mathrm{mult}_0(0,x,y) = (1-x)\,y$ for $x, y \in \{0,1\}$ and $\tilde{V}_1(x) = 6(1-x)+9x = 6+3x$. The identity says $54 = \sum_{x,y\in\{0,1\}} (1-x)y\,(6+3x)(6+3y)$ (only $(x,y) = (0,1)$ survives: $6 \cdot 9 = 54$ ✓). Run sum-check on $f(x,y) = (1-x)y(6+3x)(6+3y)$:

- $g_1(X) = f(X,0) + f(X,1) = 9(1-X)(6+3X) = 54 - 27X - 27X^2$, and $g_1(0) + g_1(1) = 54 + 0 = 54$ ✓. $r_1 = 2$: $g_1(2) = -108$.
- $g_2(Y) = f(2,Y) = (1-2)\,Y\,(12)(6+3Y) = -72Y - 36Y^2$; $g_2(0) + g_2(1) = -108$ ✓. $r_2 = 3$: $g_2(3) = -216 - 324 = -540$.
- Final: V needs $\mathrm{mult}_0(0,2,3) = (1-2)\cdot 3 = -3$ (computable by itself), $\tilde{V}_1(2) = 12$, $\tilde{V}_1(3) = 15$ (claimed by the prover): $-3 \cdot 12 \cdot 15 = -540$ ✓. The claims about $\tilde{V}_1$ at 2 and 3 are reduced (via the line trick) to a single claim about $\tilde{V}_1$ at one random point, which feeds layer 1's sum-check, and so on until the inputs.

Theorem 9.10 (GKR 2008)**.** *For every log-space uniform fan-in-two layered circuit of depth D, size S, with n inputs, there is a public-coin interactive proof with soundness error $O(D \log S/|\mathbb{F}|)$, $O(D \log S)$ rounds, communication $O(D \log S)$ field elements, verifier time $\tilde{O}(n + D)$ and prover time* $\mathrm{poly}(S)$ *(quasi-linear $O(S \log S)$ for regular circuits [14]).*

Via Fiat–Shamir, GKR gives non-interactive proofs; combined with a polynomial commitment for the inputs and zero-knowledge masking, one obtains a zk-SNARK (Libra, Virgo, Spartan).

Exercises

Exercise 9.11. Run the sum-check protocol for $g(x_1, x_2) = x_1 x_2 + 3x_2$ over $\mathbb{F}_{11}$ with $H = \sum g$, using challenges $r_1 = 4, r_2 = 7$. Then simulate a cheating prover that claims $H' = H + 1$ and find which r_1 let it succeed.

Exercise 9.12. Prove that the MLE of the equality function $\mathrm{eq}(x,y) = \prod_i (x_i y_i + (1-x_i)(1-y_i))$ is itself; deduce $\tilde{f}(r) = \sum_b f(b)\,\mathrm{eq}(b,r)$.

Exercise 9.13. Show that the sum-check prover can be implemented in $O(2^v)$ time for $g = \tilde{f} \cdot \tilde{h}$ when f, h are given as tables (hint: "bookkeeping tables" which are folded by $T[b] \leftarrow (1-r)T[0b] + rT[1b]$ after each round).

Exercise 9.14. Verify that the exact degree bound matters: why would $\mathbb{F} = \mathbb{F}_2$ break the soundness argument?

Chapter 10

Arithmetisation: From Programs to Polynomial Identities

To prove statements about computation we convert them into algebraic constraints over a large finite field $\mathbb{F}$ (typically $|\mathbb{F}| \approx 2^{254}$ or 2^{64} with extension fields). This process is called *arithmetisation*. Its aim is to turn "this program ran correctly" into "these polynomials satisfy certain identities", which Schwartz–Zippel allows us to check at a single random point.



10.1 Arithmetic circuits and constraint systems

Definition 10.1 (Arithmetic circuit)**.** A DAG whose input nodes are labelled by variables or constants of $\mathbb{F}$ and whose inner nodes are $+$ or $\times$ gates. CircuitSAT: given C and public input $\vec{x}$ does there exist $\vec{w}$ with $C(\vec{x}, \vec{w}) = 0$?

Every Boolean statement is embedded by means of the constraint $b(1-b) = 0$ ($b \in \{0,1\}$): $\mathrm{AND}(a,b) = ab$, $\mathrm{NOT}(a) = 1 - a$, $\mathrm{XOR}(a,b) = a + b - 2ab$, $\mathrm{OR}(a,b) = a + b - ab$. Comparison and range proofs use bit decomposition: $v = \sum_{i<n} 2^i b_i$ with $b_i(1-b_i) = 0$ proves $0 \le v < 2^n$ (if $2^n < |\mathbb{F}|$).

Definition 10.2 (R1CS)**.** A *Rank-1 Constraint System* over $\mathbb{F}$ with m variables and d constraints is a triple of matrices $A, B, C \in \mathbb{F}^{d \times m}$. A vector $\vec{a} = (1, \vec{x}, \vec{w}) \in \mathbb{F}^m$ (first entry the constant 1, then public inputs, then private witness) *satisfies* it if for every row $j \in [d]$

$$\Big(\sum_i A_{ji} a_i\Big) \cdot \Big(\sum_i B_{ji} a_i\Big) = \sum_i C_{ji} a_i, \qquad \text{i.e. } (A\vec{a}) \circ (B\vec{a}) = C\vec{a}.$$

Each constraint is "(linear form) $\times$ (linear form) = (linear form)"; additions are free. R1CS is equivalent to circuits up to a constant factor.

Example 10.3 (Running example: "I know x with $x^3 + x + 5 = 35$")**.** Flatten into gates:

$$v_1 = x \cdot x, \quad v_2 = v_1 \cdot x, \quad v_3 = (v_2 + x) \cdot 1, \quad \text{out} = (v_3 + 5) \cdot 1.$$

Variable vector $\vec{a} = (1, \text{out}, x, v_1, v_2, v_3)$; public inputs $(1, \text{out})$, private (x, v_1, v_2, v_3). The witness $x = 3$ gives $\vec{a} = (1, 35, 3, 9, 27, 30)$. The four constraints, rows of A, B, C (columns in the order of $\vec{a}$):

gate	A_j	B_j	C_j	constraint
1	$(0,0,1,0,0,0)$	$(0,0,1,0,0,0)$	$(0,0,0,1,0,0)$	$x \cdot x = v_1$
2	$(0,0,0,1,0,0)$	$(0,0,1,0,0,0)$	$(0,0,0,0,1,0)$	$v_1 \cdot x = v_2$
3	$(0,0,1,0,1,0)$	$(1,0,0,0,0,0)$	$(0,0,0,0,0,1)$	$(x+v_2)\cdot 1 = v_3$
4	$(5,0,0,0,0,1)$	$(1,0,0,0,0,0)$	$(0,1,0,0,0,0)$	$(5+v_3)\cdot 1 = \text{out}$

Check: gate 1: $3\cdot 3 = 9$; gate 2: $9\cdot 3 = 27$; gate 3: $(3+27)\cdot 1 = 30$; gate 4: $(5+30)\cdot 1 = 35$. ✓

10.2 Quadratic Arithmetic Programs

R1CS has d constraints that a verifier would check one by one—linear work. QAPs (Gennaro–Gentry–Parno–Raykova 2013) encode all d constraints in a *single* polynomial divisibility.

Definition 10.4 (QAP)**.** Fix distinct points $\omega_1, \dots, \omega_d \in \mathbb{F}$ and $t(X) = \prod_j (X - \omega_j)$. For each variable index $i \in \{0..m-1\}$ let $u_i(X), v_i(X), w_i(X)$ be the unique polynomials of degree $< d$ with

$$u_i(\omega_j) = A_{ji}, \quad v_i(\omega_j) = B_{ji}, \quad w_i(\omega_j) = C_{ji} \qquad (j \in [d]).$$

For a vector $\vec{a}$ define $A(X) = \sum_i a_i u_i(X)$, $B(X) = \sum a_i v_i(X)$, $C(X) = \sum a_i w_i(X)$.

Theorem 10.5 (R1CS $\Leftrightarrow$ QAP)**.** *$\vec{a}$ satisfies the R1CS if and only if $t(X)$ divides $P(X) := A(X)B(X) - C(X)$. Equivalently, there is $h(X)$ of degree $\le d-2$ with $A(X)B(X) - C(X) = h(X)t(X)$.*

Proof. For each j, $P(\omega_j) = \big(\sum_i a_i A_{ji}\big)\big(\sum_i a_i B_{ji}\big) - \sum_i a_i C_{ji}$, which is zero exactly when constraint j holds. A polynomial vanishes on all ω_j iff it is divisible by $\prod_j (X-\omega_j) = t(X)$ (Lemma 2.14 repeatedly). Degrees: $\deg A, \deg B \le d-1$, so $\deg P \le 2d-2$ and $\deg h \le d-2$. □

Intuition

Think of $\omega_1, \dots, \omega_d$ as "time stamps" of the gates. $A(X)$ is the left operand wire of the gate active at "time" X, etc. $A \cdot B - C$ is the "error polynomial"; it vanishes on all time stamps iff all gates are correct.

Example 10.6 (QAP for Example 10.3)**.** Take $\omega = (1,2,3,4)$, $t(X) = (X-1)(X-2)(X-3)(X-4) = X^4 - 10X^3 + 35X^2 - 50X + 24$. The column of A for variable x (index 2) is $(1,0,1,0)^\top$, so u_2 is the cubic through $(1,1),(2,0),(3,1),(4,0)$:

$$u_2(X) = -\tfrac{2}{3}X^3 + 5X^2 - \tfrac{34}{3}X + 8.$$

Similarly $v_2(X) = \frac{1}{3}X^3 - \frac{5}{2}X^2 + \frac{31}{6}X - 2$ (column $(1,1,0,0)$) and $w_1(X) = \frac{1}{6}X^3 - X^2 + \frac{11}{6}X - 1$ (column $(0,0,0,1)$). Summing with the witness $(1, 35, 3, 9, 27, 30)$:

$$A(X) = -\tfrac{31}{6}X^3 + \tfrac{77}{2}X^2 - \tfrac{220}{3}X + 43, \qquad B(X) = \tfrac{2}{3}X^3 - 5X^2 + \tfrac{31}{3}X - 3,$$
$$C(X) = \tfrac{17}{6}X^3 - \tfrac{49}{2}X^2 + \tfrac{215}{3}X - 41.$$

Spot checks: $A(1) = 3, B(1) = 3, C(1) = 9$ (gate 1: $3\cdot 3 = 9$). Polynomial division gives remainder 0 and

$$h(X) = \frac{A(X)B(X) - C(X)}{t(X)} = -\tfrac{31}{9}X^2 + \tfrac{307}{18}X - \tfrac{11}{3}.$$

A wrong witness, e.g. $v_2 = 28$, makes $A(3)B(3) - C(3) \neq 0$ so the division leaves a nonzero remainder.

Probabilistic check. A prover who claims $\vec{a}$ satisfies the R1CS gives h; the verifier picks random $\tau \in \mathbb{F}$ and checks $A(\tau)B(\tau) - C(\tau) = h(\tau)t(\tau)$. If the claim were false, $P - ht$ would be a nonzero polynomial of degree $\leq 2d - 2$, so by Schwartz–Zippel (Lemma 2.10) the check passes with probability $\leq (2d-2)/|\mathbb{F}|$. The chapters that follow differ in *how* the evaluation at the hidden point τ is made succinct, non-interactive and zero-knowledge.

10.3 PLONKish arithmetisation

PLONK replaces R1CS by a single universal gate type plus a *permutation* argument for wiring:

$$q_L\, a + q_R\, b + q_O\, c + q_M\, ab + q_C = 0.$$

Row i of the *execution trace* holds the wire values (a_i, b_i, c_i), together with five public selector values $(q_L, q_R, q_O, q_M, q_C)_i$ fixed by the circuit. Addition gate: $q_L = q_R = 1, q_O = -1$. Multiplication gate: $q_M = 1, q_O = -1$. Constant check: $q_L = 1, q_C = -k, \ldots$

Example 10.7 (Example 10.3 in PLONK form)**.**

row	a	b	c	q_L	q_R	q_O	q_M	q_C	meaning
1	x	x	v_1	0	0	-1	1	0	$x \cdot x - v_1 = 0$
2	v_1	x	v_2	0	0	-1	1	0	$v_1 x - v_2 = 0$
3	v_2	x	v_3	1	1	-1	0	0	$v_2 + x - v_3 = 0$
4	v_3	0	out	1	0	-1	0	5	$v_3 + 5 - \text{out} = 0$

With the witness: (a, b, c) rows are $(3, 3, 9)$, $(9, 3, 27)$, $(27, 3, 30)$, $(30, 0, 35)$. Row-wise constraints hold. What is *not* yet enforced is that the same variable is used in different cells: x appears in a_1, b_1, b_2, b_3, and $c_1 = a_2$, $c_2 = a_3$, $c_3 = a_4$. These are *copy constraints*, enforced by a permutation σ on cells (Chapter 12).

10.4 Algebraic Intermediate Representation (AIR)

For long, uniform computations (a CPU or VM running T steps), STARKs use an *execution trace*: a matrix with T rows (time) and w columns (registers), plus

- **transition constraints**: low-degree polynomials $\mathcal{P}_k(\vec{s}_i, \vec{s}_{i+1}) = 0$ relating a row to the next;
- **boundary constraints**: initial and final values.

Example 10.8 (Fibonacci)**.** Single column $s_0..s_7 = 1, 1, 2, 3, 5, 8, 13, 21$. Claim: "the 8th Fibonacci number is 21". Transition: $s_{i+2} - s_{i+1} - s_i = 0$ for $i = 0..5$. Boundary: $s_0 = 1$, $s_1 = 1$, $s_7 = 21$. Let ω be a generator of the multiplicative subgroup of order 8, $f(\omega^i) = s_i$ (interpolate). The transition constraint becomes the polynomial identity $f(\omega^2 X) - f(\omega X) - f(X) = 0$ for $X \in \{\omega^0, \ldots, \omega^5\}$, i.e. the polynomial

$$Q_{\text{tr}}(X) = \frac{f(\omega^2 X) - f(\omega X) - f(X)}{\prod_{i=0}^{5}(X - \omega^i)}$$

must be a *polynomial* (of low degree). Similarly $Q_{\text{bd}}(X) = (f(X) - 1)/(X - \omega^0)$ etc. The prover commits to f and to the quotients; the verifier checks that each is of low degree via FRI (Chapter 11).

Remark 10.9 (Comparison). R1CS/QAP: unstructured, flexible, constant-size proofs with pairings. PLONKish: custom gates and lookups, universal SRS. AIR: uniform/repeating computations, transparent (hash-based). Hybrids exist: Halo2, Plonky2/3 (AIR over small fields + FRI), Nova/HyperNova (folding R1CS instances).

Exercises

Exercise 10.10. Write an R1CS (give A, B, C) for "$y = x^4$" and for the Boolean constraint "$b \in \{0,1\}$ and $z = b \cdot c_1 + (1-b) \cdot c_0$" (a multiplexer).

Exercise 10.11. In the toy QAP above, compute the polynomial $A(X)B(X) - C(X)$ modulo $t(X)$ for the wrong witness $(1, 35, 3, 9, 28, 30)$ and find which gates it violates.

Exercise 10.12. Prove that for d points and ω_j the d-th roots of unity, $t(X) = X^d - 1$, and that Lagrange interpolation can be done by an inverse FFT in $O(d \log d)$.

Exercise 10.13. Show that R1CS satisfiability is NP-complete by reducing from CircuitSAT.

Chapter 11

Polynomial Commitment Schemes

A polynomial commitment scheme (PCS) lets a prover commit to a polynomial f with a short string C and later prove, with a short proof, that $f(z) = y$ for a point z chosen by the verifier. PCS are the cryptographic core of modern proof systems: an *information-theoretic* proof system (a "polynomial IOP") in which the verifier queries oracles to polynomials becomes a real argument by replacing the oracles with commitments (and Fiat–Shamir).

Definition 11.1 (Polynomial commitment scheme)**.** A tuple $(\mathsf{Setup}, \mathsf{Commit}, \mathsf{Open}, \mathsf{Verify})$:

- $\mathsf{Setup}(1^\lambda, D) \to \mathsf{pp}$ for degree bound D;
- $\mathsf{Commit}(\mathsf{pp}, f) \to C$;
- $\mathsf{Open}(\mathsf{pp}, f, z) \to (y, \pi)$ with $y = f(z)$;
- $\mathsf{Verify}(\mathsf{pp}, C, z, y, \pi) \to \{0, 1\}$.

Correctness; **evaluation binding** (no PPT adversary opens one C to (z, y) and (z, y'), $y \neq y'$, both accepted); **knowledge soundness** (an extractor recovers an f of degree $\leq D$ consistent with all accepted openings); optionally **hiding** (commitment and openings reveal nothing about f beyond evaluated points).

The main PCS families:

Scheme	Setup	Proof size	Verifier	Assumption
KZG	trusted (universal)	$O(1)$ (1 group element)	$O(1)$ (2 pairings)	q-SDH / AGM
IPA (Bulletproofs/Halo)	transparent	$O(\log n)$	$O(n)$ (amortisable)	DL
FRI	transparent	$O(\log^2 n)$	$O(\log^2 n)$	collision-resistant hash (PQ)
Dory / Hyrax / Brakedown	transparent	varies	varies	DLIN / hash / codes

11.1 KZG commitments (Kate–Zaverucha–Goldberg 2010)

11.1.1 The construction

Let $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ be a pairing of prime order q and generators g_1, g_2; bracket notation $[a]_i = g_i^a$.

Algorithm 11.1: KZG polynomial commitment

Setup (D): sample the *toxic waste* $\tau \xleftarrow{\$} \mathbb{F}_q$ and publish the structured reference string

$$\mathsf{srs} = \big([1]_1, [\tau]_1, [\tau^2]_1, \ldots, [\tau^D]_1;\ [1]_2, [\tau]_2\big),$$

then **delete** τ.
Commit$(f = \sum_{i \le D} f_i X^i)$: $C := \sum_i f_i [\tau^i]_1 = [f(\tau)]_1$ (a multi-scalar multiplication, no knowledge of τ needed).
Open(f, z): $y = f(z)$; quotient $\psi(X) = \dfrac{f(X) - y}{X - z}$ (a polynomial of degree $\le D - 1$ by Lemma 2.14); $\pi := [\psi(\tau)]_1$.
Verify(C, z, y, π): accept iff

$$e\big(C - [y]_1,\ [1]_2\big) = e\big(\pi,\ [\tau]_2 - [z]_2\big).$$

Theorem 11.2. *KZG is correct. It is evaluation-binding under the q-strong Diffie–Hellman assumption (q-SDH): given* $([\tau^i]_1, [\tau^i]_2)_{i \le D}$ *no PPT algorithm outputs* $(z, [1/(\tau - z)]_1)$. *Knowledge soundness holds in the algebraic group model.*

Proof. Correctness. $f(X) - y = (X - z)\psi(X)$, evaluate at τ: $f(\tau) - y = (\tau - z)\psi(\tau)$. Taking pairings, $e([f(\tau) - y]_1, [1]_2) = e([\psi(\tau)]_1, [\tau - z]_2)$, which is the verification equation.

Binding. Suppose $\mathcal{A}$ outputs C, z and two valid openings (y, π), (y', π') with $y \neq y'$. Then $e(C - [y]_1, [1]_2) = e(\pi, [\tau - z]_2)$ and $e(C - [y']_1, [1]_2) = e(\pi', [\tau - z]_2)$. Dividing, $e([y' - y]_1, [1]_2) = e(\pi - \pi', [\tau - z]_2)$; so $\pi - \pi' = \left[\frac{y' - y}{\tau - z}\right]_1$, hence $\left[\frac{1}{\tau - z}\right]_1 = (y' - y)^{-1}(\pi - \pi')$ is computed, breaking q-SDH. □

Intuition

The commitment is a *point evaluation at a secret place* τ. Since the prover does not know τ, the only way she can produce $[\psi(\tau)]_1$ with the property $\psi(X)(X - z) = f(X) - y$ is to know an actual quotient polynomial, which exists only if $f(z) = y$ really holds. The pairing checks one multiplicative relation between three hidden numbers.

Example 11.3 (KZG "in the exponent")**.** Work with scalars modulo $q = 11$ (we show the exponents; $[a]_1$ corresponds to a). Let $f(X) = X^2 + 1$, $z = 2$, $y = f(2) = 5$. Quotient: $(X^2 + 1 - 5)/(X - 2) = (X^2 - 4)/(X - 2) = X + 2 =: \psi(X)$. With secret $\tau = 7$: commitment exponent $f(\tau) = 50 \equiv 6$; proof exponent $\psi(\tau) = 9$. The pairing equation in the exponent reads $f(\tau) - y = (\tau - z)\psi(\tau)$: LHS $6 - 5 = 1$; RHS $(7 - 2) \cdot 9 = 45 \equiv 1 \pmod{11}$. ✓A false claim $y' = 4$ would require a proof element π' with $(\tau - z)\pi' = f(\tau) - y' = 2$, i.e. $\pi' = 2/(\tau - z)$. The element $[1/(\tau - z)]_1$ is not a polynomial expression of degree $\le D$ in τ, so it cannot be formed from the SRS; computing it is exactly the q-SDH problem.

11.1.2 Extensions

- **Batch opening at many points.** To prove $f(z_i) = y_i$ for $i \in [k]$ let $I(X)$ interpolate (z_i, y_i) and $Z(X) = \prod (X - z_i)$; then $\psi = (f - I)/Z$, $\pi = [\psi(\tau)]_1$, check $e(C - [I(\tau)]_1, [1]_2) = e(\pi, [Z(\tau)]_2)$.
- **Several polynomials at the same point.** Random linear combination $\sum_j \gamma^j f_j$ (challenge γ): by Schwartz–Zippel an incorrect combination is caught except with probability $k/|\mathbb{F}|$. One proof for all.

- **Hiding.** Add a random multiple of a blinding polynomial: $C = [f(\tau) + \rho\,\eta(\tau)]_1$ with an additional SRS element.
- **Universal setup.** The SRS depends only on a degree bound D, not on the circuit; the "Powers of Tau" ceremony has thousands of contributors, and security needs only *one* honest participant.

11.2 FRI: a hash-based low-degree test

11.2.1 Reed–Solomon codes and proximity

Evaluating a polynomial of degree $< k$ on a domain $D \subseteq \mathbb{F}$ with $|D| = n = k/\rho$ ($\rho = rate$, e.g. $\rho = 1/4$) gives a codeword of the Reed–Solomon code $\mathrm{RS}[\mathbb{F}, D, \rho]$. A function $f : D \to \mathbb{F}$ is δ-*far* if every codeword differs in at least δn positions. FRI (Fast Reed–Solomon IOP of Proximity, Ben-Sasson–Bentov–Horesh–Riabzev) allows a verifier to distinguish "f is a codeword" from "f is δ-far" with $O(\log n)$ queries to a Merkle commitment.

11.2.2 Folding

Any polynomial splits into even and odd parts: $f(X) = f_e(X^2) + X\,f_o(X^2)$ with $\deg f_e, \deg f_o < k/2$. For a verifier challenge β define

$$f^{(1)}(Y) = f_e(Y) + \beta f_o(Y).$$

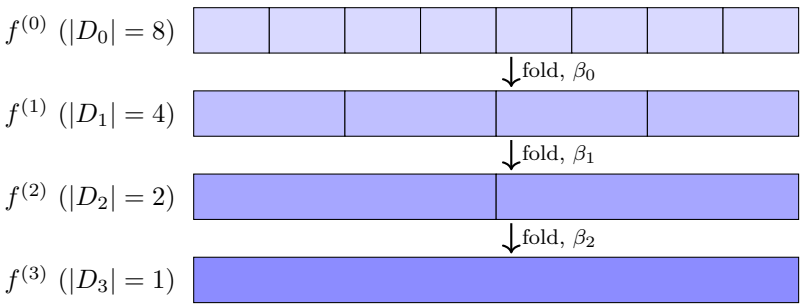
Then $\deg f^{(1)} < k/2$ and, at $Y = x^2$ for $x \in D$,

$$f^{(1)}(x^2) = \frac{f(x) + f(-x)}{2} + \beta\,\frac{f(x) - f(-x)}{2x}.$$

So the folded evaluation on $D^2 = \{x^2\}$ (half the size) is computable from just two values of f. If f is far from low degree, then so is $f^{(1)}$ for most β (the "proximity gap" lemma).

Algorithm 11.2: FRI (commit and query phases)

Commit phase. $f^{(0)} = f$ on $D_0 = D$; Merkle-commit to $f^{(0)}$. For $r = 0..R-1$: V sends β_r; P computes $f^{(r+1)} = \mathrm{Fold}(f^{(r)}, \beta_r)$ on $D_{r+1} = D_r^2$ and commits. Finally P sends the (constant or small) last polynomial in the clear.
Query phase (repeated s times): V picks a random $x \in D_0$. For each r P opens (with Merkle paths) $f^{(r)}(x_r)$ and $f^{(r)}(-x_r)$ ($x_r = x^{2^r}$), and V checks the folding equation $f^{(r+1)}(x_r^2) = \frac{f^{(r)}(x_r)+f^{(r)}(-x_r)}{2} + \beta_r \frac{f^{(r)}(x_r)-f^{(r)}(-x_r)}{2x_r}$, and finally the last-layer value.



Example 11.4 (One folding step, $\mathbb{F}_{17}$)**.** Let $\mathbb{F} = \mathbb{F}_{17}$, $D_0 = \{1, 2, 4, 8, 9, 13, 15, 16\}$ (the quadratic residues, order 8), $D_1 = D_0^2 = \{1, 4, 13, 16\}$. Take $f(X) = X^3 + 2X^2 + 3X + 4$ ($k = 4$, rate 1/2). Then $f_e(Y) = 2Y + 4$ (even-degree terms $2X^2 + 4$ with $Y = X^2$) and $f_o(Y) = Y + 3$ (as $X^3 + 3X = X(X^2 + 3)$). With $\beta = 3$: $f^{(1)}(Y) = 2Y + 4 + 3(Y + 3) = 5Y + 13$, degree $1 < k/2 = 2$.
Consistency check at $x = 2$ (so $x^2 = 4$): $f(2) = 8 + 8 + 6 + 4 = 26 \equiv 9$, $f(-2) = f(15) = -8 + 8 - 6 + 4 = -2 \equiv 15$. Then $\frac{9+15}{2} = 12$ and $\frac{9-15}{2 \cdot 2} = \frac{-6}{4} \equiv -6 \cdot 13 = -78 \equiv 7$ (as $4 \cdot 13 = 52 \equiv 1$), so $f^{(1)}(4) = 12 + 3 \cdot 7 = 33 \equiv 16$. Directly: $5 \cdot 4 + 13 = 33 \equiv 16$. ✓
After $\log_2 k$ folds the polynomial is constant and the verifier can check it directly.

Theorem 11.5 (FRI soundness, informal)**.** *If f is δ-far from $\mathrm{RS}[\mathbb{F}, D, \rho]$ then, for δ below the Johnson bound, the verifier rejects with probability $\geq 1 - (1 - \delta)^s - \varepsilon_{\text{fold}}$ where s is the number of queries and $\varepsilon_{\text{fold}} = O(n/|\mathbb{F}|)$ accounts for unlucky folding challenges. Querying $s \approx \lambda / \log_2(1/(1 - \delta))$ locations gives λ bits of security; smaller rate ρ means larger δ, i.e. fewer queries but a larger domain.*

FRI turns a Merkle commitment into a polynomial commitment: to open $f(z) = y$, run FRI on the quotient $(f(X) - y)/(X - z)$; the check that this is low-degree convinces the verifier $f(z) = y$ (otherwise the quotient has a pole at z).

11.3 Inner-product arguments and Bulletproofs

11.3.1 Pedersen vector commitment

For generators $\vec{g} = (g_1..g_n) \in \mathbb{G}^n$ with unknown relations, $\mathsf{Com}(\vec{a}; r) = \prod_i g_i^{a_i} \cdot h^r$ commits to a vector; binding under DL; perfectly hiding.

11.3.2 The core: proving an inner product in $O(\log n)$

Statement: given $P = \vec{g}^{\vec{a}} \vec{h}^{\vec{b}} u^{\langle \vec{a}, \vec{b} \rangle}$ ($\vec{g}^{\vec{a}} := \prod g_i^{a_i}$), prove knowledge of $\vec{a}, \vec{b} \in \mathbb{Z}_q^n$ (n a power of two). Naively send $\vec{a}, \vec{b}$ ($2n$ elements); the trick is *recursive folding*.

Protocol 11.3: Inner-product argument (one folding round)

Write $\vec{a} = (\vec{a}_L \| \vec{a}_R)$ and likewise $\vec{b}, \vec{g}, \vec{h}$ (halves of size $n/2$).

Prover		**Verifier**
$L = \vec{g}_R^{\vec{a}_L} \vec{h}_L^{\vec{b}_R} u^{\langle \vec{a}_L, \vec{b}_R \rangle}$		
$R = \vec{g}_L^{\vec{a}_R} \vec{h}_R^{\vec{b}_L} u^{\langle \vec{a}_R, \vec{b}_L \rangle}$	$\xrightarrow{L, R}$	
	$\xleftarrow{x}$	$x \xleftarrow{\$} \mathbb{Z}_q^*$
$\vec{a}' = x\vec{a}_L + x^{-1}\vec{a}_R$		$\vec{g}' = \vec{g}_L^{x^{-1}} \circ \vec{g}_R^{x}$
$\vec{b}' = x^{-1}\vec{b}_L + x\vec{b}_R$		$\vec{h}' = \vec{h}_L^{x} \circ \vec{h}_R^{x^{-1}}$
		$P' = L^{x^2} P R^{x^{-2}}$

Now the same statement is claimed for $(\vec{g}', \vec{h}', P', \vec{a}', \vec{b}')$ with length $n/2$. Recurse until $n = 1$, then the prover sends the two scalars a, b and the verifier checks $P = g^a h^b u^{ab}$.

Lemma 11.6 (Invariant)**.** *If $P = \vec{g}^{\vec{a}} \vec{h}^{\vec{b}} u^{\langle \vec{a}, \vec{b} \rangle}$ then $P' = \vec{g}'^{\vec{a}'} \vec{h}'^{\vec{b}'} u^{\langle \vec{a}', \vec{b}' \rangle}$.*

Proof. Expand:

$$\begin{aligned}\vec{g}'^{\vec{a}'} &= \vec{g}_L^{\vec{a}_L}\ \vec{g}_R^{\vec{a}_R}\ \vec{g}_L^{x^{-2}\vec{a}_R}\ \vec{g}_R^{x^2\vec{a}_L},\\ \vec{h}'^{\vec{b}'} &= \vec{h}_L^{\vec{b}_L}\ \vec{h}_R^{\vec{b}_R}\ \vec{h}_L^{x^2\vec{b}_R}\ \vec{h}_R^{x^{-2}\vec{b}_L},\\ u^{\langle\vec{a}',\vec{b}'\rangle} &= u^{\langle\vec{a},\vec{b}\rangle + x^2\langle\vec{a}_L,\vec{b}_R\rangle + x^{-2}\langle\vec{a}_R,\vec{b}_L\rangle}.\end{aligned}$$

Collect: $\vec{g}^{\vec{a}}\vec{h}^{\vec{b}}u^{\langle\vec{a},\vec{b}\rangle}\cdot\left(\vec{g}_R^{\vec{a}_L}\vec{h}_L^{\vec{b}_R}u^{\langle\vec{a}_L,\vec{b}_R\rangle}\right)^{x^2}\cdot\left(\vec{g}_L^{\vec{a}_R}\vec{h}_R^{\vec{b}_L}u^{\langle\vec{a}_R,\vec{b}_L\rangle}\right)^{x^{-2}} = P\,L^{x^2}R^{x^{-2}} = P'$. □

Example 11.7 (Scalar folding in $\mathbb{Z}_{11}$)**.** $\vec{a}=(1,2,3,4)$, $\vec{b}=(5,6,7,8)$, $\langle\vec{a},\vec{b}\rangle = 70\equiv 4$. Halves: $\vec{a}_L=(1,2)$, $\vec{a}_R=(3,4)$, $\vec{b}_L=(5,6)$, $\vec{b}_R=(7,8)$; cross terms $\langle\vec{a}_L,\vec{b}_R\rangle = 23$, $\langle\vec{a}_R,\vec{b}_L\rangle = 39$. Challenge $x=2$, $x^{-1}=6$, $x^2=4$, $x^{-2}=3$.
$\vec{a}' = 2(1,2)+6(3,4) = (20,28)\equiv(9,6)$; $\vec{b}' = 6(5,6)+2(7,8) = (44,52)\equiv(0,8)$; $\langle\vec{a}',\vec{b}'\rangle = 48\equiv 4$.
Predicted by the invariant: $\langle\vec{a},\vec{b}\rangle + 4\cdot 23 + 3\cdot 39 = 70+92+117 = 279 = 25\cdot 11+4\equiv 4$. ✓
Two vectors of length 4 became vectors of length 2 at the cost of two group elements L, R.

Theorem 11.8. *The protocol has* $\log_2 n$ *rounds, proof size* $2\log_2 n$ *group elements plus* 2 *scalars, and is an argument of knowledge under DL (extraction by* 3*-special soundness per round: transcripts for three challenges* x_1, x_2, x_3 *yield a "witness" of length* n *via a linear system, forming a tree of* $3^{\log n} = n^{\log 3}$ *transcripts—polynomial).*

11.3.3 Application: range proofs (Bulletproofs, Bünz et al. 2018)

To show that a Pedersen commitment $V = g^v h^\gamma$ hides $v\in[0,2^n)$:

1. Let $\vec{a}_L\in\{0,1\}^n$ be the bits of v and $\vec{a}_R = \vec{a}_L - \vec{1}^n$. The statement is equivalent to three conditions: $\langle\vec{a}_L, 2^n\rangle = v$, $\vec{a}_L\circ\vec{a}_R = \vec{0}$ (each bit is 0 or 1), $\vec{a}_L - \vec{a}_R = \vec{1}^n$. Here $2^n = (1,2,4,\ldots,2^{n-1})$.
2. The verifier sends random y, z. By Schwartz–Zippel the three conditions hold iff (w.h.p.) the single scalar identity

   $$z^2 v + \delta(y,z) = \left\langle \vec{a}_L - z\vec{1}^n,\ \vec{y}^n\circ(\vec{a}_R + z\vec{1}^n) + z^2 2^n\right\rangle, \qquad \delta(y,z) = (z-z^2)\langle\vec{1}^n,\vec{y}^n\rangle - z^3\langle\vec{1}^n,2^n\rangle$$

   holds, where $\vec{y}^n = (1,y,\ldots,y^{n-1})$.
3. To hide the vectors, the prover adds blinding vectors $\vec{s}_L,\vec{s}_R$ and defines the degree-one vector polynomials

   $$\vec{l}(X) = (\vec{a}_L - z\vec{1}^n) + \vec{s}_L X, \qquad \vec{r}(X) = \vec{y}^n\circ(\vec{a}_R + z\vec{1}^n + \vec{s}_R X) + z^2 2^n,$$

   with $t(X) = \langle\vec{l}(X),\vec{r}(X)\rangle = t_0 + t_1 X + t_2 X^2$ and $t_0 = z^2 v + \delta(y,z)$ exactly when the statement is true.
4. The prover commits to t_1, t_2 (Pedersen commitments T_1, T_2), receives a challenge x, and reveals $\vec{l} = \vec{l}(x)$, $\vec{r} = \vec{r}(x)$, $\hat{t} = t(x)$ with blinding factors. The verifier checks $g^{\hat{t}}h^{\tau_x} = V^{z^2}g^{\delta(y,z)}T_1^x T_2^{x^2}$ (which checks t_0 and the form of t at a random point), and that $\vec{l},\vec{r}$ are consistent with the commitments to $\vec{a}_L,\vec{a}_R,\vec{s}_L,\vec{s}_R$ and satisfy $\hat{t} = \langle\vec{l},\vec{r}\rangle$—the last by the inner-product argument, $O(\log n)$ communication.

Example 11.9 (Checking the identity in step 2, $n=2$, over $\mathbb{Z}$)**.** Let $v=2$ (bits $\vec{a}_L=(0,1)$, $\vec{a}_R=(-1,0)$), $y=2$, $z=3$. Then $\vec{y}^2=(1,2)$, $2^2=(1,2)$, $\langle\vec{1},\vec{y}^2\rangle = 3$, $\langle\vec{1},2^2\rangle = 3$, so $\delta = (3-9)\cdot 3 - 27\cdot 3 = -18-81 = -99$ and LHS $= 9\cdot 2 - 99 = -81$. RHS: $\vec{a}_L - z\vec{1} = (-3,-2)$; $\vec{a}_R + z\vec{1} = (2,3)$, $\vec{y}\circ(\cdot) = (2,6)$, plus $z^2 2^2 = (9,18)$ gives $(11,24)$; inner product $-33-48 = -81$. ✓

A 64-bit range proof is ≈ 700 bytes; aggregating m proofs adds only $2\log_2 m$ group elements. Used in Monero, Mimblewimble and confidential assets.

Exercises

Exercise 11.10. Prove that if the prover commits to a polynomial of degree $> D$ the KZG verification fails for most z or the prover cannot form π (hint: the SRS only contains powers up to τ^D; discuss degree enforcement through a second commitment shifted by τ^{D-d}).

Exercise 11.11. Derive the KZG multi-point opening verification equation, and the batched version for several polynomials at the same point.

Exercise 11.12. Check, for $f(X) = X^3 + 2X^2 + 3X + 4$ over $\mathbb{F}_{17}$ with $\beta = 3$, that the folding relation holds at $x = 4$ as well (compute $f(4), f(13)$, then $f^{(1)}(16)$).

Exercise 11.13. Verify the final round of the inner-product argument for $n = 2$: show that with $\vec{a} = (a_1, a_2)$, $\vec{b} = (b_1, b_2)$ and challenge x, P' equals $g'^{a'} h'^{b'} u^{a'b'}$.

Exercise 11.14. Explain why IPA verification is $O(n)$ (hint: the folded generators $\vec{g}'$ are the verifier's burden) and how a single multi-exponentiation with scalars $s_i = \prod_j x_j^{\pm 1}$ resolves it.

Chapter 12

SNARKs and STARKs

12.1 Definitions

Definition 12.1 (zk-SNARK)**.** A *zk-SNARK* (zero-knowledge Succinct Non-interactive ARgument of Knowledge) for an NP relation R is a triple $(\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$ with $\mathsf{Setup}(R) \to (\mathsf{pk}, \mathsf{vk})$ (the CRS), $\mathsf{Prove}(\mathsf{pk}, x, w) \to \pi$, $\mathsf{Verify}(\mathsf{vk}, x, \pi) \to \{0, 1\}$ such that

Completeness honest proofs verify;

Knowledge soundness for every PPT prover $\mathcal{A}$ there is a PPT extractor $\mathsf{Ext}_{\mathcal{A}}$ (non-black-box; given $\mathcal{A}$'s coins) such that whenever $\mathcal{A}$ outputs an accepting (x, π) then $\mathsf{Ext}_{\mathcal{A}}$ outputs w with $(x, w) \in R$, except with negligible probability;

Zero-knowledge there is a simulator with trapdoor that outputs proofs indistinguishable from real ones (for the same x);

Succinctness $|\pi| = \mathrm{poly}(\lambda)$ independent of $|w|$ (typically $O(1)$ or $O(\log |C|)$) and verification time $\mathrm{poly}(\lambda, |x|)$ $(+\ \mathrm{polylog}|C|)$.

Remark 12.2 (Why non-black-box extraction?). Gentry–Wichs (2011) show that succinct non-interactive arguments with black-box (falsifiable) soundness for NP cannot be proven secure under falsifiable assumptions. So SNARK security rests on knowledge assumptions (KEA, AGM, generic group model) or the random oracle model.

12.2 Groth16

Groth16 [16] is the pairing-based zk-SNARK for QAPs with the smallest known proof size (3 group elements: $\approx$ 128 bytes on BN254 / 192 bytes on BLS12-381) and cheapest verification (3 pairings), and is deployed in Zcash Sapling, Tornado Cash and many rollups.

12.2.1 Algorithm

Let the QAP be $(\{u_i, v_i, w_i\}_{i=0}^{m-1}, t)$ over $\mathbb{F}_q$ for the relation, with the first ℓ variables public ($a_0 = 1$ and the statement). Let $\pi = (A, B, C)$.

Algorithm 12.1: Groth16

Setup. Sample toxic waste $\tau, \alpha, \beta, \gamma, \delta \xleftarrow{\$} \mathbb{F}_q^*$. Set

- $\mathsf{pk} \supseteq [\alpha]_1, [\beta]_1, [\delta]_1, [\beta]_2, [\delta]_2,\ \{[\tau^k]_1\}_{k<d},\ \{[\tau^k]_2\}_{k<d}$,
- $\mathsf{pk} \supseteq \left\{\left[\frac{\beta u_i(\tau)+\alpha v_i(\tau)+w_i(\tau)}{\delta}\right]_1\right\}_{i\geq \ell}$ (private variables), $\left\{\left[\frac{\tau^k t(\tau)}{\delta}\right]_1\right\}_{k\leq d-2}$,
- $\mathsf{vk} = \Big([\alpha]_1, [\beta]_2, [\gamma]_2, [\delta]_2,\ \left\{\left[\frac{\beta u_i(\tau)+\alpha v_i(\tau)+w_i(\tau)}{\gamma}\right]_1\right\}_{i<\ell}\Big)$.

Prove$(\mathsf{pk}, \vec{a})$: compute $h(X) = (A(X)B(X) - C(X))/t(X)$ for the witness polynomials. Sample $r, s \xleftarrow{\$} \mathbb{F}_q$ (*zero-knowledge randomisers*). Output

$$A = \big[\alpha + \textstyle\sum_i a_i u_i(\tau) + r\delta\big]_1, \qquad B = \big[\beta + \sum_i a_i v_i(\tau) + s\delta\big]_2,$$
$$C = \left[\frac{\sum_{i\geq\ell} a_i(\beta u_i(\tau)+\alpha v_i(\tau)+w_i(\tau))+h(\tau)t(\tau)}{\delta} + sA_\bullet + rB_\bullet - rs\delta\right]_1$$

where $A_\bullet = \alpha + \sum a_i u_i(\tau) + r\delta$ and $B_\bullet = \beta + \sum a_i v_i(\tau) + s\delta$ are the exponents of A and B; B is also needed in $\mathbb{G}_1$ (publish $[\beta]_1, [v_i(\tau)]_1$ to form it).

Verify$(\mathsf{vk}, \vec{x}, \pi)$: with $\mathsf{IC} = \sum_{i<\ell} a_i \left[\frac{\beta u_i+\alpha v_i+w_i}{\gamma}\right]_1$, accept iff

$$e(A, B) = e([\alpha]_1, [\beta]_2) \cdot e(\mathsf{IC}, [\gamma]_2) \cdot e(C, [\delta]_2).$$

12.2.2 Correctness

Write $K_i = \beta u_i(\tau) + \alpha v_i(\tau) + w_i(\tau)$, $\bar{A} = \sum_i a_i u_i(\tau)$, $\bar{B} = \sum_i a_i v_i(\tau)$, $\bar{C} = \sum_i a_i w_i(\tau)$, so that

$$\sum_i a_i K_i = \beta\bar{A} + \alpha\bar{B} + \bar{C}, \qquad h(\tau)t(\tau) = \bar{A}\bar{B} - \bar{C}.$$

Let $\mathcal{A} = \alpha + \bar{A} + r\delta$ and $\mathcal{B} = \beta + \bar{B} + s\delta$ be the exponents of A and B, and $\mathcal{C} = \frac{\sum_{i\geq\ell} a_i K_i + h(\tau)t(\tau)}{\delta} + s\mathcal{A} + r\mathcal{B} - rs\delta$ the exponent of C. The right-hand side of the verification equation has exponent (the IC term contributes $\sum_{i<\ell} a_i K_i$ since it was divided by γ and is paired with γ):

$$\begin{aligned}\alpha\beta + \sum_{i<\ell} a_i K_i + \delta\mathcal{C} &= \alpha\beta + \sum_i a_i K_i + h(\tau)t(\tau) + \delta\big(s\mathcal{A} + r\mathcal{B} - rs\delta\big)\\ &= \alpha\beta + \beta\bar{A} + \alpha\bar{B} + \bar{C} + \bar{A}\bar{B} - \bar{C} + \delta\big(s\mathcal{A} + r\mathcal{B} - rs\delta\big)\\ &= (\alpha + \bar{A})(\beta + \bar{B}) + \delta\big(s\mathcal{A} + r\mathcal{B} - rs\delta\big).\end{aligned}$$

On the other hand

$$\mathcal{A}\mathcal{B} = (\alpha + \bar{A})(\beta + \bar{B}) + s\delta(\alpha + \bar{A}) + r\delta(\beta + \bar{B}) + rs\delta^2,$$

and $\delta(s\mathcal{A} + r\mathcal{B} - rs\delta) = s\delta(\alpha + \bar{A}) + rs\delta^2 + r\delta(\beta + \bar{B}) + rs\delta^2 - rs\delta^2 = s\delta(\alpha + \bar{A}) + r\delta(\beta + \bar{B}) + rs\delta^2$. The two expressions coincide, so $e(A, B)$ equals the right-hand side. □

Example 12.3 (Numerical Groth16 "in the exponent")**.** Take the QAP of Example 10.3 over $\mathbb{F}_{101}$, points $\omega = (1, 2, 3, 4)$ and the witness $\vec{a} = (1, 35, 3, 9, 27, 30)$. Public variables: $\ell = 2$ ($a_0 = 1$, $a_1 = 35$). Setup randomness $\tau = 7, \alpha = 2, \beta = 3, \gamma = 5, \delta = 11$. Prover randomness $r = 4$, $s = 9$. We display exponents (a real implementation hides them in $\mathbb{G}_1, \mathbb{G}_2$).

i	0	1	2	3	4	5
$u_i(\tau)$	100	0	46	36	56	20
$v_i(\tau)$	76	0	26	0	0	0
$w_i(\tau)$	0	20	0	91	36	56

$t(\tau) = (6)(5)(4)(3) = 360 \equiv 57$. Then $\bar{A} = 48$, $\bar{B} = 53$, $\bar{C} = 30$, $\bar{A}\bar{B} - \bar{C} = 2514 \equiv 90$ and $h(\tau) = 90 \cdot 57^{-1} \equiv 76$ (indeed $57 \cdot 76 = 4332 \equiv 90$).

Proof exponents: $A = \alpha + \bar{A} + r\delta = 2 + 48 + 44 = 94$; $B = \beta + \bar{B} + s\delta = 3 + 53 + 99 = 155 \equiv 54$; $C = 44$.
Verifier: $\mathsf{IC} \cdot \gamma = \sum_{i<2} a_i(\beta u_i + \alpha v_i + w_i) = (300 + 152) + 35 \cdot 20 = 452 + 700 \equiv 41$. Then

$$AB = 94 \cdot 54 = 5076 \equiv 26, \qquad \alpha\beta + \mathsf{IC}\gamma + C\delta = 6 + 41 + 44 \cdot 11 = 6 + 41 + 484 \equiv 26 \pmod{101}. \checkmark$$

A forged witness e.g. $v_2 = 28$ makes h non-polynomial; the prover cannot produce C without the exact (hidden) τ powers.

12.2.3 Zero-knowledge and soundness

Theorem 12.4 (Groth 2016)**.** *In the generic bilinear group model, the above is a perfectly zero-knowledge, knowledge-sound SNARK for QAPs.*

Proof idea. ZK. Given the trapdoor $(\tau, \alpha, \beta, \gamma, \delta)$ the simulator picks A, B uniformly and sets $C = \frac{AB - \alpha\beta - \mathsf{IC}\gamma}{\delta}$, satisfying the verification equation by definition. A real proof has $A = \alpha + \bar{A} + r\delta$ and $B = \beta + \bar{B} + s\delta$ with r, s uniform, so (A, B) is uniform, and C is determined by the verification equation: same distribution. *Soundness.* Any proof is a generic-group computation; the adversary's A, B, C are linear combinations of the SRS elements. Matching the formal polynomial identity in the indeterminates $(\tau, \alpha, \beta, \gamma, \delta)$ shows that the coefficient of $\alpha\beta$ forces the shape $A = \alpha + \sum a_i u_i + \ldots$; the γ and δ divisions separate public from private parts; finally the polynomial identity holds over the indeterminate τ, so $(\sum a_i u_i)(\sum a_i v_i) - \sum a_i w_i$ is divisible by $t(X)$, i.e. $\vec{a}$ is a QAP witness (Theorem 10.5), and the extractor reads off $\vec{a}$ from the coefficients. □

Trusted setup

Whoever knows $\tau, \alpha, \beta, \gamma, \delta$ can forge proofs of false statements (the simulator above). Groth16 requires a *circuit-specific* setup (phase 2 of an MPC ceremony after a universal phase 1 "Powers of Tau"); as long as one participant destroys her share, the CRS is secure. Another limitation: Groth16 proofs are malleable (re-randomisable) unless bound to a context.

12.3 PLONK

PLONK [17] uses the PLONKish arithmetisation (Section 10.3) and KZG to obtain a SNARK with *universal and updatable* setup: one SRS for all circuits of size $\leq D$.

12.3.1 Setup of the argument

Let $H = \{\omega^0, \ldots, \omega^{n-1}\}$ be the subgroup of n-th roots of unity in $\mathbb{F}$ (so $Z_H(X) = X^n - 1$). The prover interpolates wire polynomials $a(X), b(X), c(X)$ with $a(\omega^i) = a_i$, etc.; selector polynomials q_L, q_R, q_O, q_M, q_C are fixed by the circuit. The *gate constraint*

$$G(X) = q_L a + q_R b + q_O c + q_M ab + q_C + \mathrm{PI}(X) \;\equiv\; 0 \quad \text{on } H,$$

(PI encodes public inputs) holds iff $Z_H(X)$ divides $G(X)$.

12.3.2 Copy constraints: the permutation argument

Let the $3n$ cells be indexed by $\{1..3n\}$ and σ a permutation mapping each cell to the *next* cell containing the same variable (cycles = equivalence classes). Define $f : \{1..3n\} \to \mathbb{F}$ the cell values. Copy constraints hold iff $f(i) = f(\sigma(i))$ for all i.

Lemma 12.5 (Multiset equality via grand product)**.** *For random β, γ: $f(i) = f(\sigma(i))\ \forall i$ iff (w.h.p.) $\prod_i \big(f(i) + \beta i + \gamma\big) = \prod_i \big(f(i) + \beta\sigma(i) + \gamma\big)$.*

Proof. ($\Rightarrow$) If $f = f \circ \sigma$ then the right product is a reordering of the left. ($\Leftarrow$) Consider the pairs $(i, f(i))$ and $(\sigma(i), f(i))$ as multisets. The identity as polynomials in β, γ is $\prod_i(\gamma + f(i) + \beta i) = \prod_i(\gamma + f(i) + \beta\sigma(i))$; by unique factorisation of polynomials in $\mathbb{F}[\beta, \gamma]$ the multisets of linear factors $\{(f(i), i)\}$ and $\{(f(i), \sigma(i))\}$ coincide, i.e. $\{(i, f(i))\} = \{(\sigma(i), f(i))\}$, which means $f(i) = f(\sigma(i))$ exactly (as σ is a permutation). If they are unequal as polynomials, Schwartz–Zippel shows the identity holds for random (β, γ) with probability $\le 3n/|\mathbb{F}|$. □

Example 12.6. Three cells with values $f = (3, 9, 3)$ and the copy constraint "cell 1 equals cell 3", i.e. $\sigma = (1\ 3)$ $(\sigma(1) = 3, \sigma(3) = 1, \sigma(2) = 2)$. Take $\beta = 2, \gamma = 1$. LHS $= (3 + 2 + 1)(9 + 4 + 1)(3+6+1) = 6 \cdot 14 \cdot 10 = 840$; RHS $= (3+6+1)(9+4+1)(3+2+1) = 10 \cdot 14 \cdot 6 = 840$. Equal. If the prover cheats with $f = (3, 9, 4)$: LHS $= 6 \cdot 14 \cdot 11 = 924$, RHS $= (3+6+1)(14)(4+2+1) = 980 \neq 924$ — detected.

To verify a product over H with a low-degree identity, the prover defines the accumulator $Z(\omega^0) = 1$ and

$$Z(\omega^{i+1}) = Z(\omega^i) \prod_{\mathsf{col} \in \{a,b,c\}} \frac{w_{\mathsf{col},i} + \beta\, \mathrm{id}_{\mathsf{col}}(\omega^i) + \gamma}{w_{\mathsf{col},i} + \beta\, S_{\sigma,\mathsf{col}}(\omega^i) + \gamma},$$

where $w_{\mathsf{col},i}$ is the wire value in column col of row i, the *identity labels* are $\mathrm{id}_a(\omega^i) = \omega^i$, $\mathrm{id}_b(\omega^i) = k_1\omega^i$, $\mathrm{id}_c(\omega^i) = k_2\omega^i$ with k_1, k_2 chosen so that H, k_1H, k_2H are disjoint cosets (they give each of the $3n$ cells a distinct label), and $S_{\sigma,\mathsf{col}}$ are the (public, circuit-specific) permutation polynomials that evaluate to the label of the cell σ maps to. The prover requires $Z(\omega^n) = Z(\omega^0) = 1$ and checks the recurrence as polynomial identity on H.

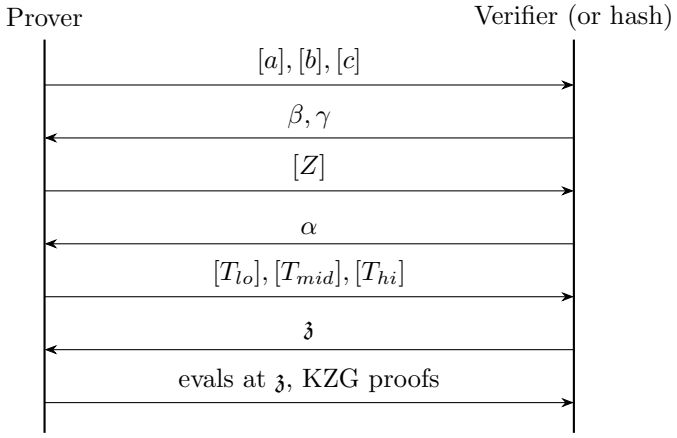
12.3.3 Putting it together

Algorithm 12.2: PLONK prover (simplified)

1. **Round 1.** Commit (KZG) to $a(X), b(X), c(X)$ (blinded).
2. **Round 2.** Challenges β, γ; commit to the accumulator $Z(X)$.
3. **Round 3.** Challenge α. Form the *quotient polynomial* $T(X) = \frac{G(X) + \alpha\,(\text{perm. constraint}) + \alpha^2\,(Z(X)-1)L_0(X)}{Z_H(X)}$. Split T into pieces of degree $< n$ and commit.
4. **Round 4.** Challenge $\mathfrak{z}$ (evaluation point). Send the evaluations $\bar{a} = a(\mathfrak{z}), \bar{b}, \bar{c}, \bar{s}_{\sigma 1}, \bar{s}_{\sigma 2}, \overline{Z\omega} = Z(\mathfrak{z}\omega)$.
5. **Round 5.** Challenge v. Send two KZG opening proofs (batched with powers of v) for the linearisation polynomial at $\mathfrak{z}$ and for Z at $\mathfrak{z}\omega$.

Make non-interactive by Fiat–Shamir.

The verifier recomputes $T(\mathfrak{z})$ from the evaluations, checks the quotient identity at the single point $\mathfrak{z}$ and verifies the two KZG openings with one pairing equation (batched). By Schwartz–Zippel a polynomial identity checked at a random $\mathfrak{z} \notin H$ implies the identity holds, except with probability $O(n/|\mathbb{F}|)$. Proof size: 9 group elements + 6 field elements ($\approx$ 600 bytes); verifier: 2 pairings + $O(\ell)$ group ops.



Example 12.7 (PLONK gate and permutation on Example 10.3)**.** The rows (a, b, c) are $(3, 3, 9)$, $(9, 3, 27)$, $(27, 3, 30)$, $(30, 0, 35)$; take $n = 4$ and let $H = \{1, \omega, \omega^2, \omega^3\}$ with ω of order 4. The gate polynomial at ω^2 (row 3, where $q_L = q_R = 1$, $q_O = -1$) gives $27 + 3 - 30 = 0$. ✓

The copy constraints partition the twelve cells into cycles:

- $\{a_1, b_1, b_2, b_3\}$ (all equal to $x = 3$),
- $\{c_1, a_2\}$ (value 9), $\{c_2, a_3\}$ (value 27), $\{c_3, a_4\}$ (value 30).

The grand product equals 1 iff every cycle carries a single value. Changing b_2 to 4 would break the gate ($9 \cdot 4 \neq 27$) and the cycle of x.

Extensions. *Custom gates* (e.g. a degree-5 gate for Poseidon S-boxes), *lookup arguments* (plookup, Halo2 lookups, Lasso/LogUp: prove $f \subseteq T$ for a table T, replacing bit decompositions for range checks and bitwise ops), *recursion* (Halo: accumulate IPA openings so a proof can verify a proof).

12.4 STARKs

STARKs (Ben-Sasson, Bentov, Horesh, Riabzev 2018; "Scalable Transparent ARguments of Knowledge") eliminate trusted setup and pairings: only hash functions (collision-resistant / random oracle), hence plausibly post-quantum.

Algorithm 12.3: STARK for an AIR

Input: AIR over trace length $T = 2^k$, trace polynomials $f_1..f_w$ over the subgroup $G = \langle g \rangle$, $|G| = T$.

1. **Trace commitment.** Prover interpolates each column, evaluates on a larger *LDE domain* $D \supset G$ ($|D| = \rho^{-1}T$ — blow-up e.g. $\times 4$ or $\times 8$), Merkle-commits.
2. **Constraint composition.** Verifier sends random α_j. Prover forms the

composition polynomial

$$C(X) = \sum_j \alpha_j \frac{P_j\big(f_1(X), .., f_w(X), f_1(gX), .., f_w(gX)\big)}{Z_j(X)}$$

where Z_j is the vanishing polynomial of the set of points where the j-th constraint must hold. If the trace is valid, each quotient is a polynomial, hence C has low degree; otherwise, some quotient has a pole and C is far from low degree. Commit to C on D.

3. **OOD/DEEP sampling** (optional) fix consistency between C and f_i at a random point $\mathfrak{z}$ outside D.
4. **FRI.** Run FRI on C (and the DEEP quotient) to prove low degree (Algorithm 11.2).
5. **Queries.** Verifier checks, at random positions $x \in D$, that $C(x)$ equals the constraint combination computed from the opened trace values $f_i(x), f_i(gx)$ (Merkle openings).

Example 12.8 (Fibonacci STARK)**.** Trace $s = (1, 1, 2, 3, 5, 8, 13, 21)$, $T = 8$. Let g generate a multiplicative subgroup G of order 8 of the field and let f be the degree-≤ 7 polynomial with $f(g^i) = s_i$. The transition polynomial $R(X) = f(g^2X) - f(gX) - f(X)$ must vanish at $g^0, \ldots, g^5$, so

$$Q_{\mathrm{tr}}(X) = \frac{R(X)}{\prod_{i=0}^{5}(X - g^i)}$$

is a polynomial of degree $\leq 7 - 6 = 1$. The boundary quotients $\frac{f(X)-1}{X-g^0}$, $\frac{f(X)-1}{X-g^1}$, $\frac{f(X)-21}{X-g^7}$ have degree ≤ 6. The prover Merkle-commits to f on an evaluation domain of size 32 (blow-up 4), commits to the random combination of the quotients, and runs FRI. If the prover cheats by changing one entry (say $s_4 = 6$), then R is nonzero at g^2 and g^3 (because s_4 occurs in two transitions), so the corresponding quotient has a pole there and is far from a low-degree polynomial; FRI rejects with high probability.

Properties. Transparent (public randomness only); proof size $O(\log^2 T)$ (50–200 KB in practice); prover quasi-linear $O(T \log T)$ (FFTs and hashing); verifier $O(\log^2 T)$; plausibly post-quantum. ZK is obtained by randomising the trace (adding random rows and masking polynomials) so that the $O(\lambda)$ opened values reveal nothing.

12.5 Comparison

	Groth16	PLONK (KZG)	Bulletproofs	STARK	Spartan/Libra
Setup	circuit-specific	universal	none	none	none / universal
Proof size	≈ 128 B	≈ 0.5 KB	$O(\log n)$ (1–2 KB)	$\approx$ 50–200 KB	$O(\log^2)$
Verifier	3 pairings	2 pairings	$O(n)$	polylog	polylog / $\sqrt{n}$
Prover	$O(n \log n)$ FFT+MSM	$O(n \log n)$	$O(n)$ MSM	$O(n \log n)$ hashes	$O(n)$
Assumption	GGM/AGM (pairings)	AGM, q-SDH	DL	Hash (PQ)	DL / hash

12.6 Recursion and folding

A SNARK verifier is itself a small circuit, so a prover can prove "I verified the previous proof". This *proof-carrying data* / recursion approach yields incrementally verifiable computation (IVC): constant-size proof for an unbounded computation. Folding schemes (Nova, SuperNova, HyperNova, ProtoStar) avoid verifying a full SNARK at each step: two R1CS instances (u_1, w_1), (u_2, w_2) are linearly combined, $u = u_1 + ru_2$ with random r, into one relaxed instance; the cross-term E is committed; only the final folded instance is proven.

Exercises

Exercise 12.9. Re-derive the Groth16 verification equation starting from the requirement that the verifier tests $AB = \alpha\beta + \ldots$; explain the role of γ and δ (they separate public from private parts and prevent mix-and-match attacks).

Exercise 12.10. Recompute Example 12.3 with a different τ and randomisers; check the pairing equation. What happens if you change $a_5 = 30$ to 31 (and keep h as computed from the honest witness)?

Exercise 12.11. Prove Lemma 12.5 carefully, including the soundness error $\leq 3n/|\mathbb{F}|$.

Exercise 12.12. Compute, for a STARK over a field with $\lambda = 100$ bits of security, blow-up factor 8 (rate 1/8) and proximity parameter $\delta \approx 0.4$, the number of FRI queries needed (use $s \approx \lambda / \log_2 \frac{1}{1-\delta}$).

Exercise 12.13. Show that Groth16's (A, B) is uniform (zero-knowledge) when r, s are uniform, and explain what leaks if $r = s = 0$.

Chapter 13

Complexity-Theoretic Foundations and Limits

13.1 Classes of zero-knowledge languages

Let $\mathsf{PZK}, \mathsf{SZK}, \mathsf{CZK}$ be the classes of languages with perfect, statistical, and computational zero-knowledge *proof* systems (against malicious verifiers); HVSZK restricts to honest verifiers.

Theorem 13.1 (Landscape)**.** *1.* $\mathsf{BPP} \subseteq \mathsf{PZK} \subseteq \mathsf{SZK} \subseteq \mathsf{CZK} \subseteq \mathsf{IP} = \mathsf{PSPACE}$.

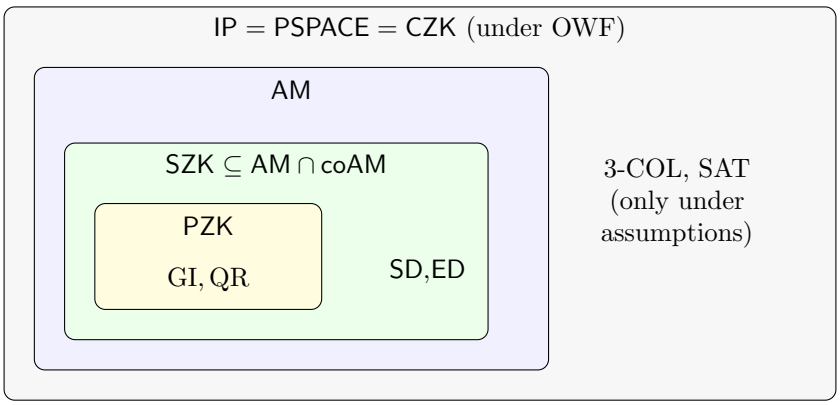
2. $\mathrm{GI}, \mathrm{GNI}, \mathrm{QR}, \mathrm{QNR} \in \mathsf{PZK}/\mathsf{SZK}$ *(Sections 5.1–5.2, GMW, GMR).*

3. (Goldreich–Micali–Wigderson; Impagliazzo–Yung; Ben-Or et al.) If one-way functions exist then $\mathsf{CZK} = \mathsf{IP} = \mathsf{PSPACE}$*. Conversely, if* CZK *contains a language outside* BPP *then (Ostrovsky–Wigderson) a weak form of one-way function exists.*

4. $\mathsf{HVSZK} = \mathsf{SZK}$ *(Goldreich–Sahai–Vadhan).*

5. $\mathsf{SZK} \subseteq \mathsf{AM} \cap \mathsf{coAM}$ *(Fortnow; Aiello–Håstad). Hence an* NP*-complete language is not in* SZK *unless the polynomial hierarchy collapses.*

6. SZK *is closed under complement (Okamoto) and has the complete problems Statistical Difference and Entropy Difference (Sahai–Vadhan).*



Theorem 13.2 ($\mathsf{SZK} \subseteq \mathsf{AM} \cap \mathsf{coAM}$, sketch)**.** $\mathsf{SZK} \subseteq \mathsf{AM}$*: it is enough to put Entropy Difference in* AM*. Statistical zero-knowledge implies that the simulator's output on* $x \in L$ *has some specific entropy relation with the output on* $x \notin L$*. The* AM *verifier uses the Goldwasser–Sipser set lower bound protocol to certify that the entropy of the simulator's output distribution is above a threshold. Since* SZK *is closed under complement,* $\mathsf{SZK} \subseteq \mathsf{coAM}$ *as well.*

Interpretation: statistical ZK proofs are for problems of "intermediate" complexity (factoring-like, lattice-like), not for NP-hard problems. Statistical ZK *arguments* for NP do exist (from collision-resistant hash functions: Brassard–Chaum–Crépeau; Naor–Ostrovsky–Venkatesan–Yung), since computationally bounded provers cannot exploit binding.

13.2 Sahai–Vadhan problems: the SD problem

Definition 13.3 (Statistical Difference, $\mathrm{SD}_{\alpha,\beta}$)**.** Input: circuits C_0, C_1 sampling distributions D_0, D_1. YES if $\mathsf{SD}(D_0, D_1) \geq \beta$; NO if $\mathsf{SD}(D_0, D_1) \leq \alpha$, with $\alpha^2 < \beta$ (e.g. $\alpha = 1/3$, $\beta = 2/3$); the condition $\alpha^2 < \beta$ is exactly what Sahai–Vadhan's *polarization lemma* needs in order to amplify the gap.

Theorem 13.4 (Sahai–Vadhan)**.** SD *is* SZK*-complete; for* GI*,* GNI*,* QR *this is seen by taking* D_b *= random isomorphic copies of* G_b*.*

Example 13.5 (GNI $\leq$ SD)**.** Let D_b be the distribution of $\sigma(G_b)$, $\sigma \xleftarrow{\$} S_n$. If $G_0 \cong G_1$ then $D_0 = D_1$ ($\mathsf{SD} = 0$); if not then their supports are disjoint ($\mathsf{SD} = 1$). So $(G_0, G_1) \in$ GNI iff $\mathsf{SD}(D_0, D_1) = 1$. Conversely every SD instance has a GNI-like ZK proof: the verifier flips a coin b and sends a sample from D_b; the prover guesses b; its success probability is $\frac{1}{2} + \frac{1}{2}\mathsf{SD}(D_0, D_1)$. This is exactly the protocol of Section 3.2, generalised.

13.3 Impossibility results

Theorem 13.6 (Goldreich–Oren 1994)**.** *If* L *has a ZK proof system (with auxiliary input) then the verifier must be randomized and interactive: non-interactive ZK in the plain model is possible only for* $L \in \mathsf{BPP}$*.*

Proof. In a one-message protocol the verifier's decision is a function of (x, π) and its coins. Run $\mathsf{Sim}(x)$ (the simulator for the honest verifier): by ZK, on $x \in L$ the simulator outputs views indistinguishable from accepting ones; on $x \notin L$, soundness says no π is accepted except w.p. s, so the simulator's output (which can be checked by running V) is rejected w.h.p. Hence $L \in \mathsf{BPP}$ is decided by "simulate and verify". □

Theorem 13.7 (Goldreich–Krawczyk 1996)**.** *If a language has a 3-round (auxiliary-input) black-box ZK proof system, then* $L \in \mathsf{BPP}$*. If it has a constant-round public-coin black-box ZK proof system, then* $L \in \mathsf{BPP}$*.*

Intuition

A black-box simulator can only rewind. With a verifier that answers with $c = \mathrm{PRF}(a)$, a pseudorandom function of the first message, rewinding gains nothing: the simulator must predict the challenge. A simulator that produces a convincing transcript for a (pseudo)random challenge must, by an extractor-like argument, have been able to answer *two* challenges, whence it decides L.

Theorem 13.8 (Barak 2001)**.** *Non-black-box simulation circumvents these: there is a constant-round public-coin ZK argument for* NP *with negligible soundness error. The simulator uses the* code *of* V^**: the protocol includes a branch that is provable only by knowing a short commitment to a program that predicts the verifier's next message; the simulator commits to the verifier's own code.*

Theorem 13.9 (Gentry–Wichs 2011)**.** *No SNARG for* NP *has a security reduction to a falsifiable assumption in a black-box way; knowledge assumptions are unavoidable for succinct non-interactive* arguments *(hence SNARK's extractability assumptions and ROM-based STARKs).*

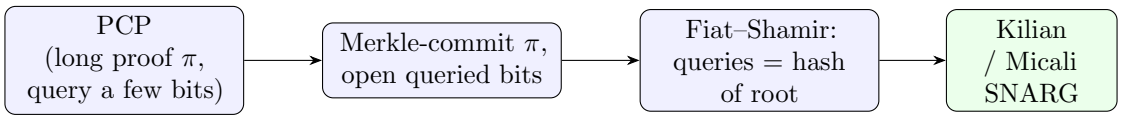
Theorem 13.10 (Composition: parallel and concurrent)**.** *The sequential composition theorem (Thm. 4.4) holds, but parallel composition of ZK may fail (Goldreich–Krawczyk construct, under standard assumptions, a ZK proof system whose two-fold parallel repetition is not ZK). Concurrent ZK requires* $\tilde{\Omega}(\log \lambda)$ *rounds for black-box simulation (Canetti–Kilian–Petrank–Rosen) and is achievable (Richardson–Kilian, Prabhakaran–Rosen–Sahai).*

13.4 PCPs, IOPs and the origins of succinctness

The sum-check protocol (Ch. 9) led to the PCP theorem: an NP statement has a proof that can be verified by reading only $O(1)$ bits of it.

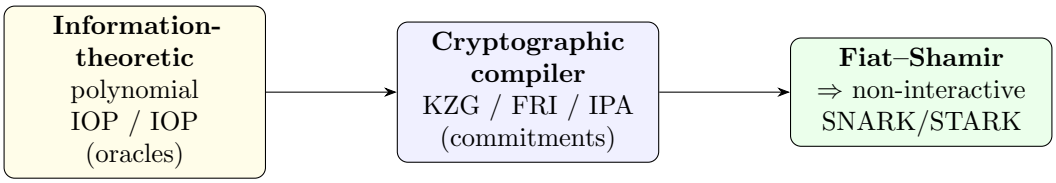
Definition 13.11 (PCP)**.** $L \in \mathsf{PCP}[r(n), q(n)]$ if there is a polynomial-time verifier that tosses $r(n)$ coins, queries $q(n)$ positions of an oracle π, accepts $x \in L$ with some π with probability 1 and rejects $x \notin L$ with probability $\geq 1/2$ for all π.

Theorem 13.12 (ALMSS 1992)**.** $\mathsf{NP} = \mathsf{PCP}[O(\log n), O(1)]$.



Theorem 13.13 (Kilian 1992; Micali 2000)**.** *PCP + Merkle commitments (collision-resistant hash) gives a 4-message succinct argument for* NP *(Kilian), which Fiat–Shamir makes non-interactive in the random oracle model (Micali's computationally sound proofs).*

Interactive oracle proofs (IOPs) (Ben-Sasson–Chiesa–Spooner; Reingold–Rothblum–Rothblum) generalise PCPs and interactive proofs: in each round the prover sends an oracle (a long string) and the verifier sends randomness; the verifier queries only a few symbols. STARKs, Aurora, Fractal, Plonky2, Ligero are IOP + hash-based compilations; Marlin/PLONK are *polynomial IOPs* compiled using a polynomial commitment scheme.



This modular "IOP → PCS → FS" pipeline explains the modern landscape: choosing a different PCS changes setup, proof size and assumptions (see the comparison table in Section 12.5) without touching the arithmetisation.

13.5 Quantum considerations

- *Soundness against quantum provers.* Rewinding is problematic for quantum adversaries (no-cloning): classical proofs of knowledge via rewinding do not carry over (Unruh's quantum rewinding, quantum PoK).

- *Fiat–Shamir in the quantum random oracle model* (QROM): Don et al., Liu–Zhandry give tight-ish reductions for some protocols; Dilithium/Falcon-like signatures (Fiat–Shamir with aborts) use lattice Σ-protocols.
- *Post-quantum ZK:* lattice-based (Lyubashevsky; LaBRADOR), hash-based (STARKs, Ligero++, Brakedown), MPC-in-the-head (Ishai–Kushilevitz–Ostrovsky–Sahai: simulate an MPC protocol among n virtual parties, open t views; used in Picnic and Banquet).
- *Quantum advantage:* $\mathsf{MIP}^* = \mathsf{RE}$ (Ji–Natarajan–Vidick–Wright–Yuen 2020) shows entangled provers can prove astonishingly hard statements; ZK analogues exist.

Exercises

Exercise 13.14. Prove that $\mathsf{PZK} \subseteq \mathsf{SZK} \subseteq \mathsf{CZK}$ and that $\mathsf{BPP} \subseteq \mathsf{PZK}$.

Exercise 13.15. Show that GNI $\leq$ SD and QNR $\leq$ SD by exhibiting the sampling circuits.

Exercise 13.16. Give the details of the argument that a one-message ZK proof system in the plain model implies $L \in \mathsf{BPP}$ (Goldreich–Oren). Where does the argument fail in the CRS model?

Exercise 13.17. State precisely why the Kilian PCP-to-argument transformation needs a *commitment with local opening* and construct one from a Merkle tree; analyse the soundness error when the verifier makes q queries.

Chapter 14

Applications, Practical Considerations and Further Reading

14.1 Applications

Identification and authentication. Schnorr/FFS identification; passwordless authentication (OPAQUE with ZK password proofs); proving age or residency from a government credential without revealing the credential (ISO 18013-5 mDL + zk).

Signatures. Schnorr/EdDSA, BLS (not FS-based), Dilithium; ring signatures and group signatures (CDS OR-proofs, linkable ring signatures in Monero); blind signatures.

Anonymous credentials. Camenisch–Lysyanskaya, U-Prove, BBS+ signatures with selective disclosure: the holder proves in ZK possession of a signed credential with attribute predicates.

Private payments. Zerocash/Zcash: a note commitment $\mathsf{cm} = \mathsf{Com}(v, \mathsf{pk}; \rho)$ is inserted into a Merkle tree; spending proves (in ZK) knowledge of the opening, Merkle membership, a correctly derived nullifier nf, and balance $\sum v_{in} = \sum v_{out}$. Confidential transactions in Monero use Pedersen commitments and Bulletproofs for range.

Verifiable computation and blockchain scaling. zk-rollups (zkSync, StarkNet, Polygon zkEVM, Scroll, Linea): a prover executes thousands of transactions off-chain and posts a SNARK/STARK of the state transition; L1 verifies one proof. zkVMs (RISC Zero, SP1, Jolt) prove arbitrary RISC-V execution.

Verifiable machine learning, data and databases. zkML proofs of inference; proofs of SQL queries; proofs of solvency of an exchange (Merkle sum tree + ZK range proofs).

Secure multi-party computation. GMW compiler: each party attaches a ZK proof of correct behaviour. Also MPC-in-the-head signatures and ZK proofs.

Voting and auctions. Verifiable shuffles (Neff/Groth), proofs of correct ElGamal decryption (Chaum–Pedersen), sealed-bid auctions.

14.2 Choosing a proof system

Requirement	Suggested tool
Prove discrete-log relations, few statements	Σ-protocols + Fiat–Shamir
Range proofs, no trusted setup, small statement	Bulletproofs
Smallest proof, on-chain verification, fixed circuit	Groth16
Evolving circuits, universal setup	PLONK/Halo2 with KZG
No trusted setup, large computation, post-quantum	STARKs (FRI)
Proving long iterative computations	Folding (Nova) / recursion
Maximum prover speed, structured computations	sum-check based (Spartan, Jolt, Lasso)

14.3 Implementation pitfalls (a checklist)

1. Hash *everything* in Fiat–Shamir (statement, public parameters, all prior messages); use domain separation.
2. Check group membership/subgroup of every received point (small-subgroup and invalid-curve attacks); reject the identity where inappropriate.
3. Use constant-time scalar arithmetic, and generate nonces either deterministically or from a CSPRNG.
4. Under-constrained circuits are the dominant real-world bug: every witness variable must be pinned by constraints (range checks, boolean checks, non-zero checks via inverse witnesses $x \cdot x^{-1} = 1$). Audit with formal tools (Circomspect, Picus, ZKAP).
5. Trusted setup hygiene: verify transcripts of the ceremony; sample τ in an MPC; use a beacon.
6. Soundness bits: with $|\mathbb{F}| \approx 2^{64}$, Schwartz–Zippel error $d/|\mathbb{F}|$ is not negligible; sample challenges from an extension field.
7. Malleability and replay: bind proofs to application context (contract address, nonce) by including them as public inputs.

14.4 Where to go from here

- Foundational texts: Goldreich, *Foundations of Cryptography Vol. 1* (Ch. 4); Goldwasser–Micali–Rackoff; Arora–Barak Ch. 8–11; Vadhan's thesis on SZK.
- Σ-protocols and applications: Damgård's lecture notes "On Σ-protocols"; Boneh–Shoup, *A Graduate Course in Applied Cryptography* (Ch. 19, 20).
- SNARKs/STARKs: Thaler, *Proofs, Arguments, and Zero-Knowledge*; Ben-Sasson et al. STARK papers; Justin Thaler's survey; Vitalik Buterin's QAP/PLONK blog posts; Groth16 paper; the PLONK paper; ZK Hack lectures.

- Open problems: tight Fiat–Shamir security in the standard model; transparent SNARKs with $O(1)$ verification; post-quantum SNARKs with short proofs; ZK with optimal prover overhead (linear time, small constant); concurrent security of recursion; formal verification of circuits.

Capstone exercises

Exercise 14.1. Design a Schnorr-based protocol that lets a prover show that a Pedersen commitment $C = g^v h^r$ hides a value $v \in \{3, 5, 7\}$. Give completeness, special soundness and SHVZK, and run it with the toy group.

Exercise 14.2. Implement (in Python, using the toy group and a prime field of 17 elements) Schnorr, Chaum–Pedersen, the OR-proof, sum-check (Example 9.4), the KZG "in the exponent" check and one FRI fold; test the soundness of each by simulating cheating provers.

Exercise 14.3. Write an R1CS for the statement "I know a preimage s with $s^2 + 3s + 2 = 0$ over $\mathbb{F}_{101}$", convert to QAP (with points $1, 2, \ldots$), and produce the Groth16 "exponent-level" proof with your own toxic waste.

Exercise 14.4. Prove that Protocol 7.5 (OR-proof) is not zero-knowledge against a malicious verifier unless the challenge is committed in advance, by exhibiting the verifier that biases challenges.

Exercise 14.5. Survey: pick a deployed system (Zcash Sapling/Orchard, a zk-rollup, or a zkVM) and identify: the arithmetisation, the PCS, the setup, the Fiat–Shamir hashes, and the assumptions; compare with the comparison table in Section 12.5.

Appendix A

Notation and Toy-Parameter Reference

A.1 Notation

Symbol	Meaning
λ, negl, poly	security parameter; negligible / polynomial functions
$x \xleftarrow{\$} S$	sample x uniformly from the finite set S
$\langle \mathsf{P}(w), \mathsf{V} \rangle (x)$	output of the verifier in the interaction on common input x, prover witness w
$\mathsf{View}^{\mathsf{P}(w)}_{\mathsf{V}^*(z)}(x)$	view of V^* (random tape and received messages), auxiliary input z
Sim, Ext	simulator, knowledge extractor
$\equiv$, $\approx_s$, $\approx_c$	perfectly / statistically / computationally indistinguishable
R, L_R	NP relation and its language
$\mathbb{G}, g, q$	cyclic group, generator, prime order
$[a]_1, [a]_2, [a]_T$	$g_1^a, g_2^a, e(g_1, g_2)^a$
$\tilde{f}$	multilinear extension of $f : \{0,1\}^v \to \mathbb{F}$
u_i, v_i, w_i, t, h	QAP polynomials, vanishing polynomial, quotient
$\xrightarrow{m}$, $\xleftarrow{m}$	message m sent prover→verifier / verifier→prover

A.2 Toy group and fields used in the examples

- $\mathbb{G} = \langle 2 \rangle \subset \mathbb{Z}_{23}^*$, order $q = 11$, $g = 2$, $h = 3 = 2^8$. Discrete-log table: 2^k for $k = 0..10$ is $1, 2, 4, 8, 16, 9, 18, 13, 3, 6, 12$.
- Inverses mod 11: $2^{-1} = 6$, $3^{-1} = 4$, $4^{-1} = 3$, $5^{-1} = 9$, $6^{-1} = 2$, $7^{-1} = 8$, $8^{-1} = 7$, $9^{-1} = 5$, $10^{-1} = 10$.
- Inverses mod 23 used: $18^{-1} = 9$, $4^{-1} = 6$, $13^{-1} = 16$.
- $N = 77 = 7 \cdot 11$: $23^{-1} = 67$; $10^2 = 23$; secret roots $s = 10, 4$ with $v = 23, 16$.
- Prime fields: $\mathbb{F}_{17}$ (FRI, Schwartz–Zippel), $\mathbb{F}_{101}$ (Groth16 example), $\mathbb{Z}_{11}$ (IPA, KZG exponents).

Appendix B

Hints and Answers to Selected Exercises

Exercise 3.7 If V is deterministic, a prover's best strategy against it is a fixed sequence of answers; the witness is the full list of prover messages, and the NP verifier replays V on it.

Exercise 4.11 / Theorem 7.2 Mimic the proof of the extractor theorem with $N = 2$: sample until accept, rewind once and sample the other challenge; expected cost $O(1/\varepsilon)$.

Exercise 6.9 Need $(1-1/1000)^N \le 2^{-80}$, i.e. $N \ge 80\ln 2/(-\ln 0.999) \approx 5.54 \times 10^4$ sequential rounds of Protocol 6.1 — which illustrates why Blum's protocol (error $1/2$ per round, 80 rounds) is far more efficient.

Exercise 7.15 (nonce reuse) Two Schnorr transcripts (t, c, s), (t', c', s') with the same r have $t = t'$ and $s - s' = (c - c')x$, so $x = (s - s')(c - c')^{-1}$.

Exercise 8.11 (weak FS) The attacker picks t, computes $c = H(t)$, picks any s and sets $y = (g^s t^{-1})^{1/c}$; then (t, s) is accepted for the statement y, whose discrete logarithm the attacker does not know.

Exercise 8.13 (forking loss) A forger with running time T_F and success probability ε gives a DL solver with time $\approx 2T_F$ and success $\approx \varepsilon^2/Q$. If solving DL needs 2^{128} time at success probability ≈ 1, this only yields $T_F/\varepsilon^2 \gtrsim 2^{127}/Q$; for $Q = 2^{60}$ and $\varepsilon \approx 1$ the reduction guarantees only about 67 bits.

Exercise 9.11 (sum-check) $g = x_1x_2 + 3x_2$, $H = 0 + 3 + 0 + 4 = 7$. $g_1(X) = X + 3$ with $g_1(0) + g_1(1) = 7$; $r_1 = 4$, $g_1(4) = 7$. $g_2(Y) = g(4, Y) = 7Y$, $g_2(0) + g_2(1) = 7$; $r_2 = 7$, $g_2(7) = 49 \equiv 5$; final $g(4, 7) = 28 + 21 = 49 \equiv 5$. A prover claiming $H' = 8$ must send $g_1^* = g_1 + d$ with $d(0) + d(1) = 1$; if $d(X) = aX + b$ with $a \neq 0$ it passes round 1 only when $r_1 = -b/a$, probability $1/11$.

Exercise 10.10 For $y = x^4$: $x \cdot x = v_1$, $v_1 \cdot v_1 = y$. Multiplexer: $b(1 - b) = 0$ is the constraint $b \cdot (1 - b) = 0$, and $z - c_0 = b \cdot (c_1 - c_0)$ is $b \cdot (c_1 - c_0) = z - c_0$.

Exercise 11.12 $f(4) = 64 + 32 + 12 + 4 = 112 \equiv 10$, $f(13) = f(-4) = -64 + 32 - 12 + 4 = -40 \equiv 11$, so $\frac{10+11}{2} = \frac{21}{2} \equiv 4 \cdot 9 = 36 \equiv 2$ ($2^{-1} = 9$) and $\frac{10-11}{2 \cdot 4} = \frac{-1}{8} \equiv -15 \equiv 2$ ($8^{-1} = 15$); $f^{(1)}(16) = 2 + 3 \cdot 2 = 8$. Directly: $5 \cdot 16 + 13 = 93 \equiv 8$. ✓

Exercise 12.12 With $\delta = 0.4$: $\log_2(1/0.6) \approx 0.737$, so $s \approx 100/0.737 \approx 136$ queries.

Bibliography

[1] S. Goldwasser, S. Micali, C. Rackoff. The knowledge complexity of interactive proof systems. *SIAM J. Comput.* 18(1):186–208, 1989 (STOC 1985).

[2] O. Goldreich, S. Micali, A. Wigderson. Proofs that yield nothing but their validity or all languages in NP have zero-knowledge proof systems. *J. ACM* 38(3):690–728, 1991.

[3] U. Feige, A. Fiat, A. Shamir. Zero-knowledge proofs of identity. *J. Cryptology* 1(2):77–94, 1988.

[4] A. Fiat, A. Shamir. How to prove yourself. CRYPTO 1986.

[5] C.-P. Schnorr. Efficient signature generation by smart cards. *J. Cryptology* 4(3):161–174, 1991.

[6] D. Chaum, T. Pedersen. Wallet databases with observers. CRYPTO 1992.

[7] R. Cramer, I. Damgård, B. Schoenmakers. Proofs of partial knowledge and simplified design of witness hiding protocols. CRYPTO 1994.

[8] T. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. CRYPTO 1991.

[9] D. Pointcheval, J. Stern. Security proofs for signature schemes. EUROCRYPT 1996.

[10] M. Bellare, G. Neven. Multi-signatures in the plain public-key model and a general forking lemma. CCS 2006.

[11] A. Shamir. IP = PSPACE. *J. ACM* 39(4):869–877, 1992.

[12] C. Lund, L. Fortnow, H. Karloff, N. Nisan. Algebraic methods for interactive proof systems. *J. ACM* 39(4):859–868, 1992.

[13] S. Goldwasser, Y. Kalai, G. Rothblum. Delegating computation: interactive proofs for muggles. *J. ACM* 62(4), 2015.

[14] J. Thaler. Time-optimal interactive proofs for circuit evaluation. CRYPTO 2013.

[15] R. Gennaro, C. Gentry, B. Parno, M. Raykova. Quadratic span programs and succinct NIZKs without PCPs. EUROCRYPT 2013.

[16] J. Groth. On the size of pairing-based non-interactive arguments. EUROCRYPT 2016.

[17] A. Gabizon, Z. Williamson, O. Ciobotaru. PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. ePrint 2019/953.

[18] A. Kate, G. Zaverucha, I. Goldberg. Constant-size commitments to polynomials and their applications. ASIACRYPT 2010.

[19] E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev. Fast Reed–Solomon interactive oracle proofs of proximity. ICALP 2018.

[20] E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev. Scalable zero knowledge with no trusted setup. CRYPTO 2019.

[21] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, G. Maxwell. Bulletproofs: short proofs for confidential transactions and more. IEEE S&P 2018.

[22] E. Ben-Sasson, A. Chiesa, N. Spooner. Interactive oracle proofs. TCC 2016.

[23] J. Kilian. A note on efficient zero-knowledge proofs and arguments. STOC 1992.

[24] S. Micali. Computationally sound proofs. *SIAM J. Comput.* 30(4), 2000.

[25] M. Blum, P. Feldman, S. Micali. Non-interactive zero-knowledge and its applications. STOC 1988.

[26] U. Feige, D. Lapidot, A. Shamir. Multiple non-interactive zero knowledge proofs under general assumptions. *SIAM J. Comput.* 29(1), 1999.

[27] J. Groth, R. Ostrovsky, A. Sahai. Perfect non-interactive zero knowledge for NP. EUROCRYPT 2006.

[28] J. Groth, A. Sahai. Efficient non-interactive proof systems for bilinear groups. EUROCRYPT 2008.

[29] M. Naor. Bit commitment using pseudorandomness. *J. Cryptology* 4(2), 1991.

[30] J. Håstad, R. Impagliazzo, L. Levin, M. Luby. A pseudorandom generator from any one-way function. *SIAM J. Comput.* 28(4), 1999.

[31] O. Goldreich, H. Krawczyk. On the composition of zero-knowledge proof systems. *SIAM J. Comput.* 25(1), 1996.

[32] O. Goldreich, Y. Oren. Definitions and properties of zero-knowledge proof systems. *J. Cryptology* 7(1), 1994.

[33] B. Barak. How to go beyond the black-box simulation barrier. FOCS 2001.

[34] M. Bellare, R. Impagliazzo, M. Naor. Does parallel repetition lower the error in computationally sound protocols? FOCS 1997.

[35] S. Goldwasser, Y. Kalai. On the (in)security of the Fiat–Shamir paradigm. FOCS 2003.

[36] R. Canetti, O. Goldreich, S. Halevi. The random oracle methodology, revisited. *J. ACM* 51(4), 2004.

[37] A. Sahai, S. Vadhan. A complete problem for statistical zero knowledge. *J. ACM* 50(2), 2003.

[38] C. Gentry, D. Wichs. Separating succinct non-interactive arguments from all falsifiable assumptions. STOC 2011.

[39] R. Canetti et al. Fiat–Shamir: from practice to theory. STOC 2019.

[40] C. Peikert, S. Shiehian. Noninteractive zero knowledge for NP from (plain) LWE. CRYPTO 2019.

[41] U. Maurer. Unifying zero-knowledge proofs of knowledge. AFRICACRYPT 2009.

[42] J. Thaler. *Proofs, Arguments, and Zero-Knowledge*. Foundations and Trends in Privacy and Security, 2022.

[43] O. Goldreich. *Foundations of Cryptography, Vol. 1: Basic Tools*. Cambridge University Press, 2001.

[44] D. Boneh, V. Shoup. *A Graduate Course in Applied Cryptography*. 2023.

[45] S. Arora, C. Lund, R. Motwani, M. Sudan, M. Szegedy. Proof verification and the hardness of approximation problems. *J. ACM* 45(3), 1998.

[46] N. Bitansky, R. Canetti, A. Chiesa, E. Tromer. From extractable collision resistance to succinct non-interactive arguments of knowledge. ITCS 2012.

[47] A. Kothapalli, S. Setty, I. Tzialla. Nova: recursive zero-knowledge arguments from folding schemes. CRYPTO 2022.